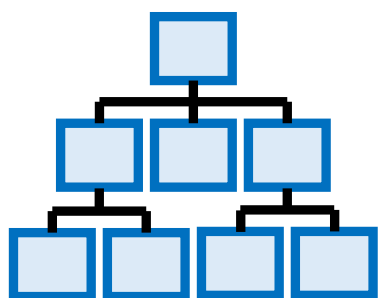


Space Efficient Mapping Structures for Learned Indexes

NEDB 2026

Anwasha Saha, Aneesh Raman, Ryan Marcus, Manos Athanassoulis

Indexes in Databases

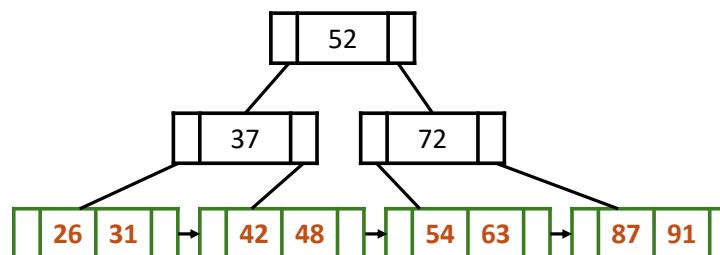


Organize data

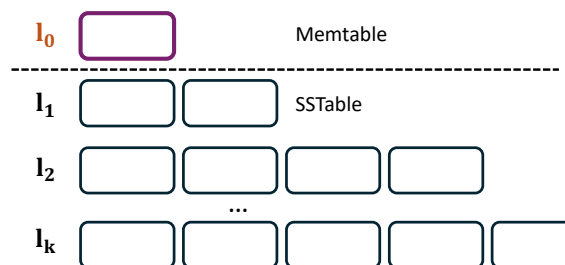


Efficient queries

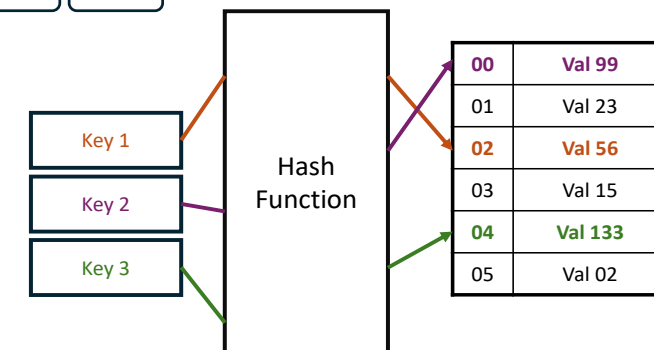
B⁺-Trees



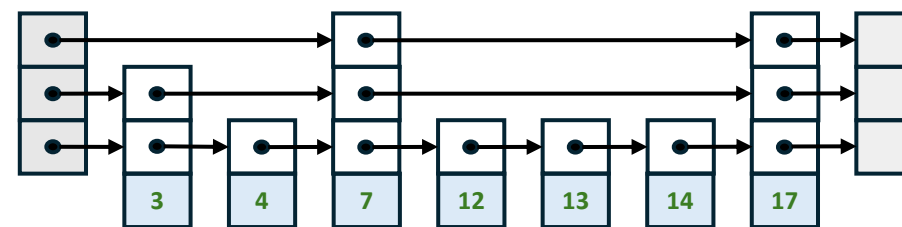
LSM Trees



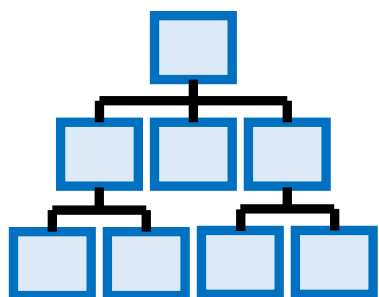
Hash maps



Skip Lists



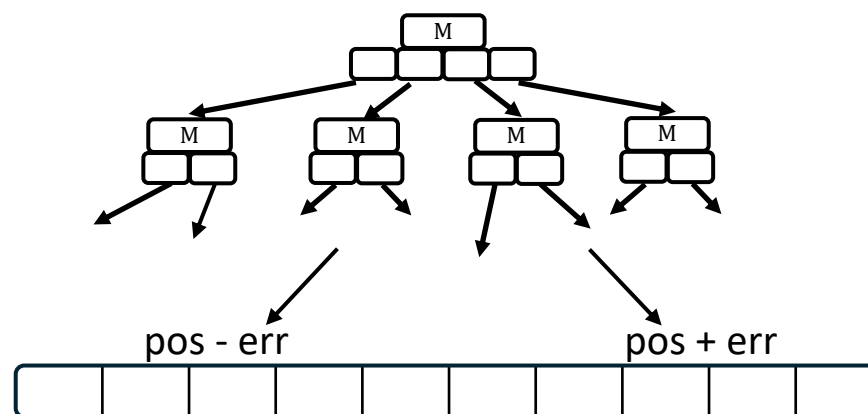
Indexes in Databases



Organize data

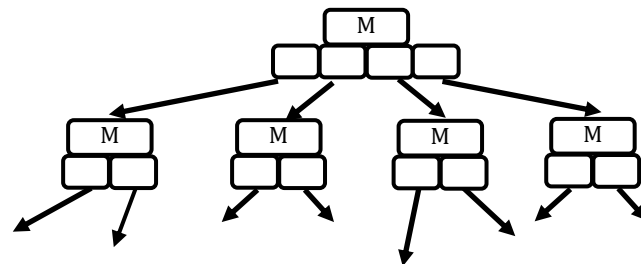


Efficient queries

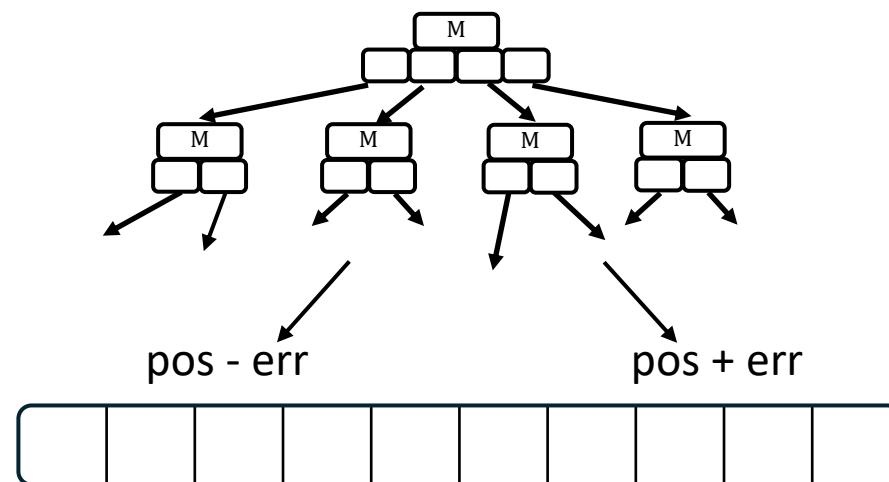


Learned Indexes

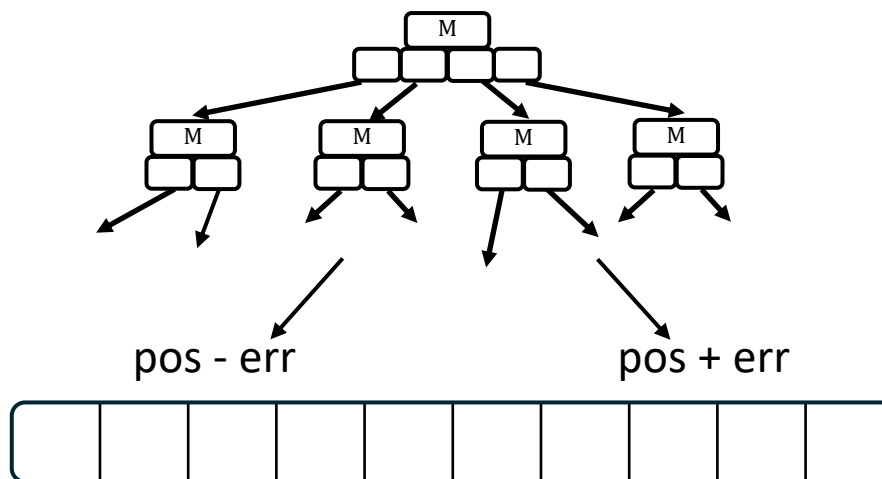
Learned Indexes



Learned Indexes

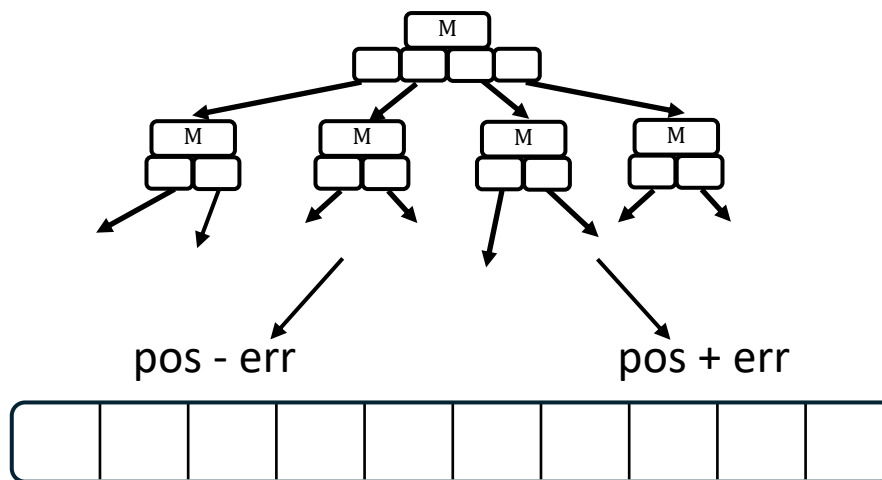


Learned Indexes



Lower Memory

Learned Indexes

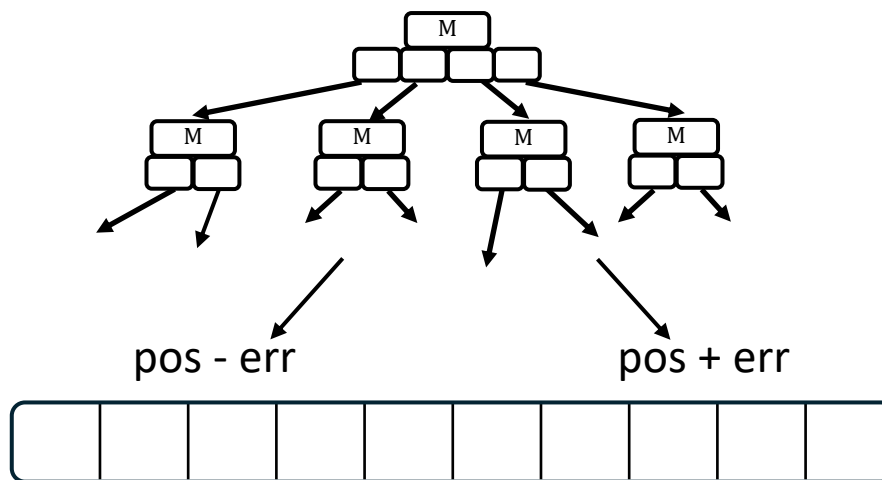


Lower Memory



Faster Access

Learned Indexes



Lower Memory



Faster Access

Data assumed to be sorted!

Prior Work

Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation										
Sorted Data										
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation										
Sorted Data	56									
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation										
Sorted Data	56									
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation	9									
Sorted Data	56									
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation	9									
Sorted Data	56	68								
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation	9 6									
Sorted Data	56	68								
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation	9	6	2	8	3	7	1	5	4	0
Sorted Data	56	68	98	114	265	345	430	578	810	923
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9



Prior Work

Permutation	9 6 2 8 3 7 1 5 4 0									
Sorted Data	56	68	98	114	265	345	430	578	810	923
Base Data	923	430	98	265	810	578	68	345	114	56
Offsets	0	1	2	3	4	5	6	7	8	9

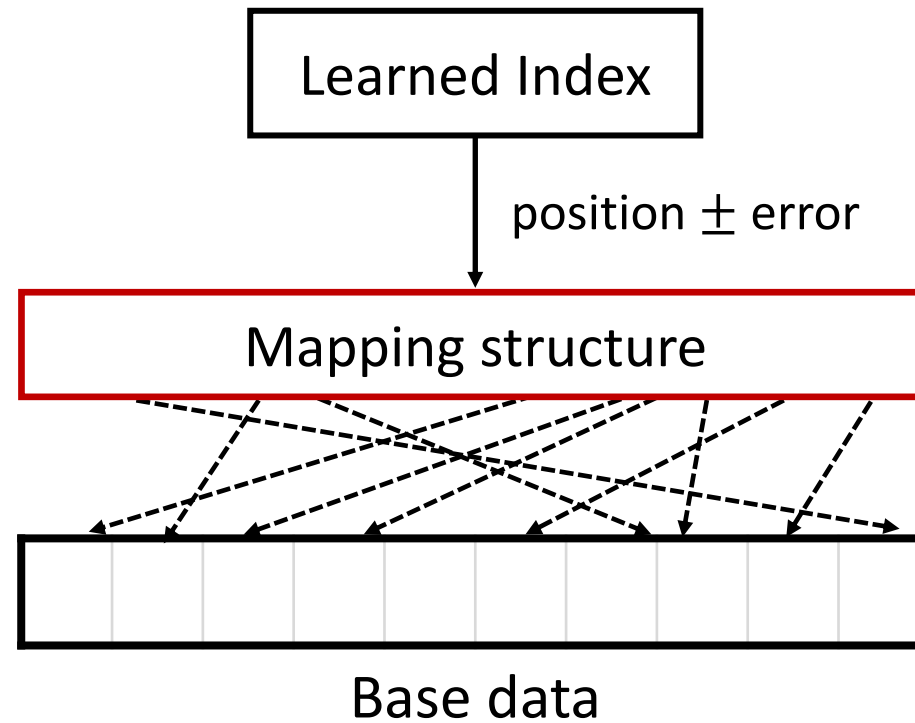
✗ Permutation Vector size increases with size of data



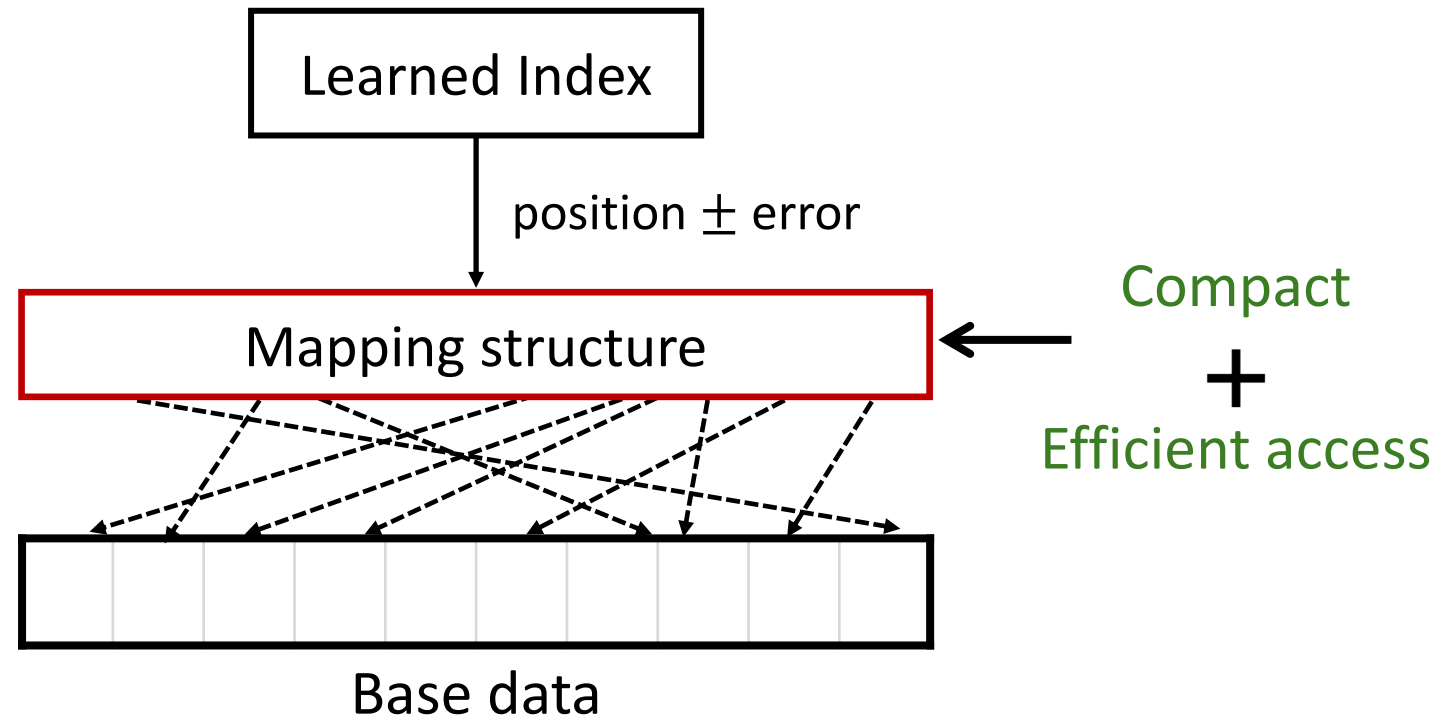
Permutation Mappings

Method	Description	Pros	Cons	Paper
LSI	Bit packed permutation vector	Fast	$O(n)$ growth	

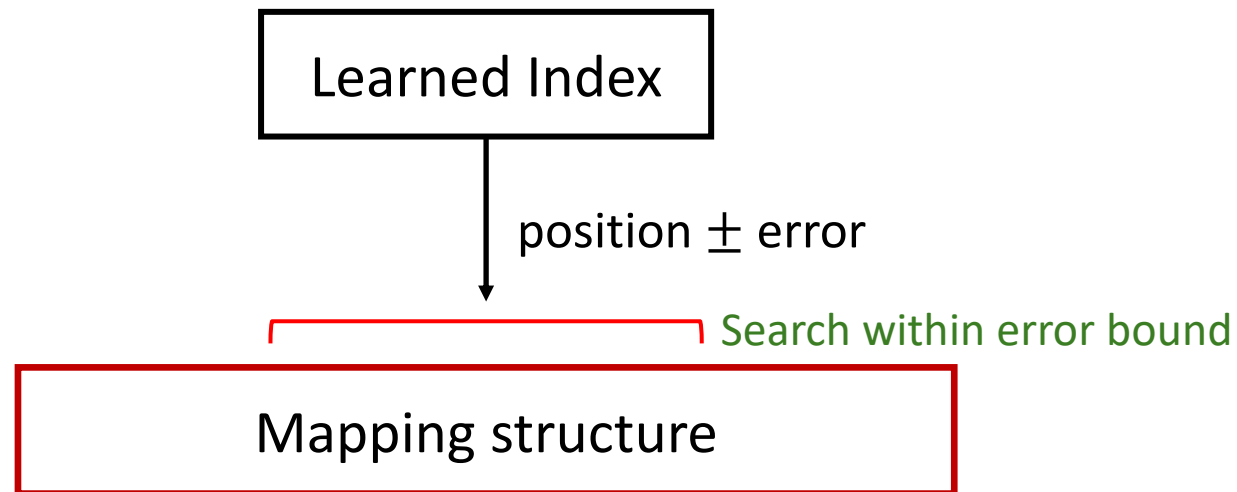
What do we Want?



What do we Want?

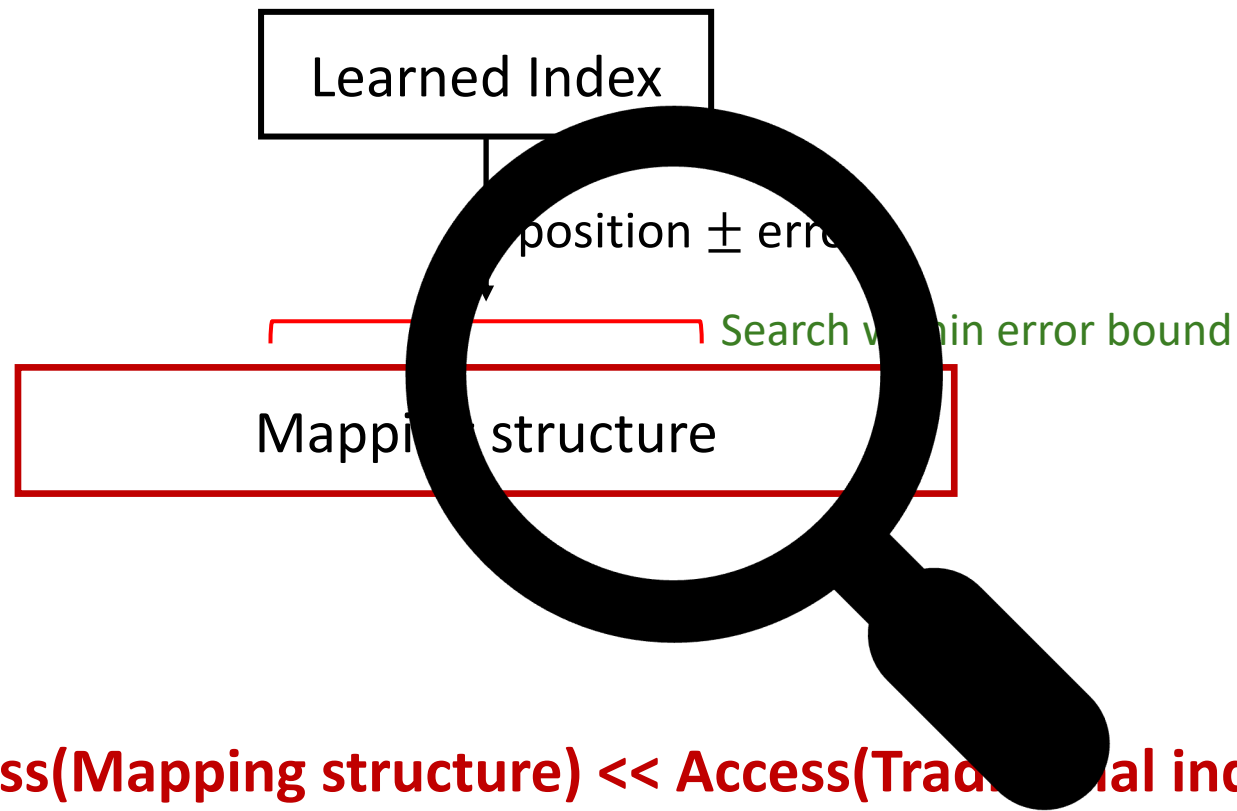


What do we Want?



$\text{Access}(\text{Mapping structure}) \ll \text{Access}(\text{Traditional index})$

What structure can we use?



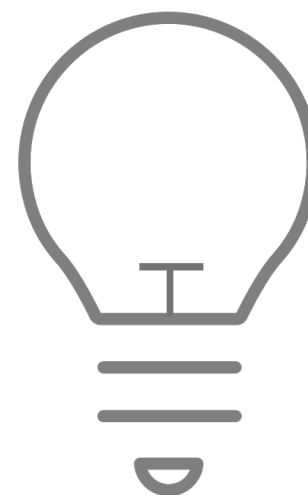
$\text{Access}(\text{Mapping structure}) \ll \text{Access}(\text{Traditional index})$

What can we use?

Learned Index

position $\rightarrow$

Mapping structure

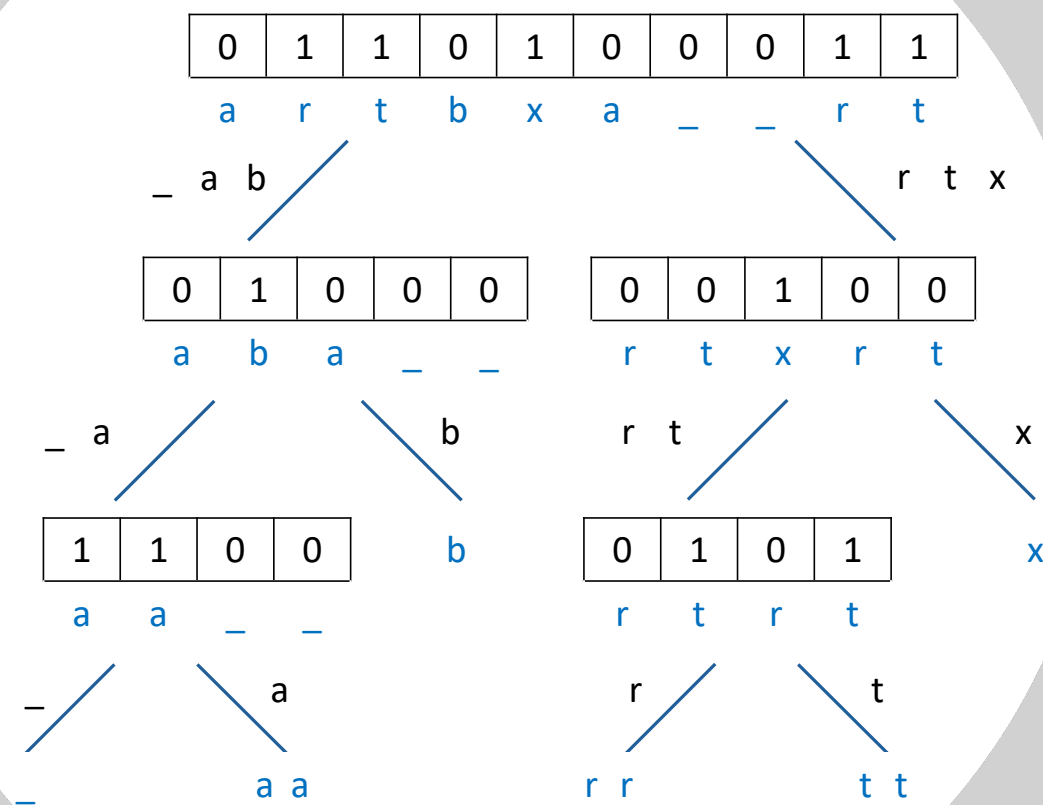


What can we use?

Learned Index

position =

Mapping structure

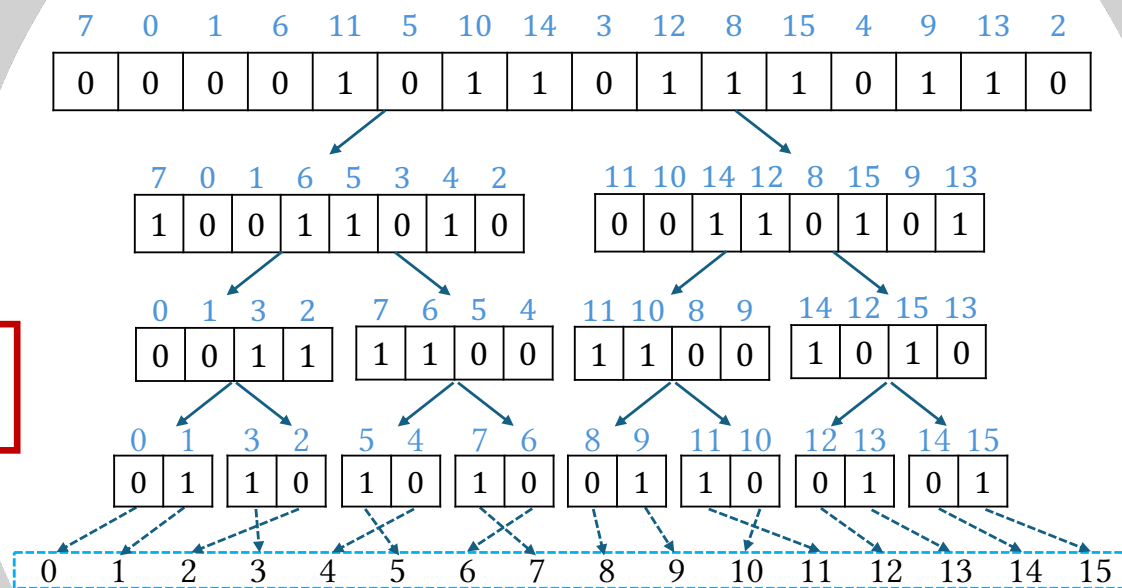


What can we use?

Learned Index

position

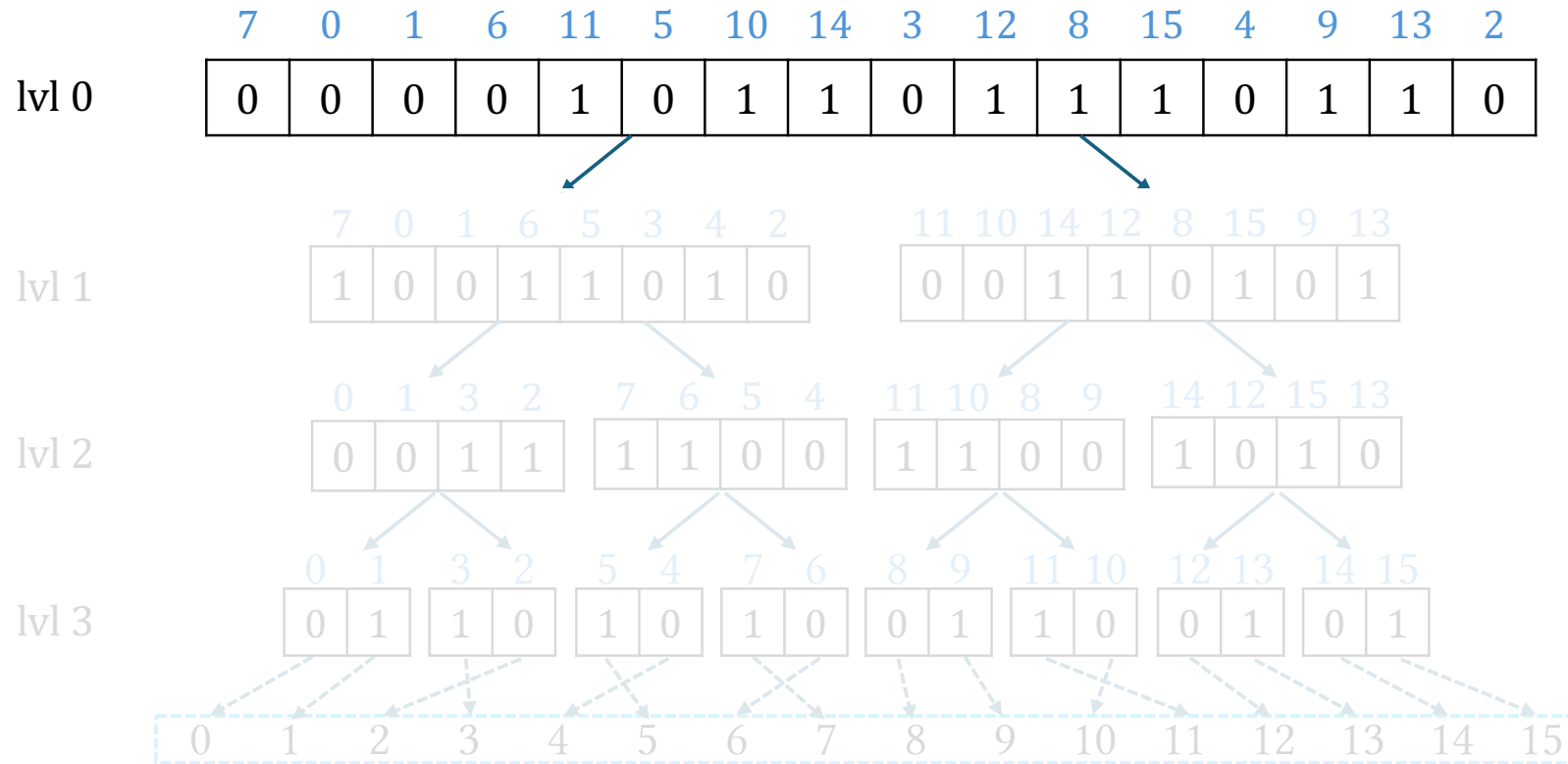
Mapping structure



Permutation Mappings

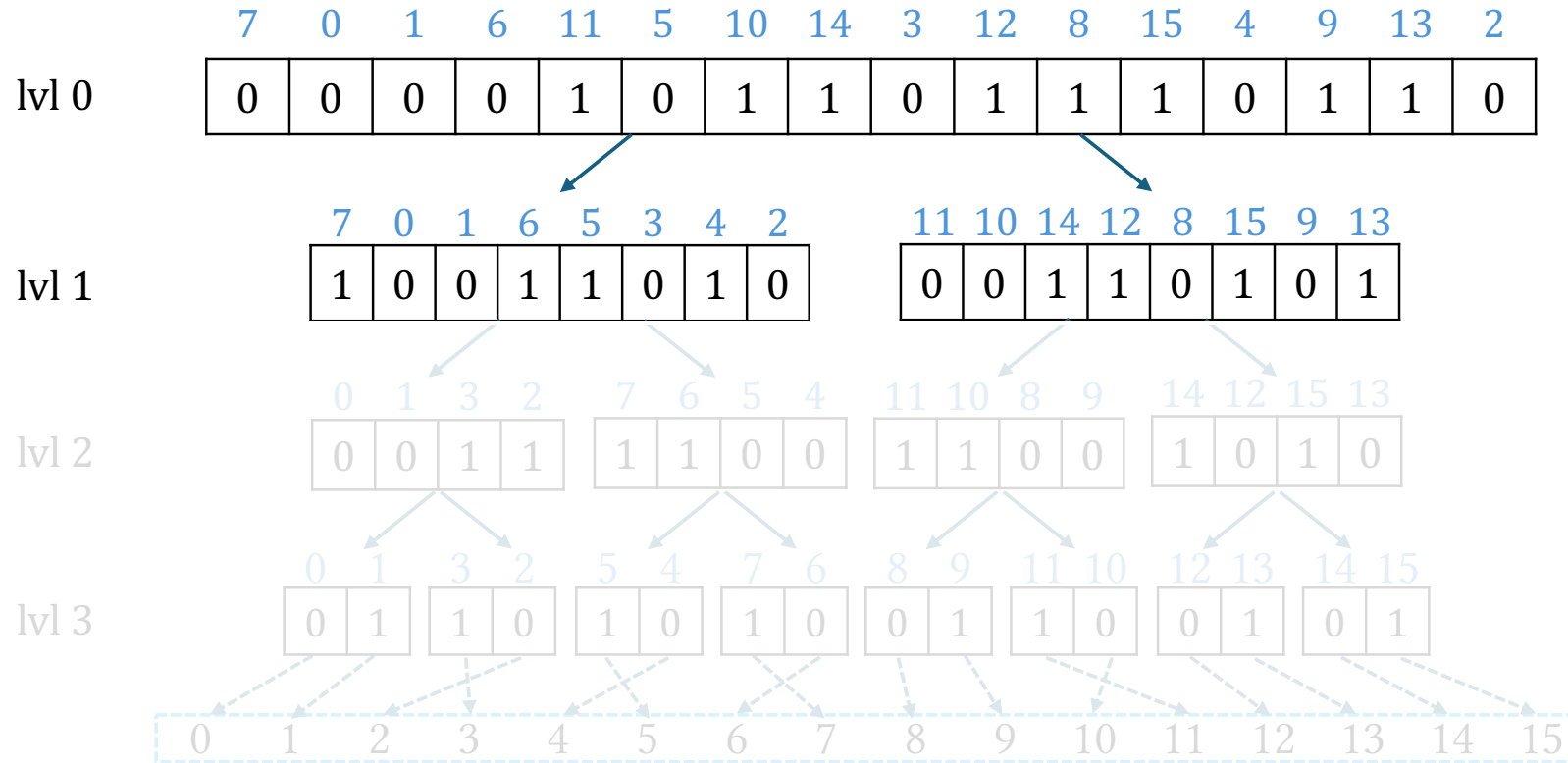
Method	Description	Pros	Cons	Paper
LSI	Bit packed permutation vector	Fast	$O(n)$ growth	
IWT	Wavelet trees for integers	Upto 11x smaller (B+ Tree)	Order of magnitude slower (B+ Tree)	

Wavelet Trees to IWT



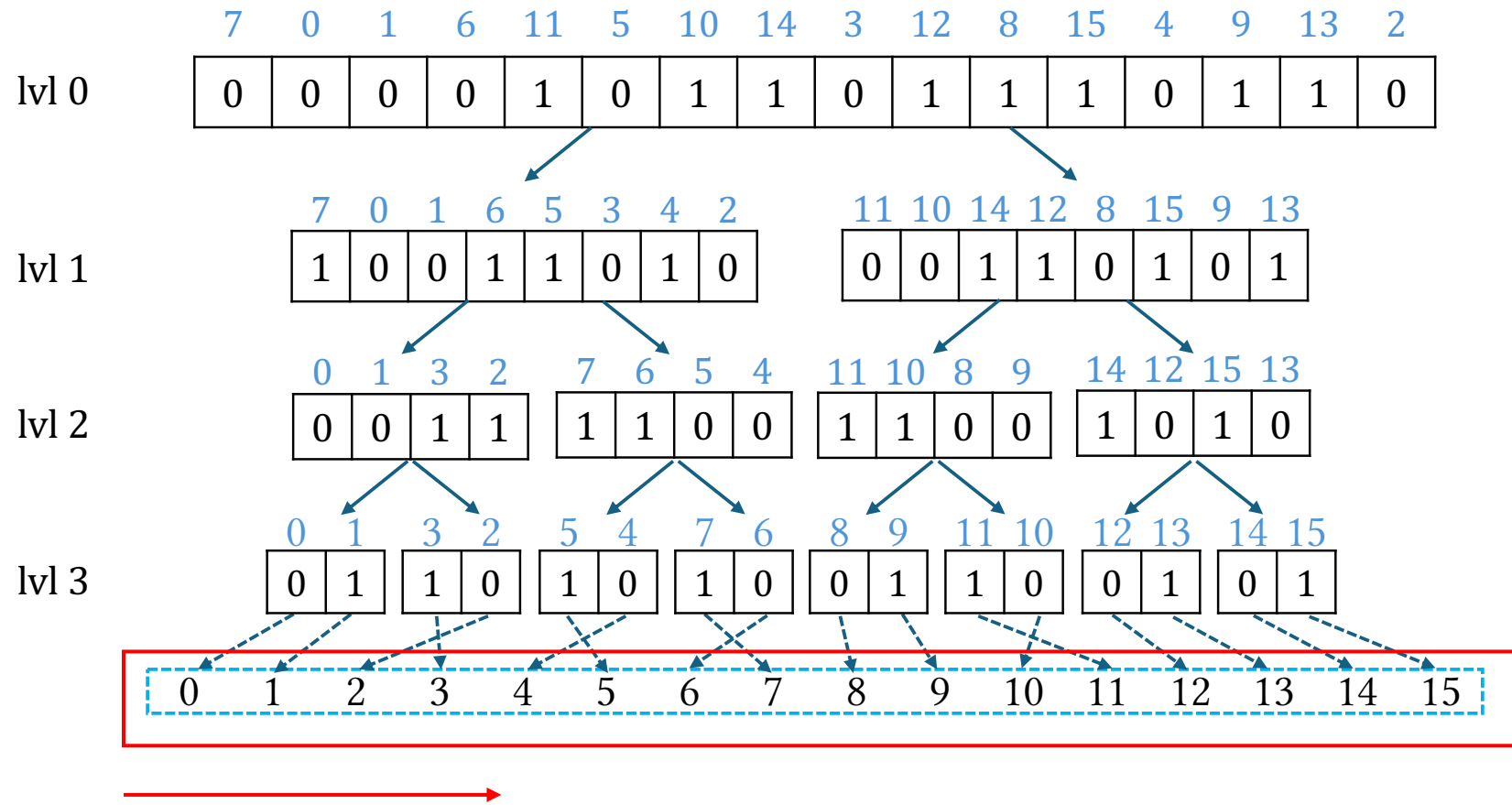
Wavelet trees for integers → IWT

Wavelet Trees to IWT



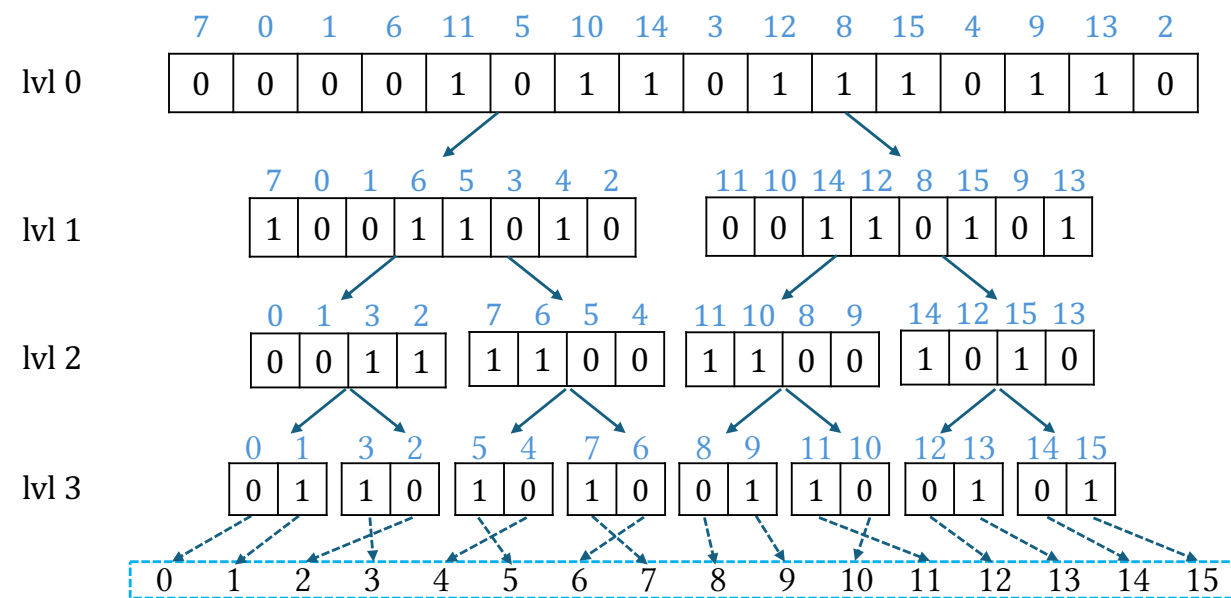
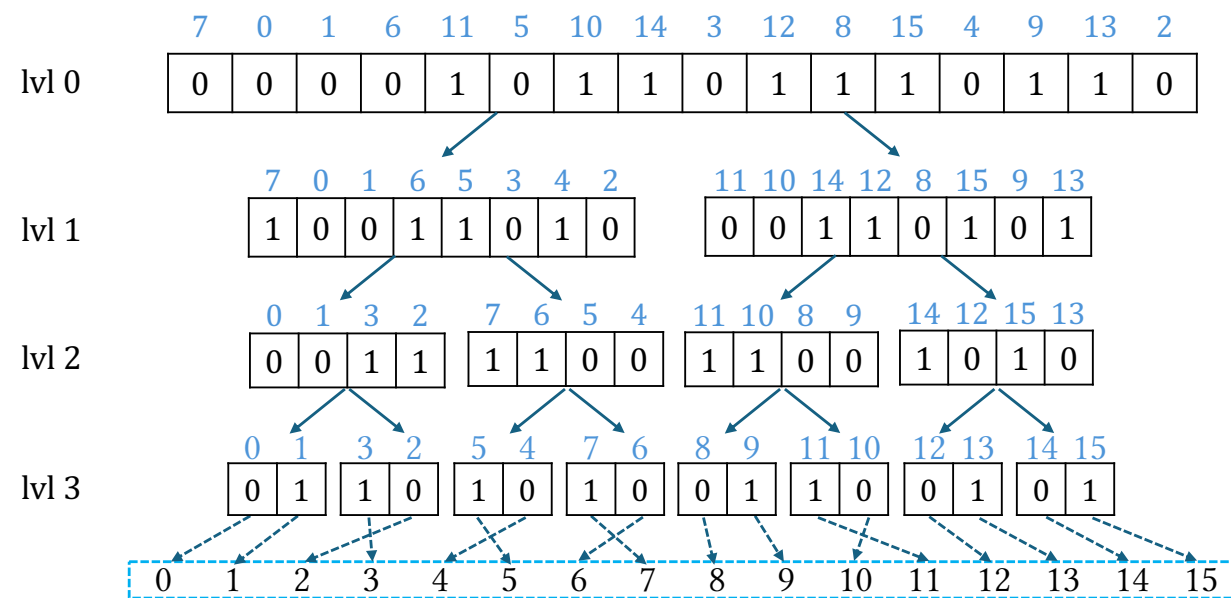
Wavelet trees for integers → IWT

Wavelet Trees to IWT



Wavelet trees for integers → IWT

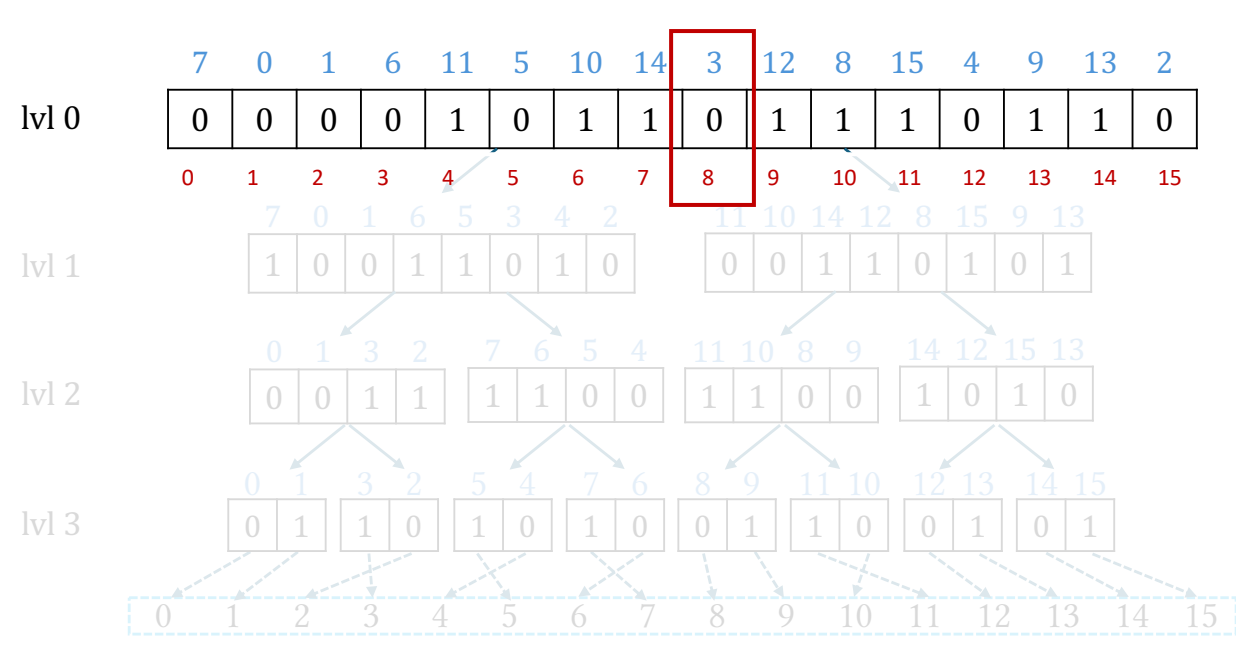
Integer Wavelet Trees (IWT)



Wavelet trees for integers → IWT

Integer Wavelet Trees (IWT)

access(8) = 3

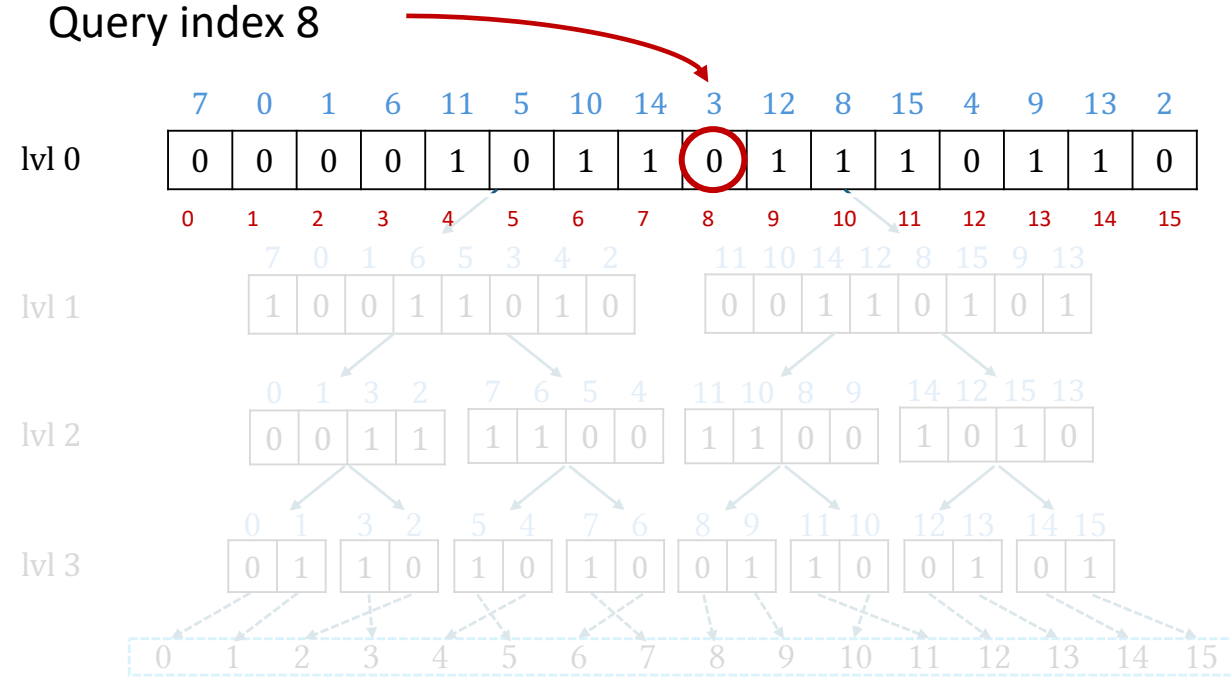


0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0
1	0	0	1	1	0	1	0	0	0	1	1	0	1	0	1
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1

Wavelet trees for integers → IWT

Integer Wavelet Trees (IWT)

Query index 8



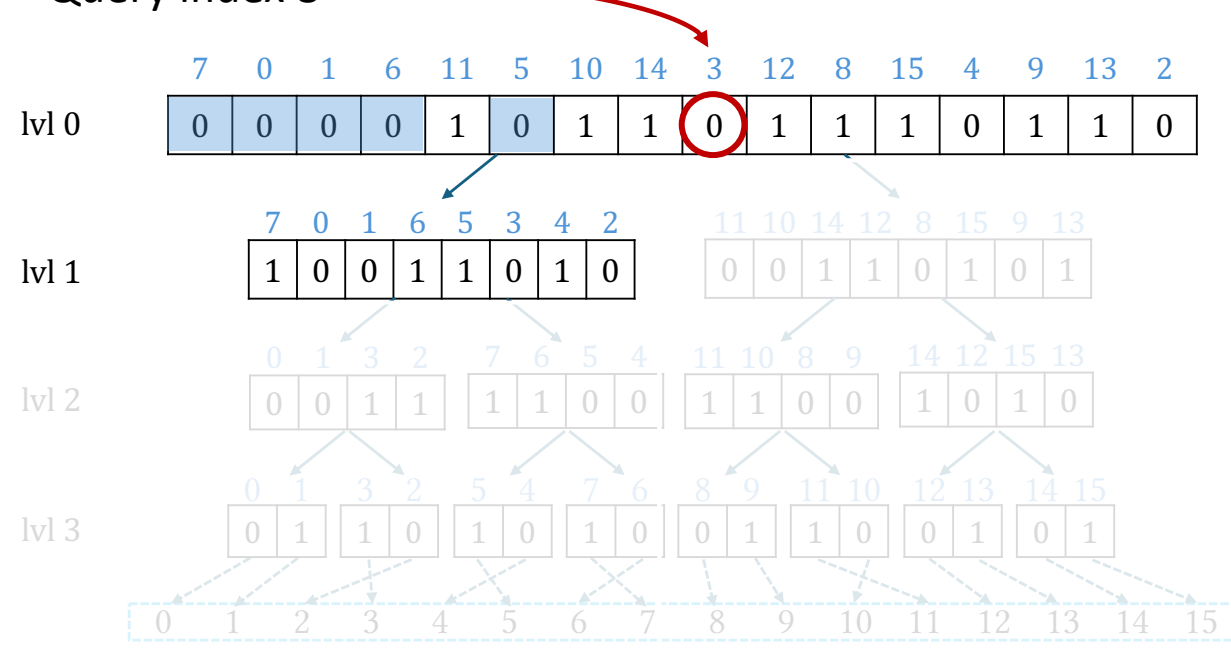
0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0
1	0	0	1	1	0	1	0	0	0	0	1	1	0	1	0
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1

Wavelet trees for integers → IWT

Integer Wavelet Trees (IWT)

Query index 8

This is the 6th 0 at this level

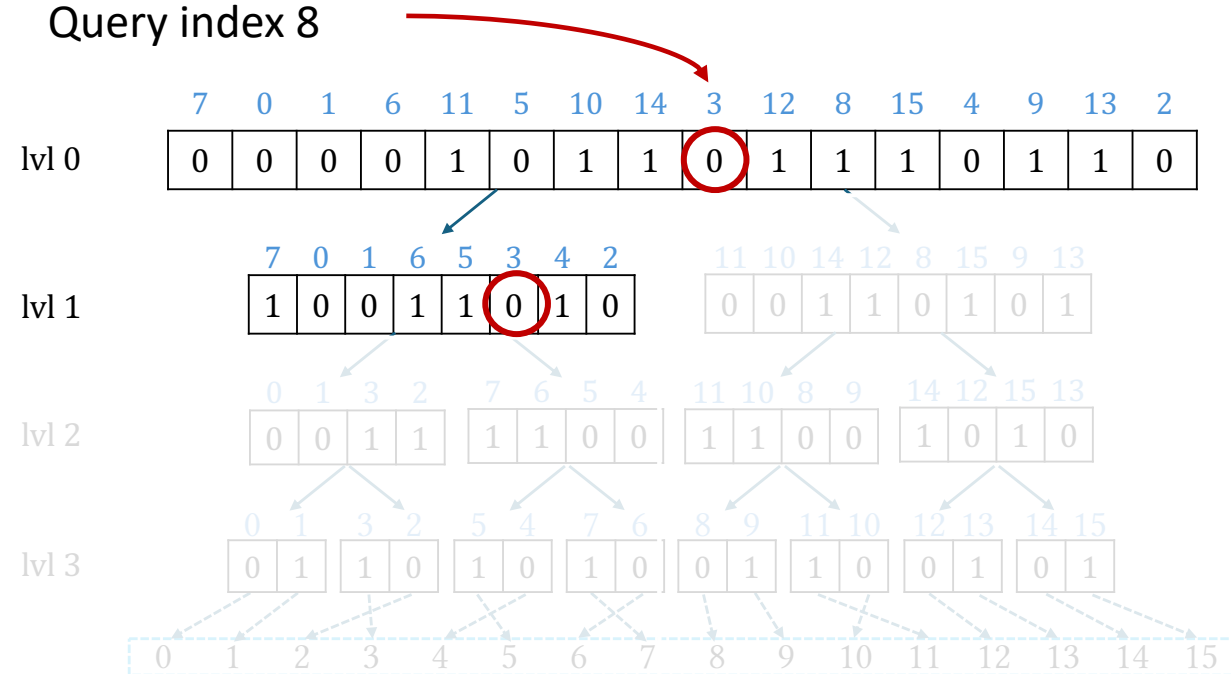


0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0
1	0	0	1	1	0	1	0	0	0	0	1	1	0	1	0
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1

Rank determines position within subtree

Integer Wavelet Trees (IWT)

Query index 8

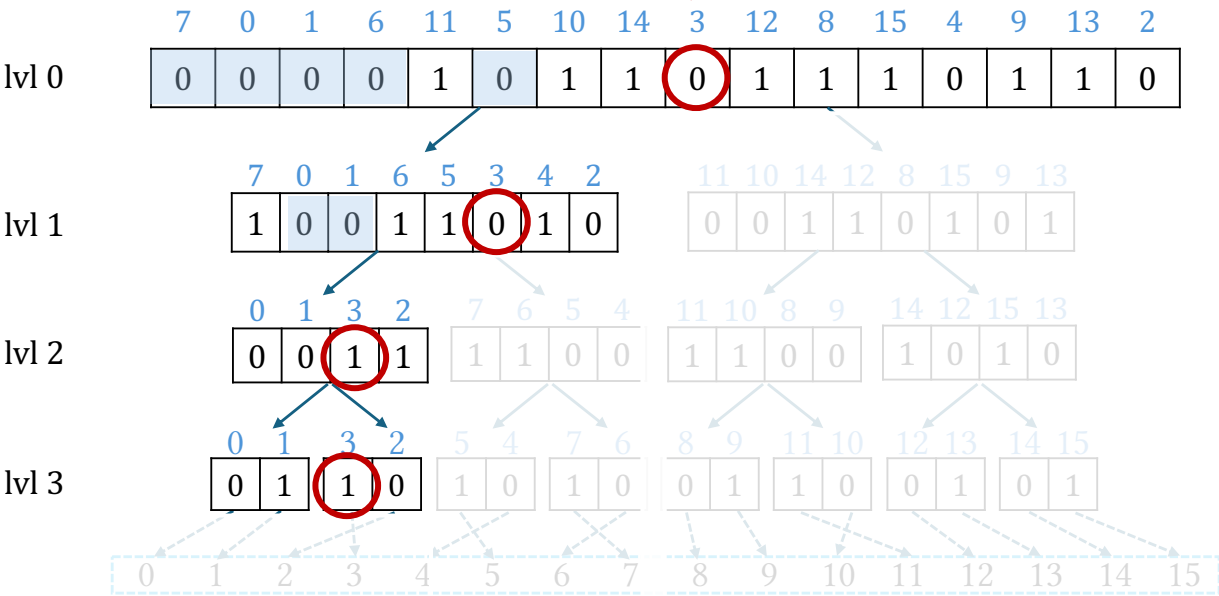


Query index 8

0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0
1	0	0	1	1	0	1	0	0	0	1	1	0	1	0	1
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1

Integer Wavelet Trees (IWT)

Query index 8

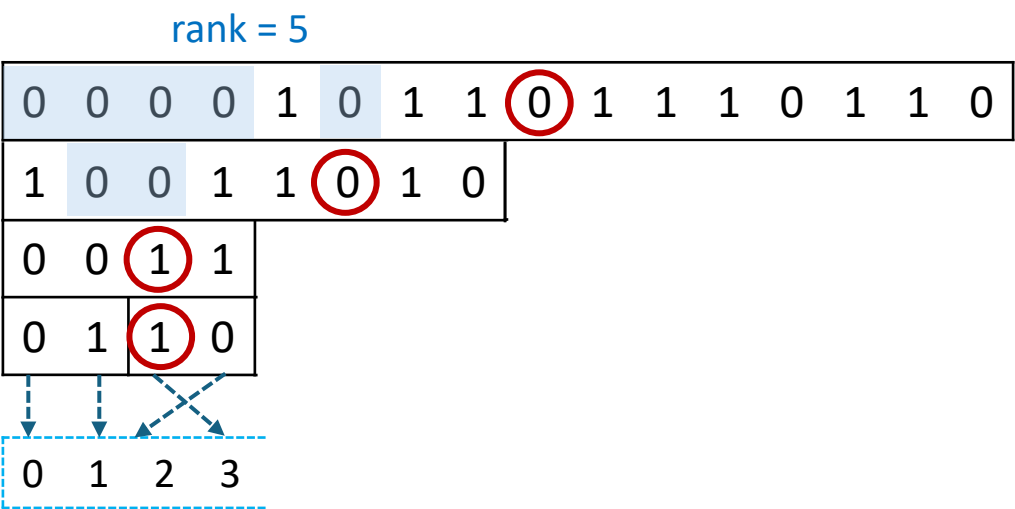
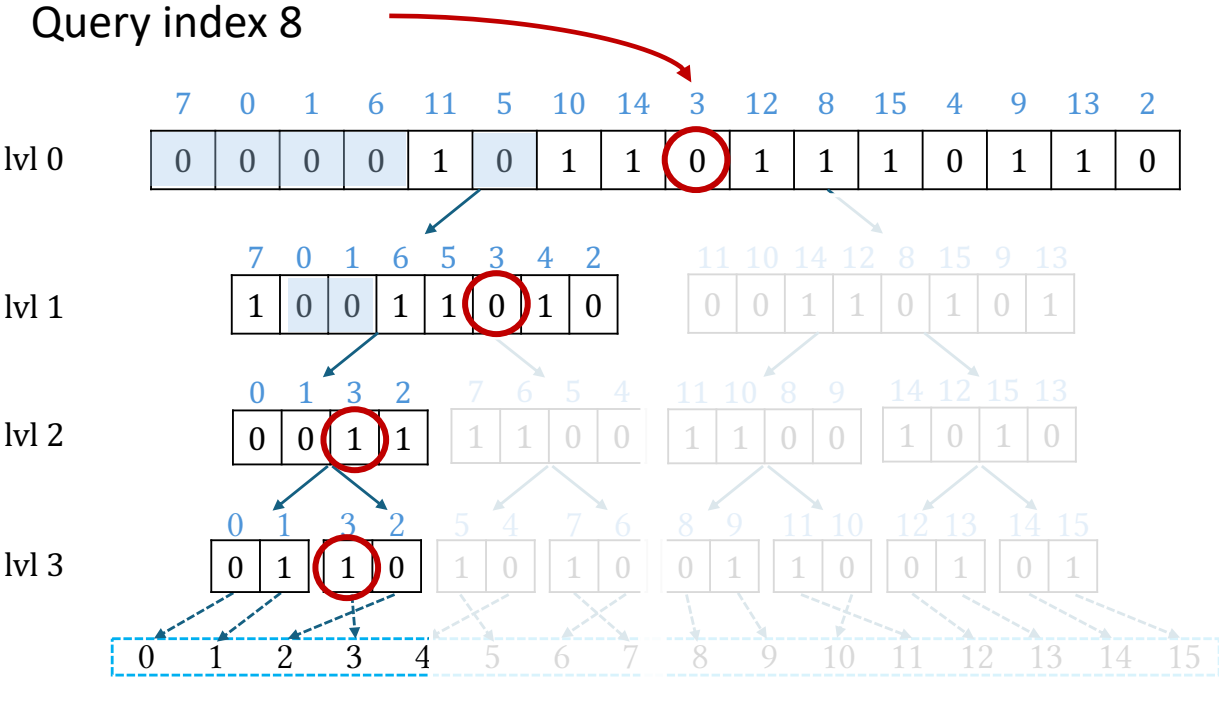


rank = 5

0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0
1	0	0	1	1	0	1	0	0	0	1	1	0	1	0	1
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1

Integer Wavelet Trees (IWT)

Query index 8



Only iterate till number of nodes > fanout

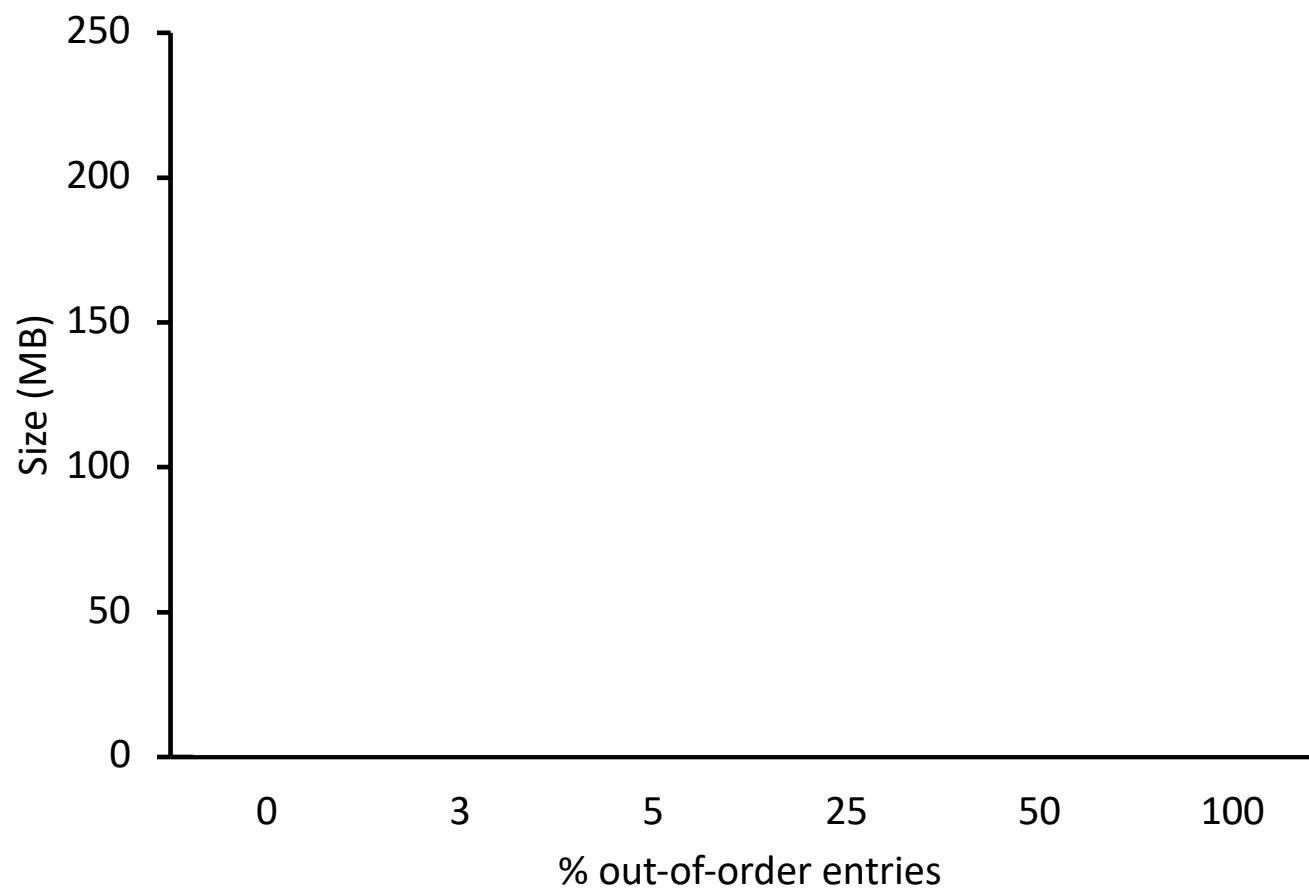
Integer Wavelet Trees (IWT)

Use RLE where advantageous

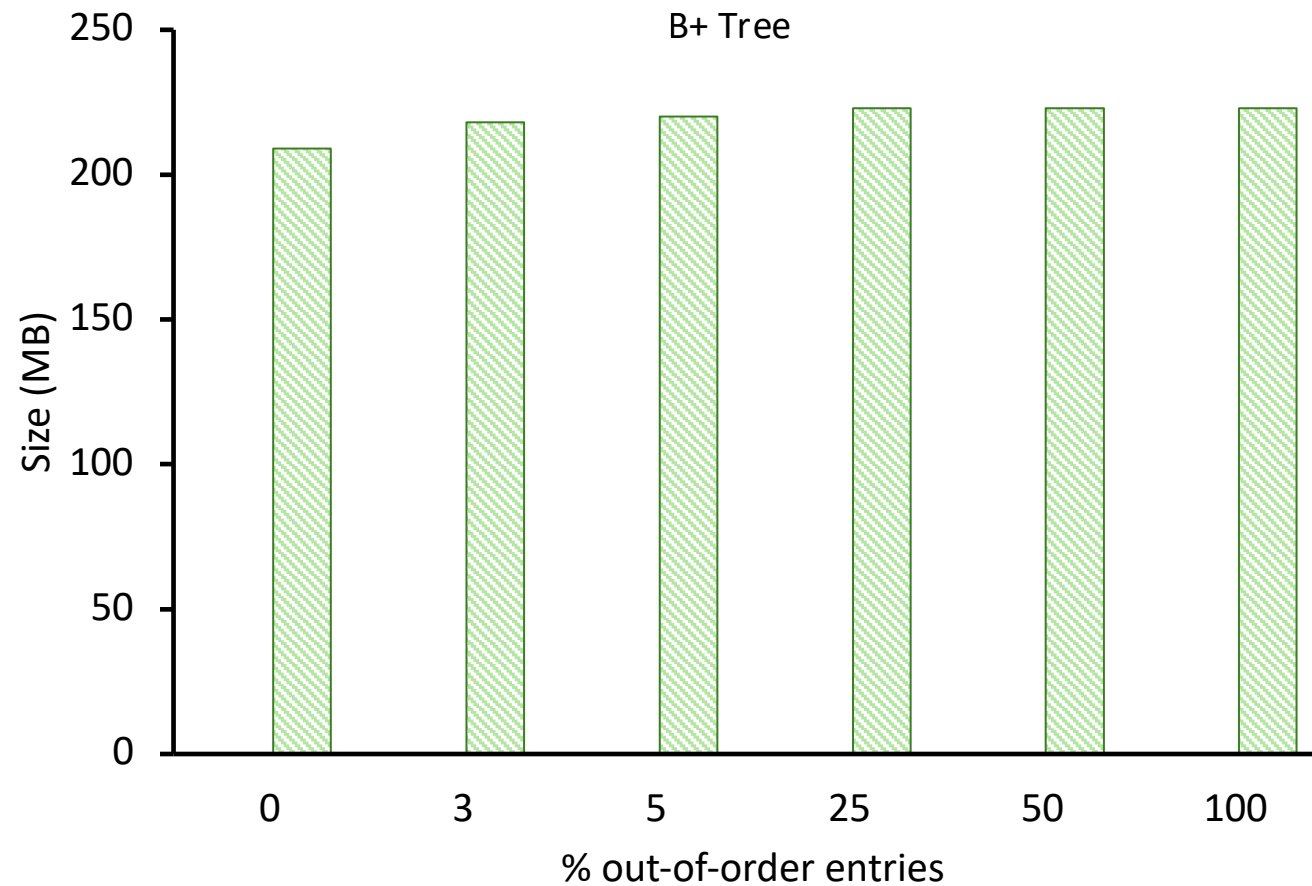
4											3					
0	0	0	0	1	0	1	1	0	1	1	1	0	1	1	0	
1	0	0	1	1	0	1	0	0	0	1	1	0	1	0	1	
0	0	1	1	1	1	0	0	1	1	0	0	1	0	1	0	
0	1	1	0	1	0	1	0	0	1	1	0	0	1	0	1	

More sorted → Less Space!

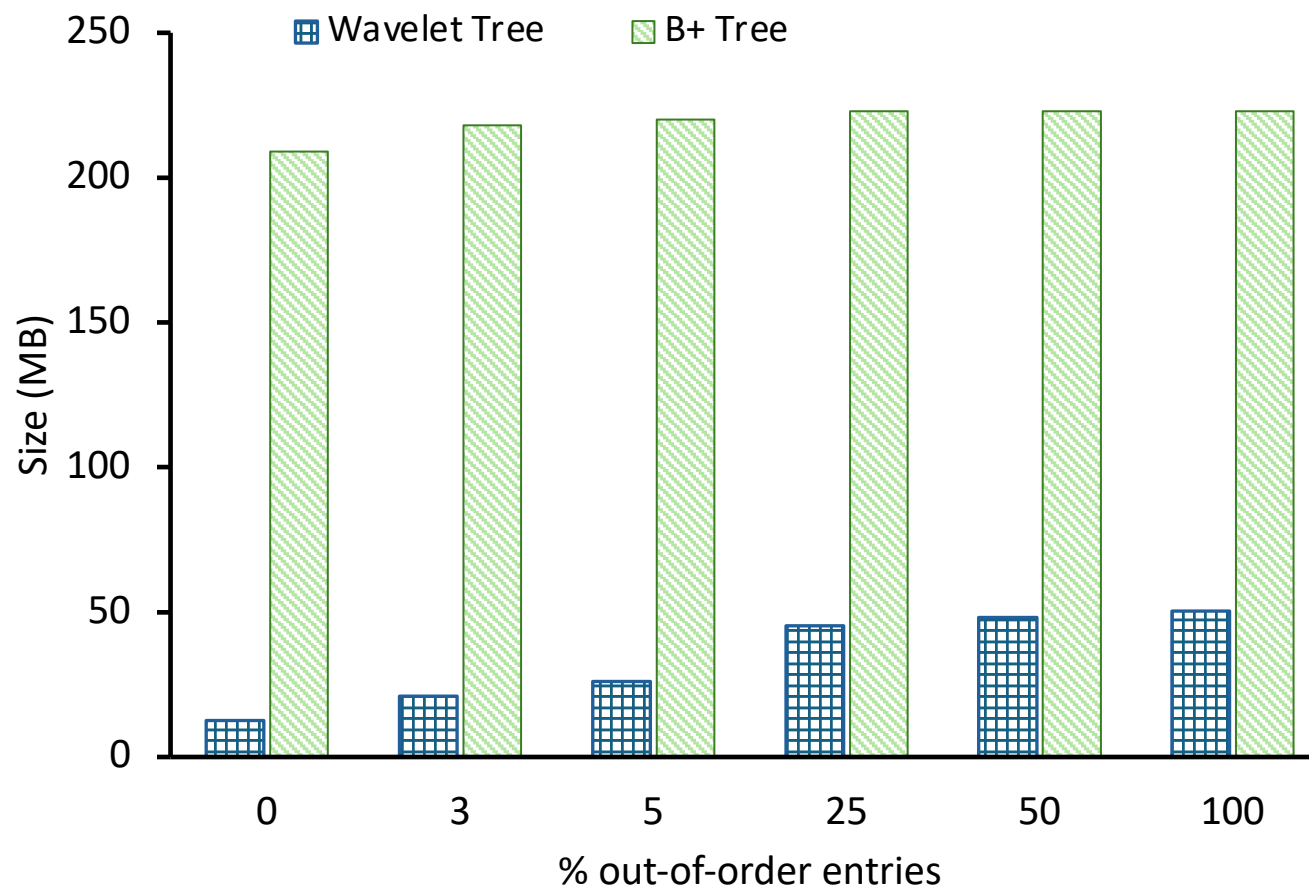
IWT in Action



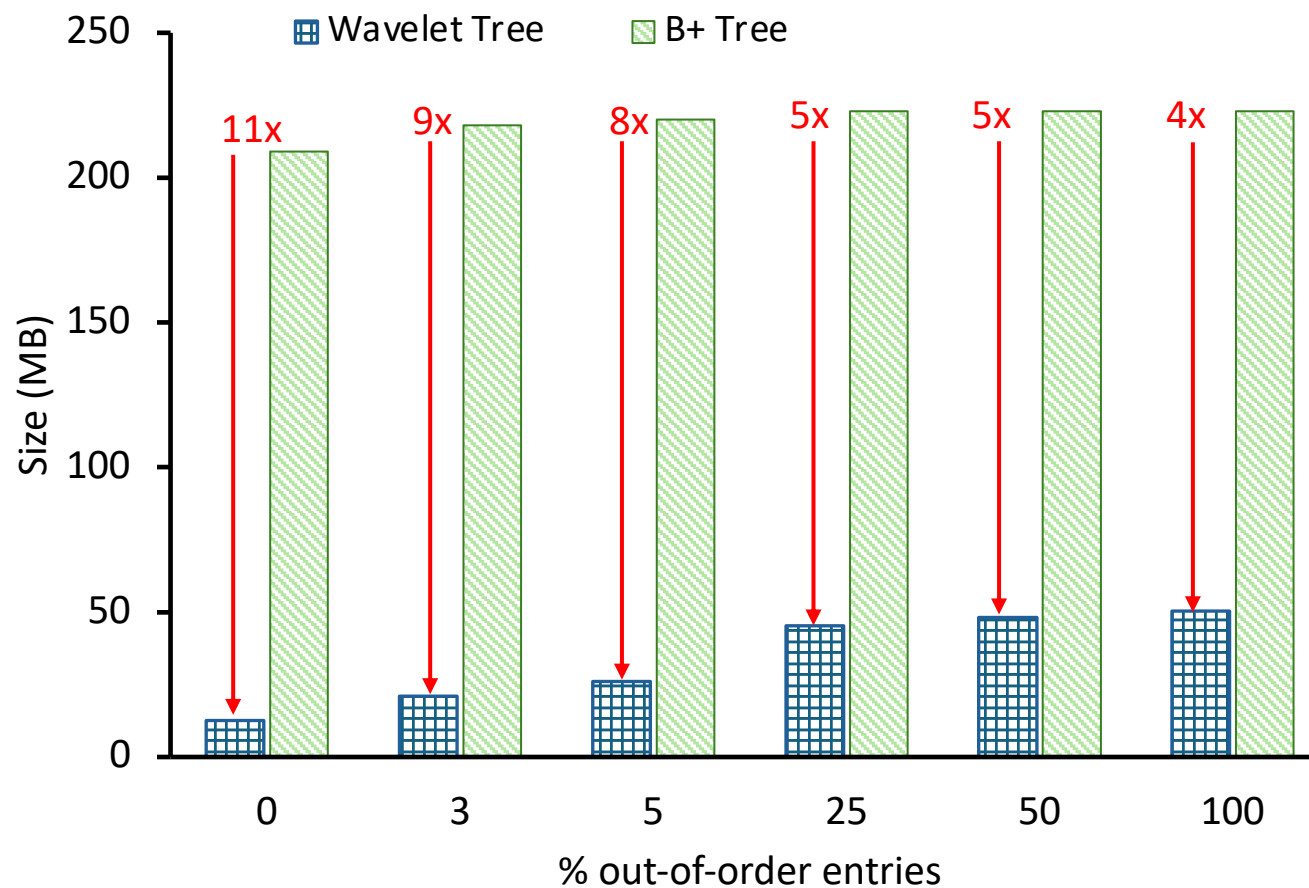
IWT has low memory footprint



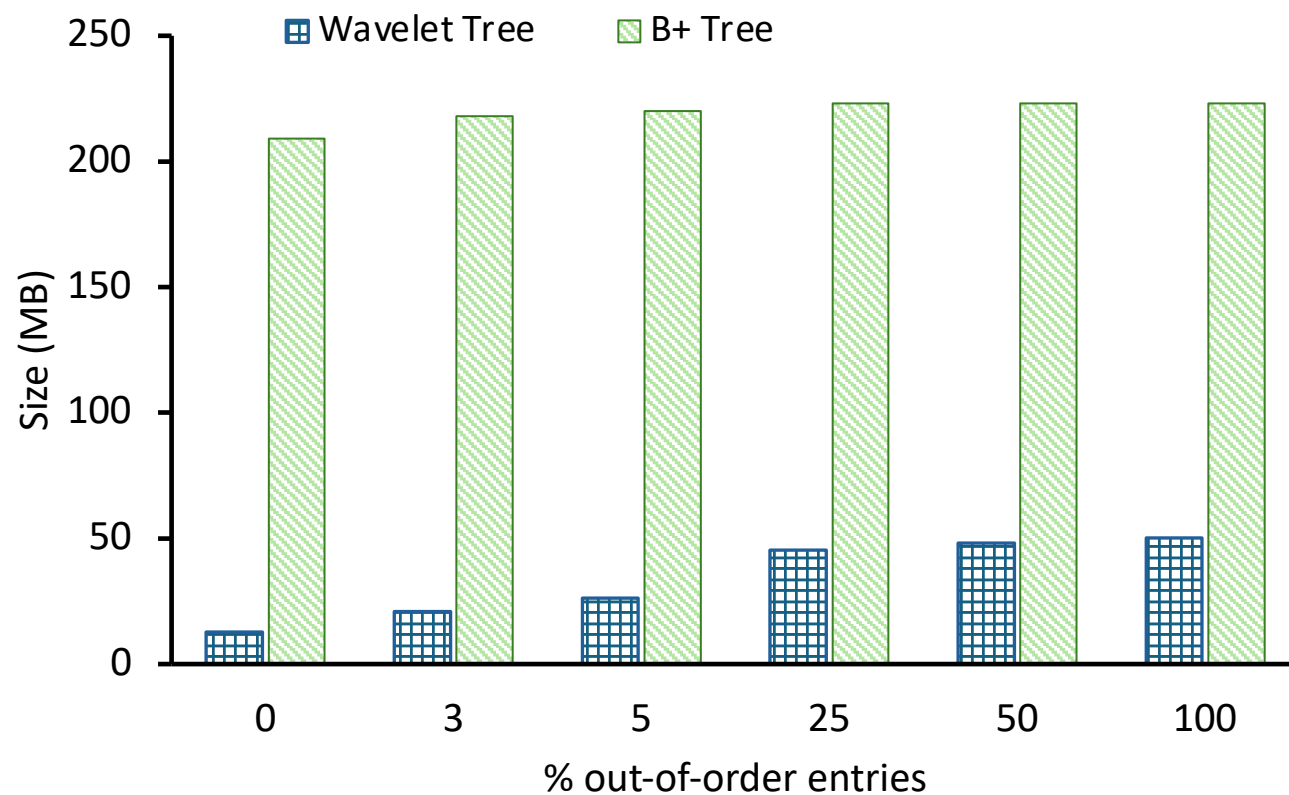
IWT has low memory footprint



IWT has low memory footprint



Access is slower!

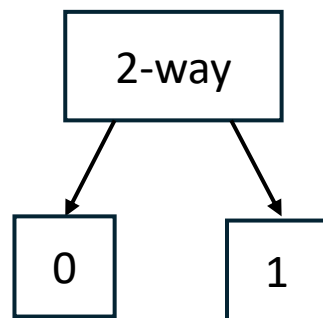


Order of magnitude slower!

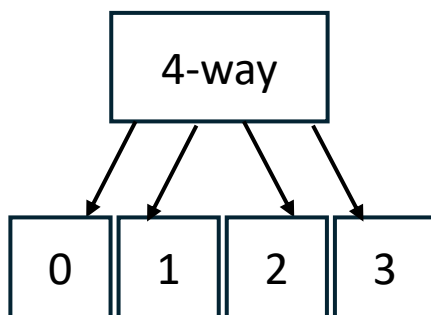
IWT is small but slow

Can we do better?

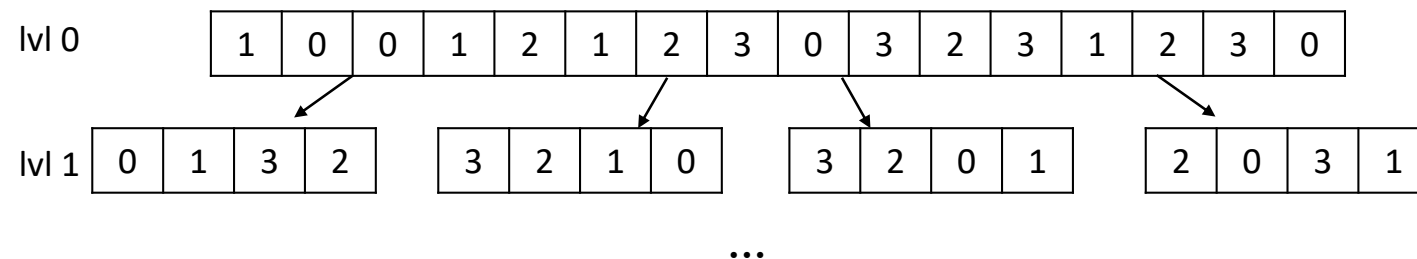
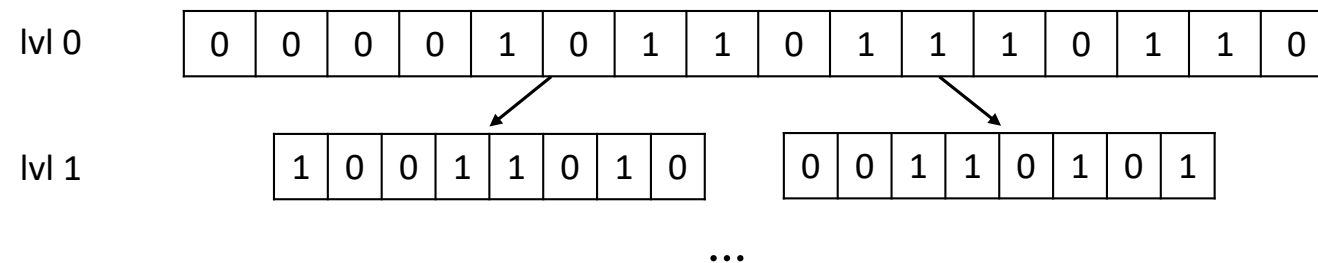
IWT to T-Way IWT



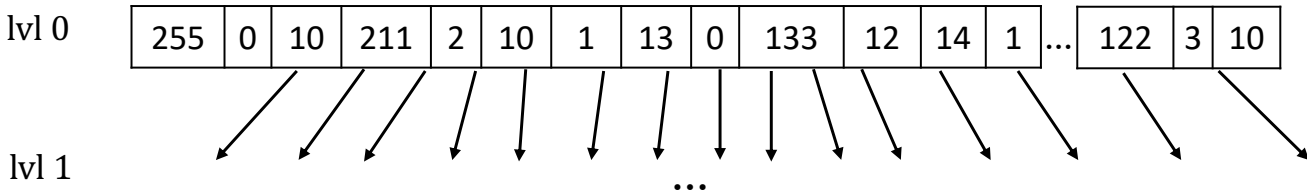
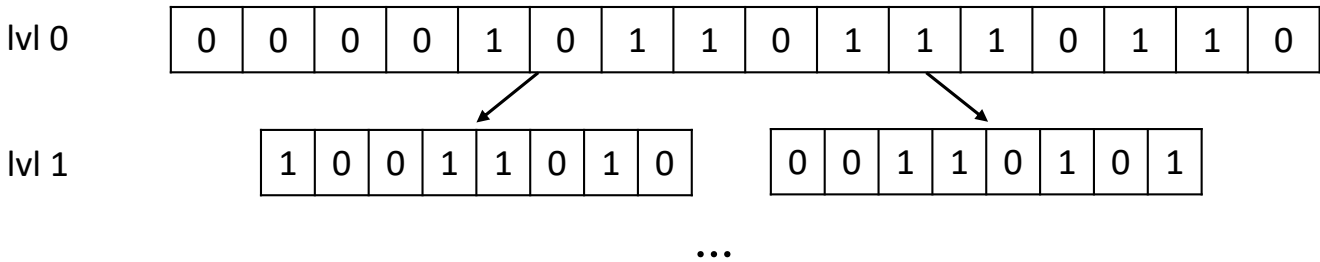
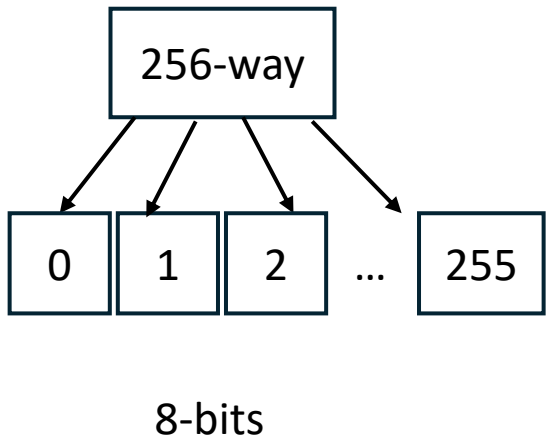
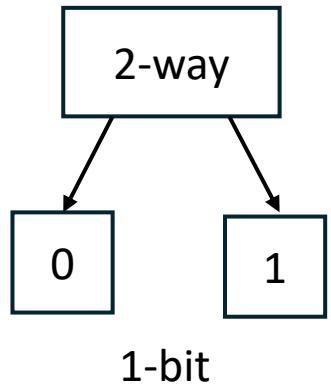
1-bit



2-bits



Increase fanout



Size of each entry increases with fanout

Permutation Mappings

Method	Description	Pros	Cons	Paper
LSI	Bit packed permutation vector	Fast	$O(n)$ growth	
2-Way IWT	Wavelet trees for integers	Upto 11x smaller (B+ Tree)	Order of magnitude slower (B+ Tree)	
T-Way IWT	IWT extended to fanout=T	Faster than B+ Tree	Loses space saving nature (metadata)	

How do we calculate rank?

0	0	0	0	1	1	1	1	1	0
0	1	0	0	1	0	1	1	1	0
1	0	0	1	0	0	1	0	1	0
0	1		0	1	1	0		0	1

Rank₁(7) = Index position – Rank₀(7)

Knowing rank of either bit allows us to calculate rank of the other!

How do we calculate rank?

0	0	0	0	1	1	1	1	1	0
0	1	0	0	1	0	1	1	1	0
1	0	0	1	0	0	1	0	1	0
0	1		0	1	1	0		0	1

~~$\text{Rank}_1(7) = \text{Index position} - \text{Rank}_0(7)$~~

0	0	1	0	1	0	2	1	2	2	1	3	3	2	3	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Does not hold for higher fanout!

Lets store rank separately

Indexed entries

L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

How do we calculate rank?

Indexed entries

L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

How do we calculate rank?

Count of 2s till this position = 3

Data matrix

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

R0																
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

How do we calculate rank?

Count of 2s till this position = 3

Data matrix

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

R0													3			
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

How do we calculate rank?

Data matrix

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

R0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	0	0	1	0	1	0	2	1	2	2	1	3	3	2	3	3

cumulative count for specific encoding

How do we calculate rank?

Data matrix

L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

R0	0	0	1	0	1	0	2	1	2	2	1	3	3	2	3	3
R1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

How do we calculate rank?

Data matrix

L0	1	3	1	0	0	2	0	2	1	2	3	2	1	3	3	0
L1	0	3	1	2	1	0	3	2	3	1	2	0	0	2	1	3

Rank matrix

R0	0	0	1	0	1	0	2	1	2	2	1	3	3	2	3	3
R1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Rank matrix has 1 less level than Data matrix

Evaluating Wavelet Trees

System

Dual Intel Xeon Gold 6230 CPUs

375 GiB RAM

Rocky Linux 8.10

– T-Way IWTs fanouts

4, 16, 64, 256

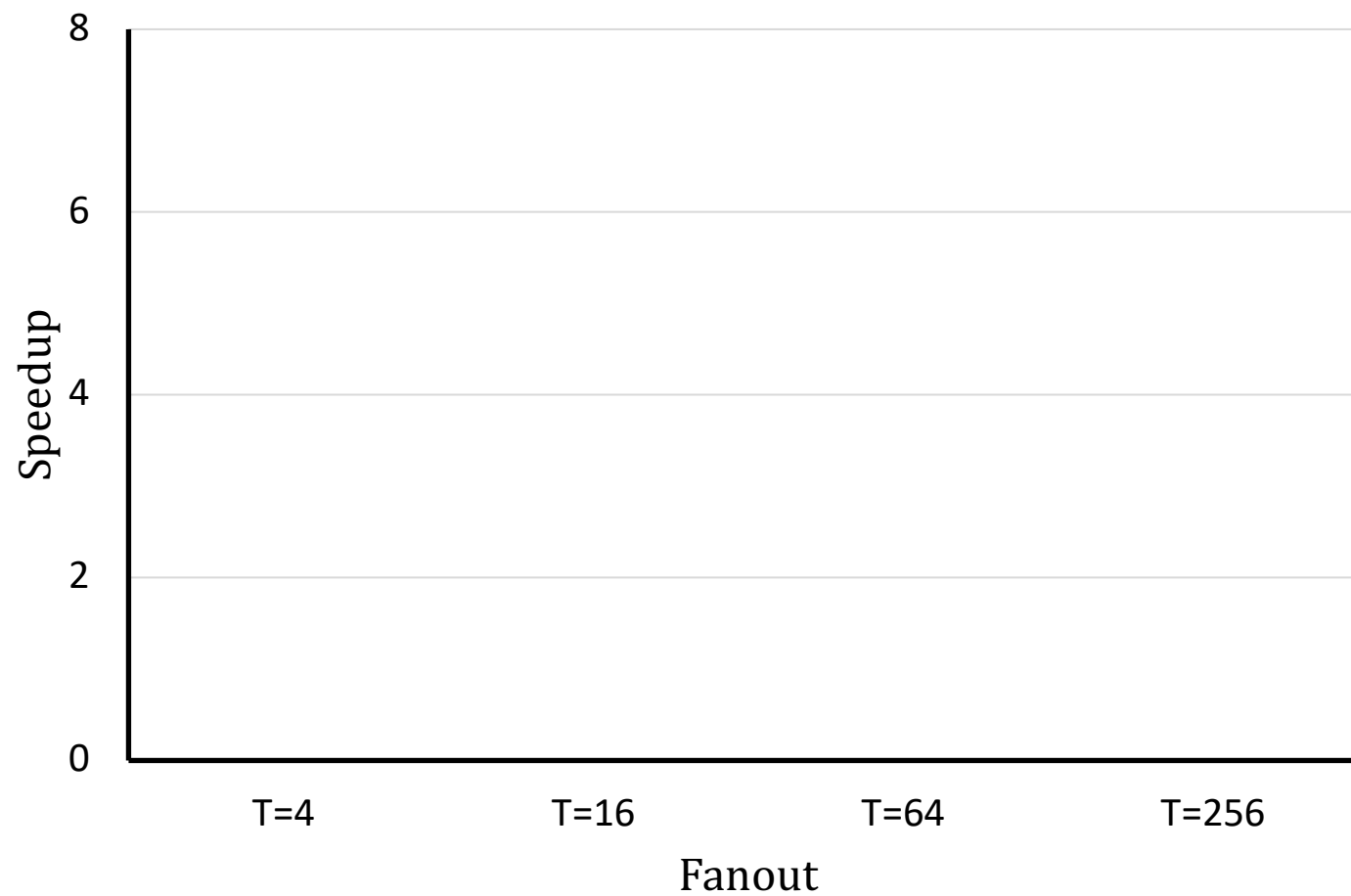
– Baseline B⁺-tree:

4 KB block size, 510 entries per leaf, QuIT

– Learned index:

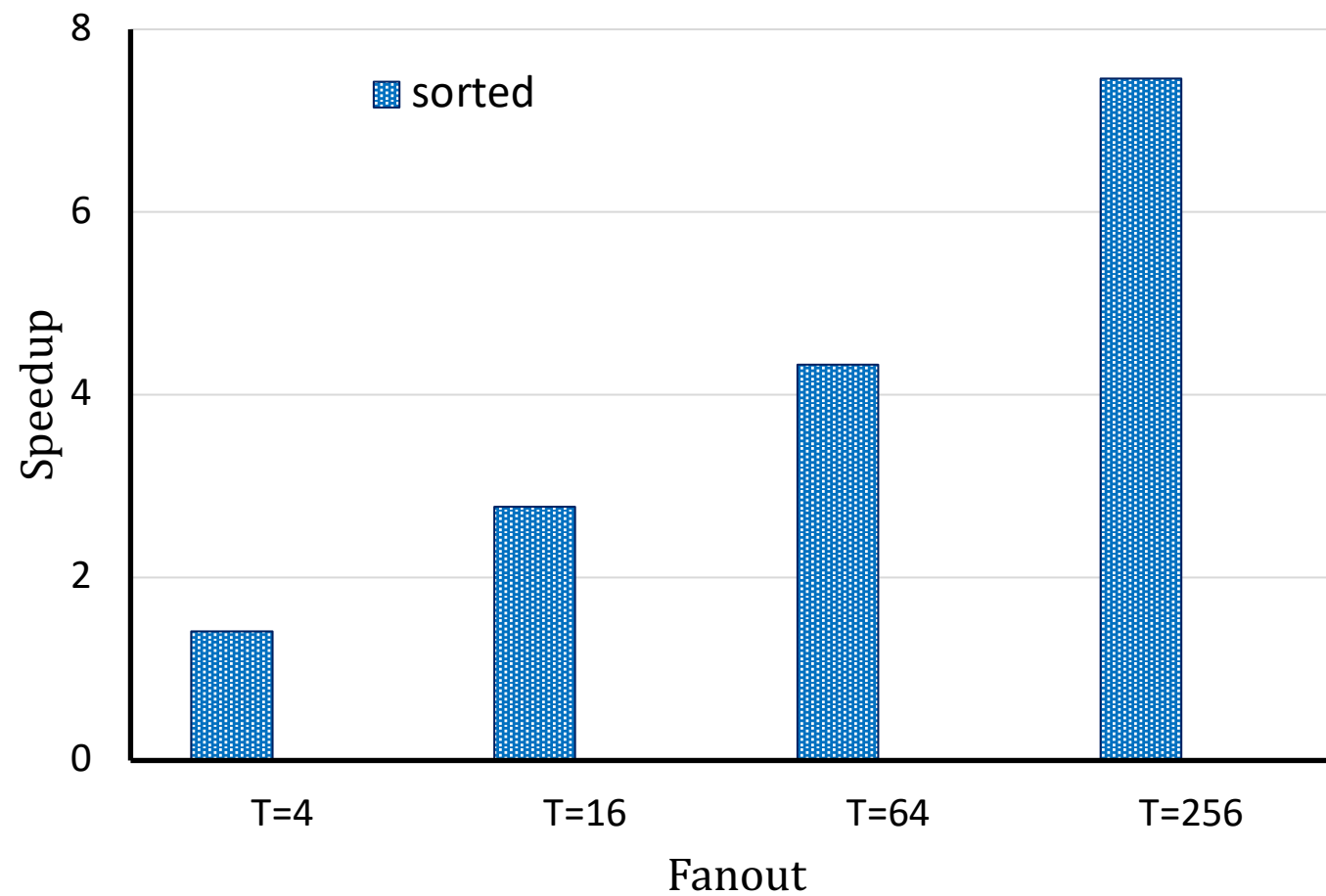
RadixSpline

T-Way IWT in Action



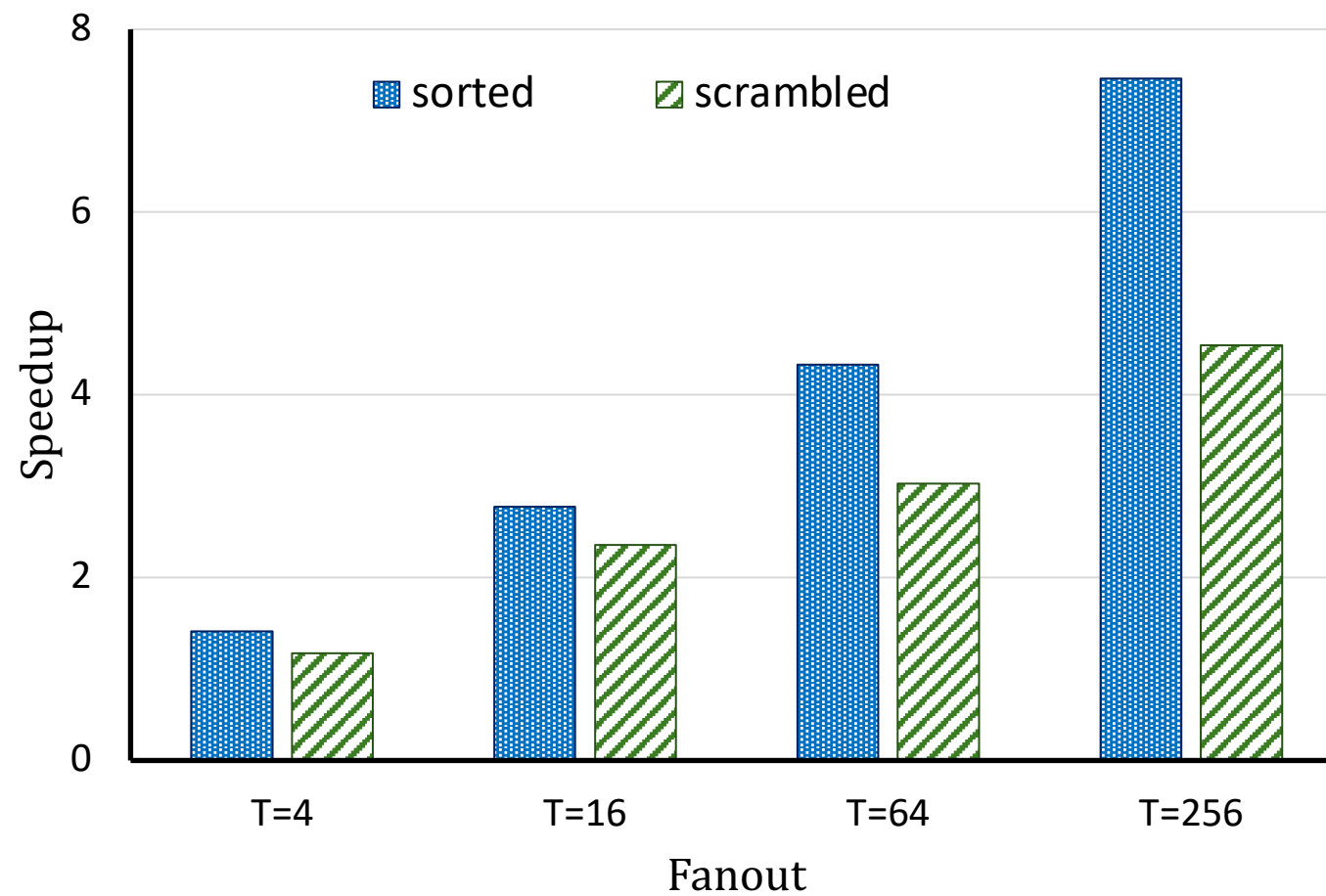
Baseline is a sortedness adaptive B+ Tree - QulT

T-Way IWT Speedup



IWT with fanout ≥ 4 , has lower access cost than QuIT

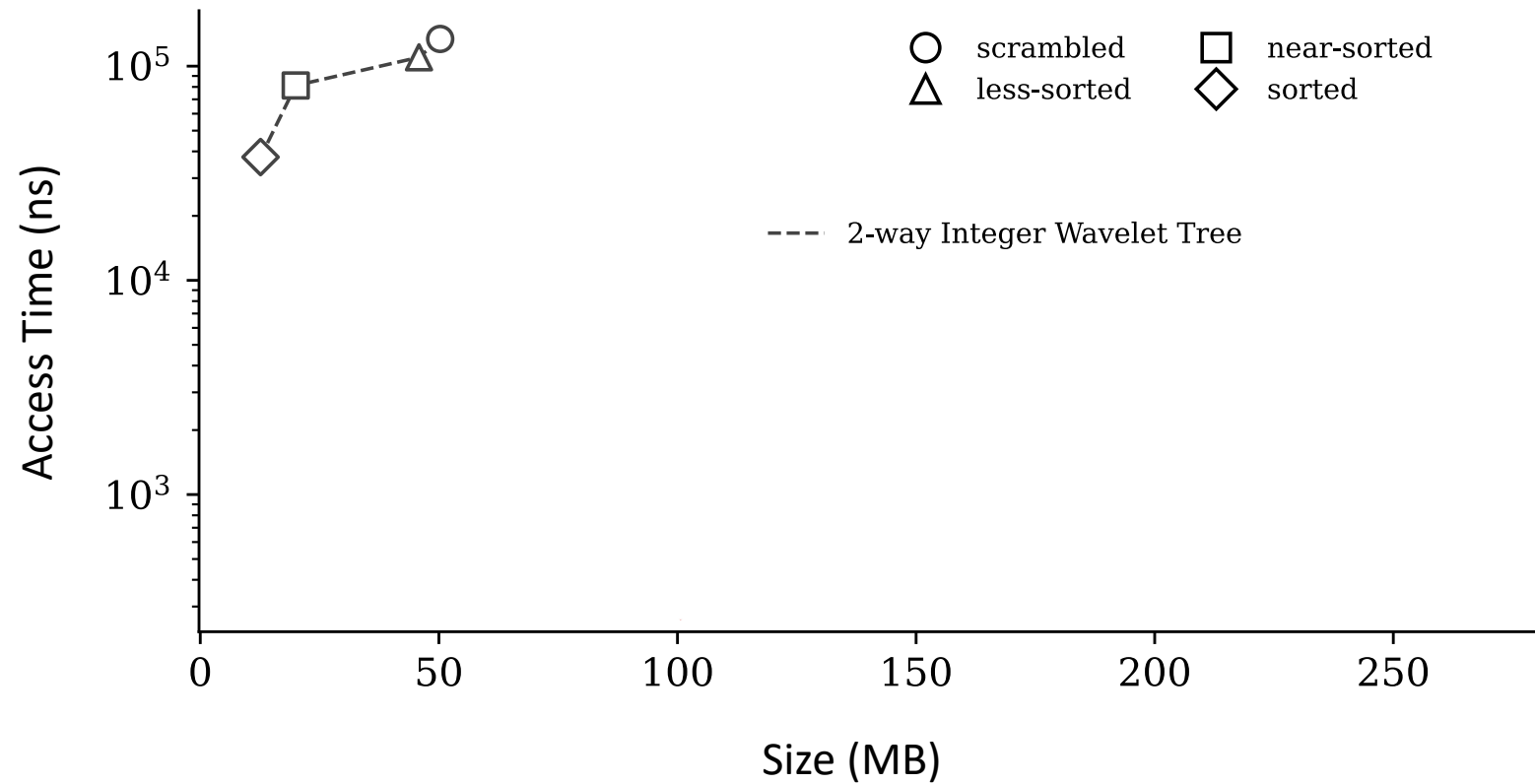
T-Way IWT Speedup



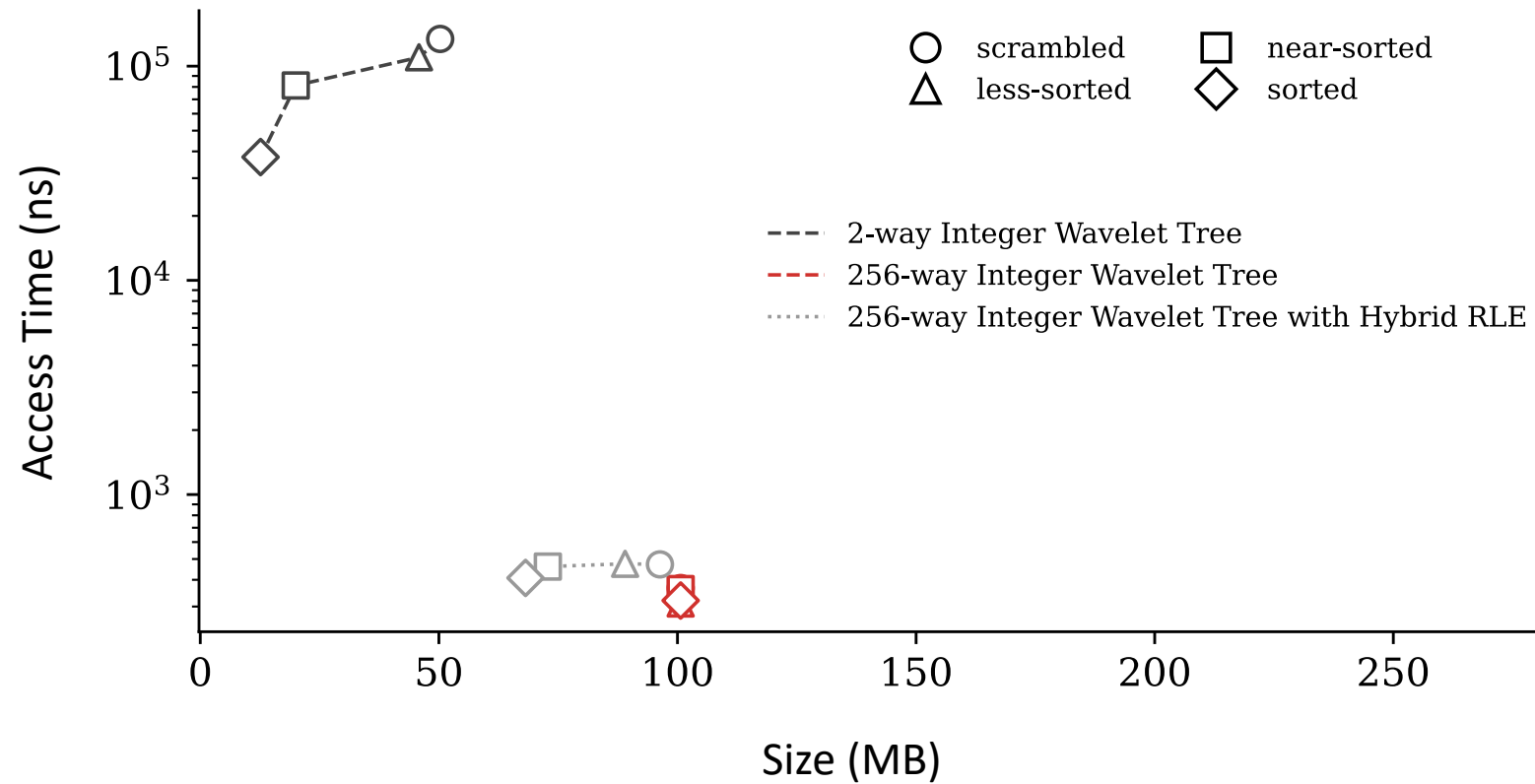
Increasing fanout decreases access cost proportionally

Putting it together

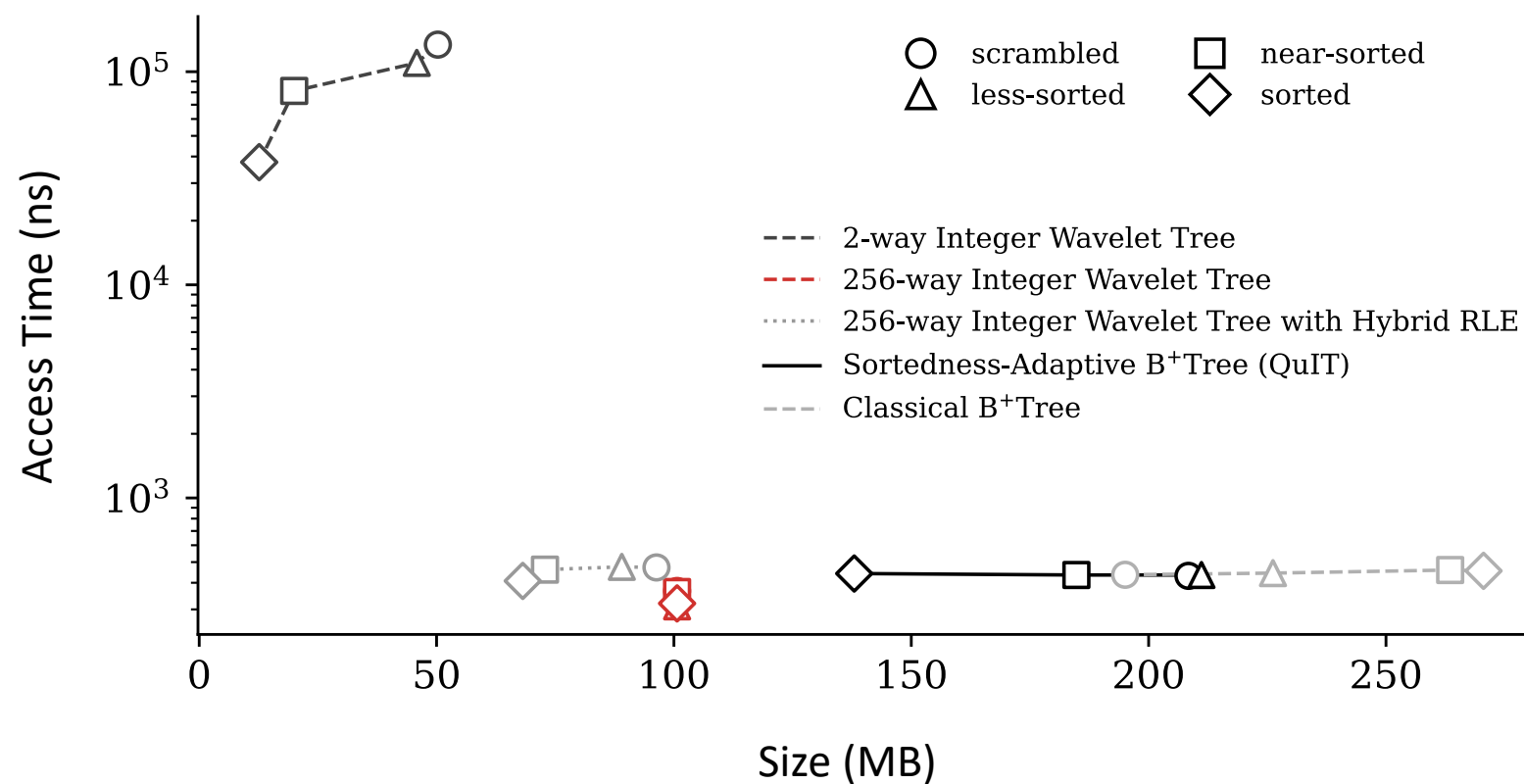
IWTs navigate space-time tradeoff



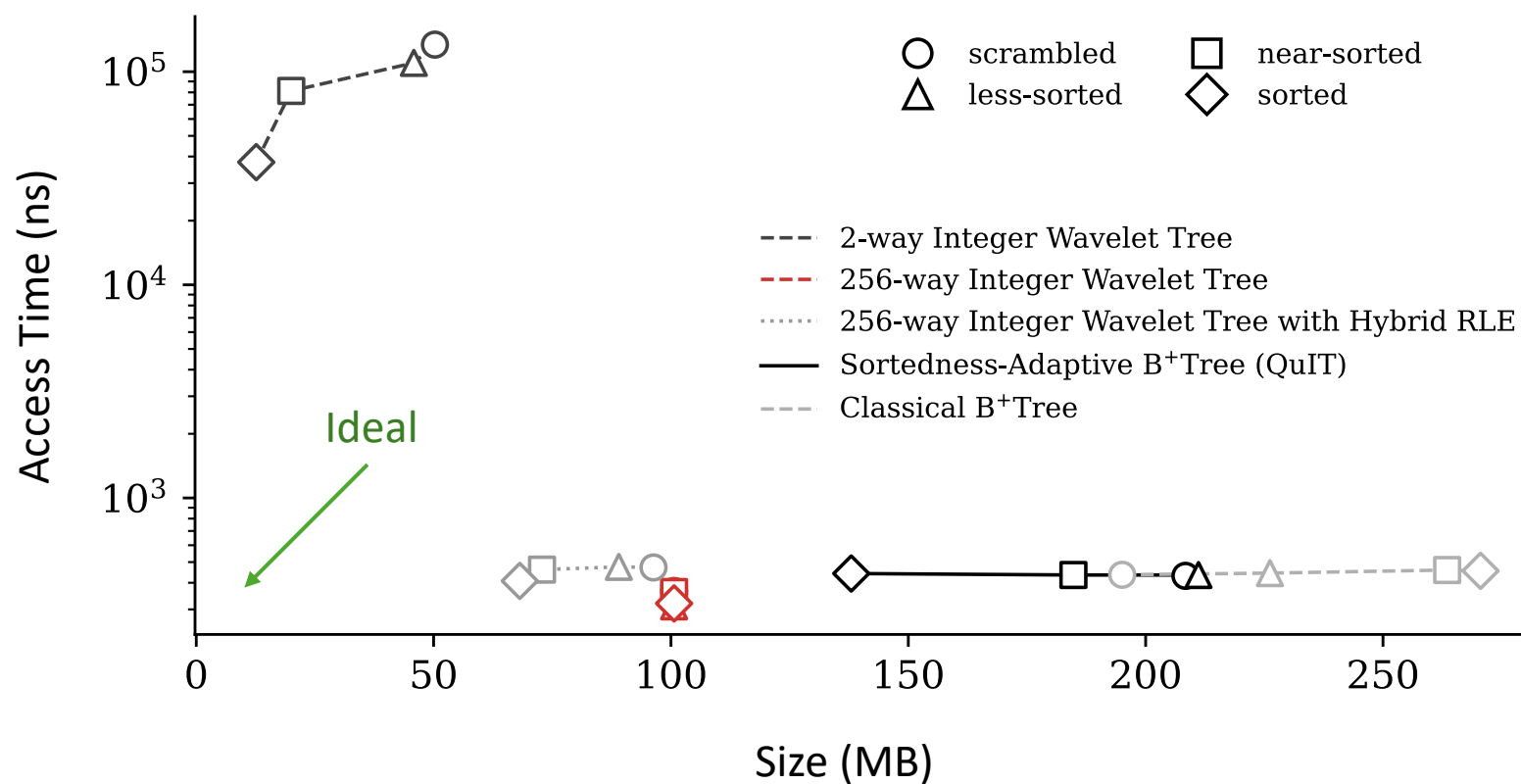
IWTs navigate space-time tradeoff



IWTs navigate space-time tradeoff



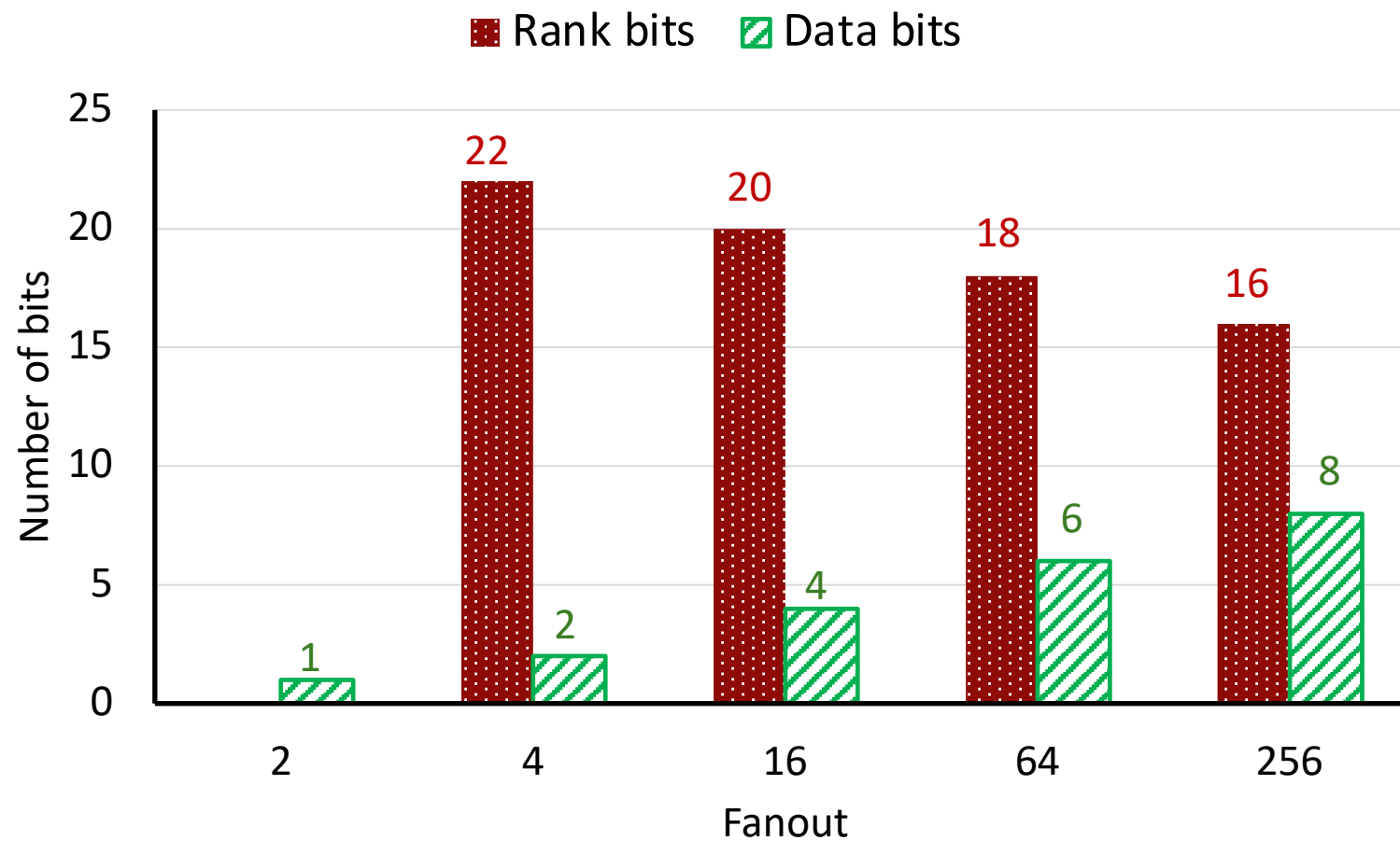
IWTs navigate space-time tradeoff



The 2-Way IWT is small but slow; the 256-Way Tree is fast but large

But why is the IWT so large?

Metadata occupies space!



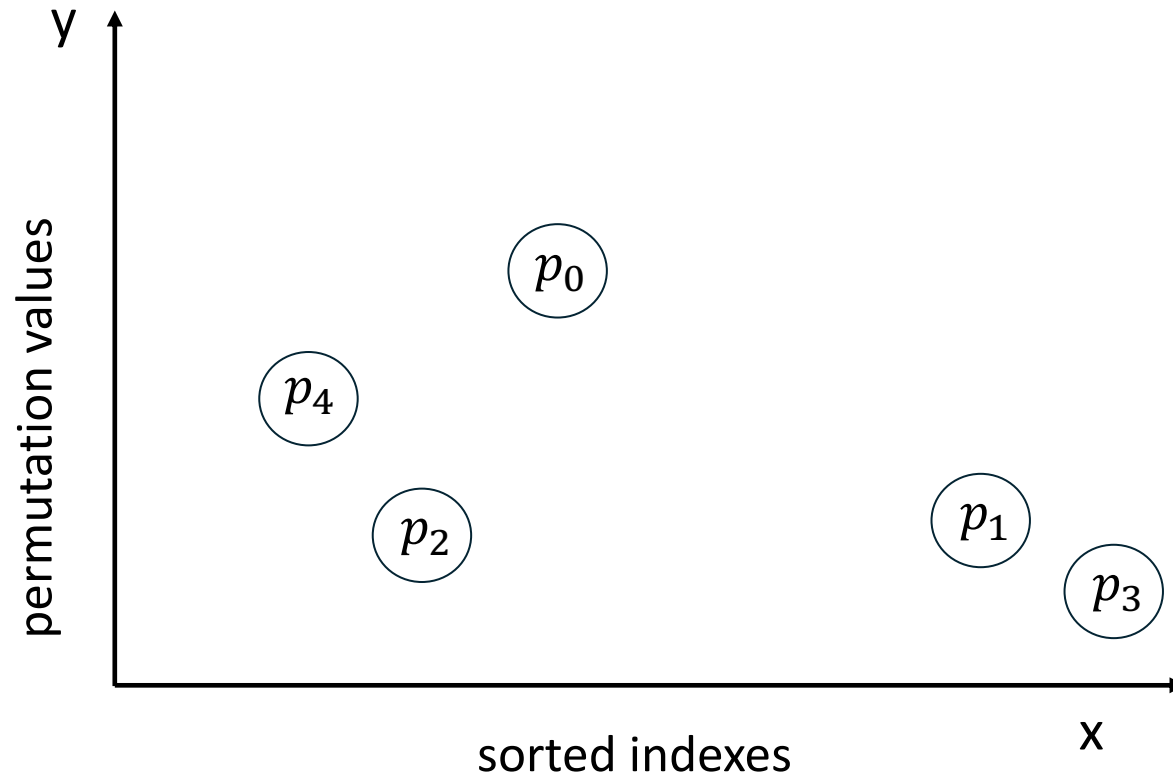
At T=256 each rank entry uses 2x bits compared to data entry!

Can we fix this?

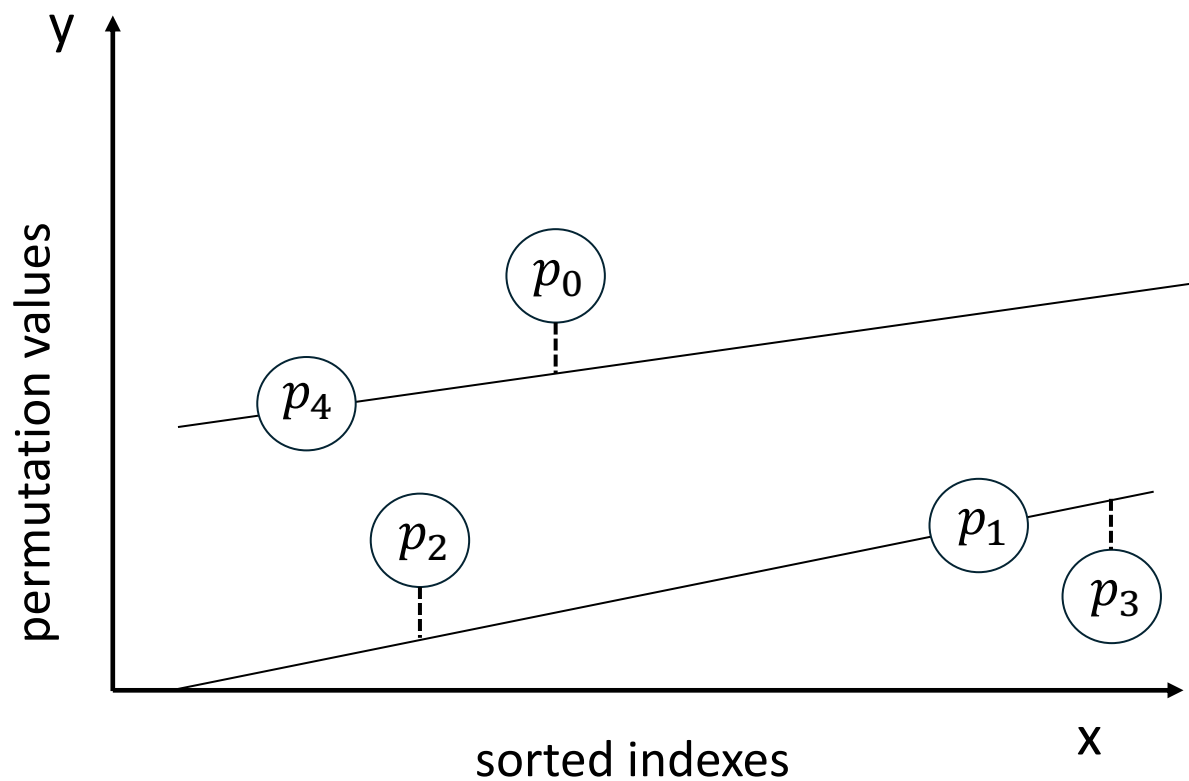
Rethinking the Mapping Structure



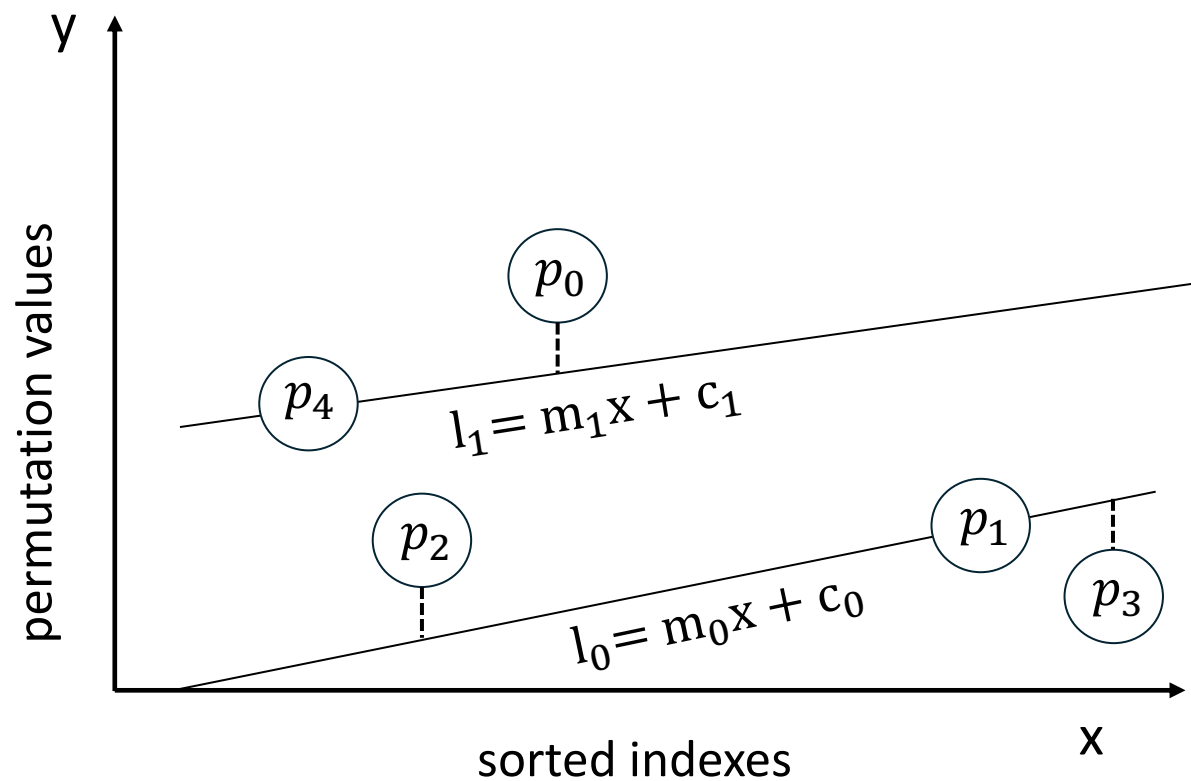
Mapping points in 2-D space



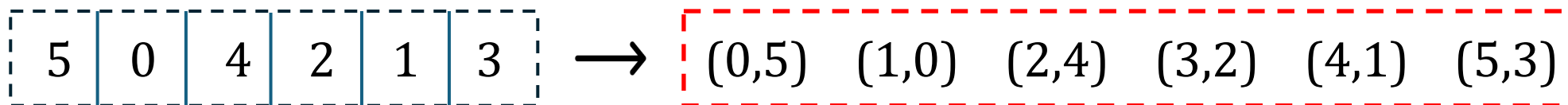
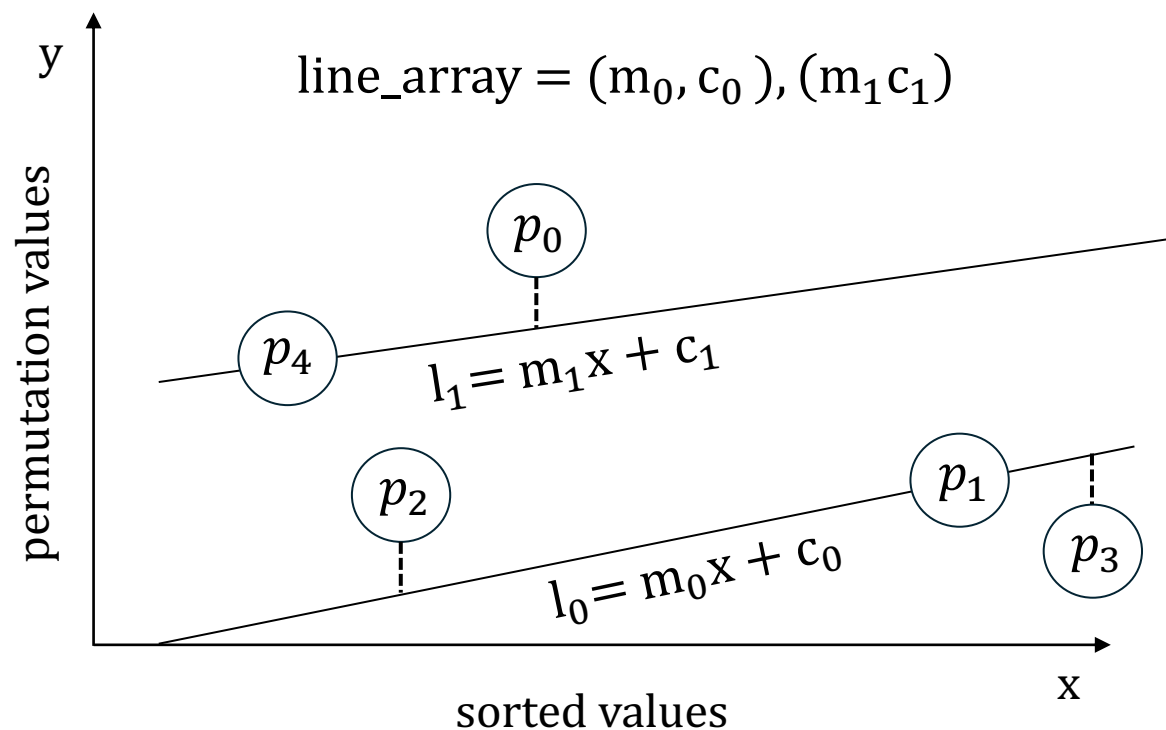
Constellation Maps



Constellation Maps

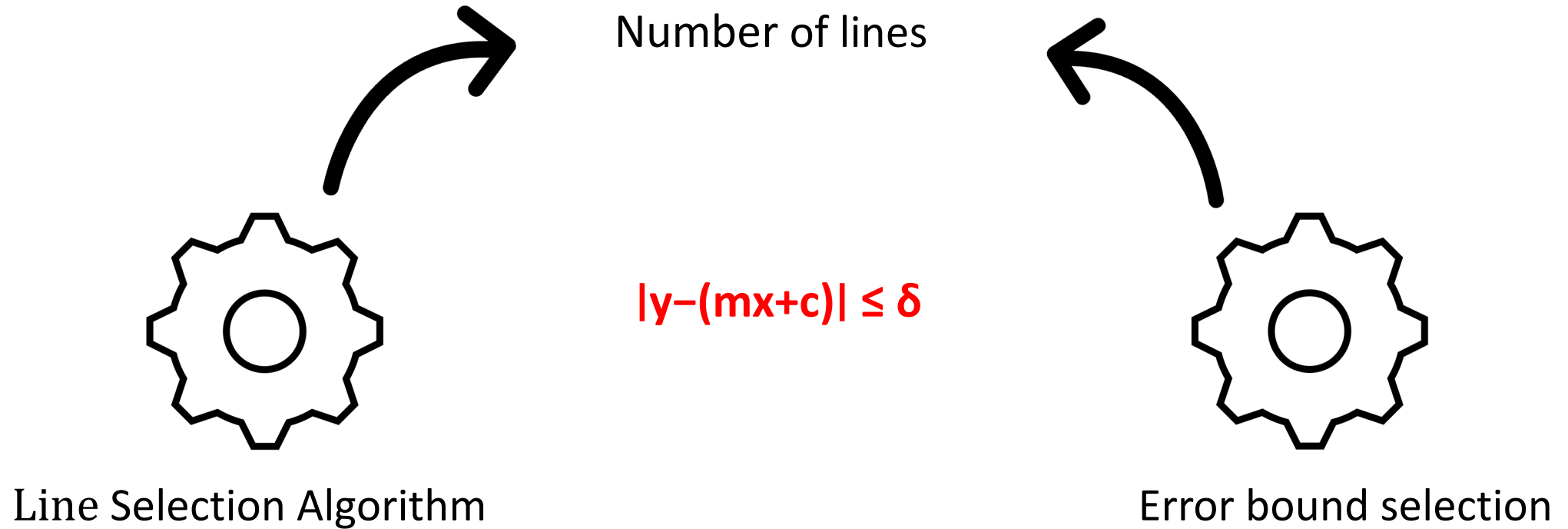


Constellation Maps

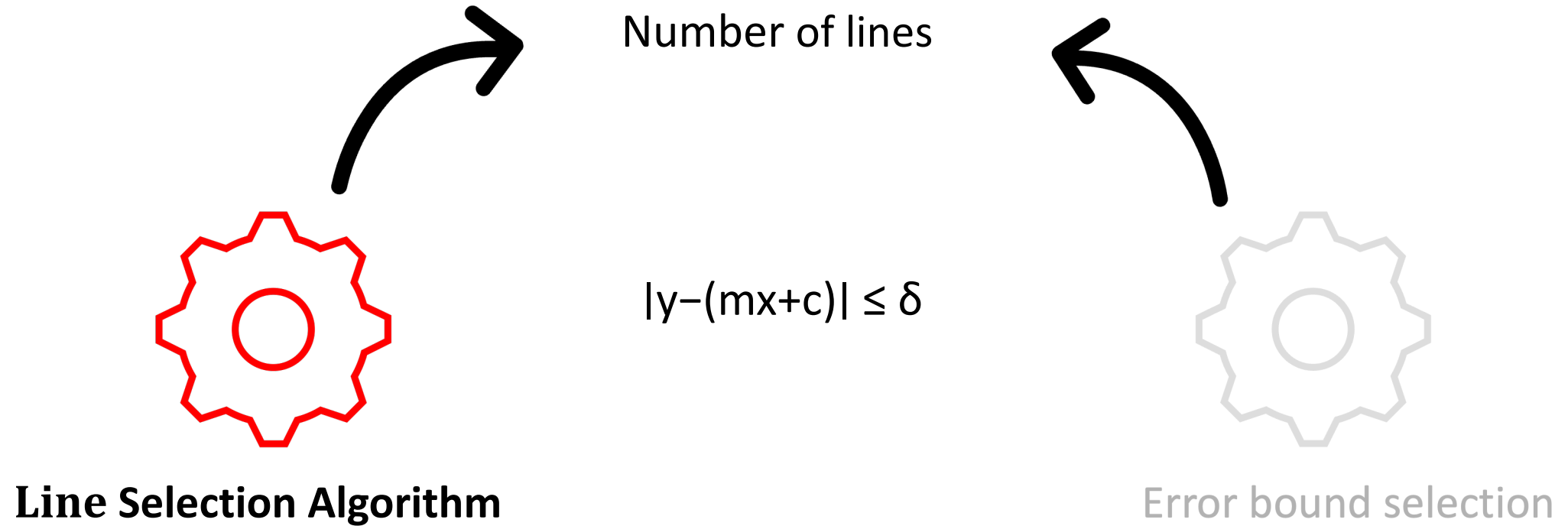


What design decisions exist?

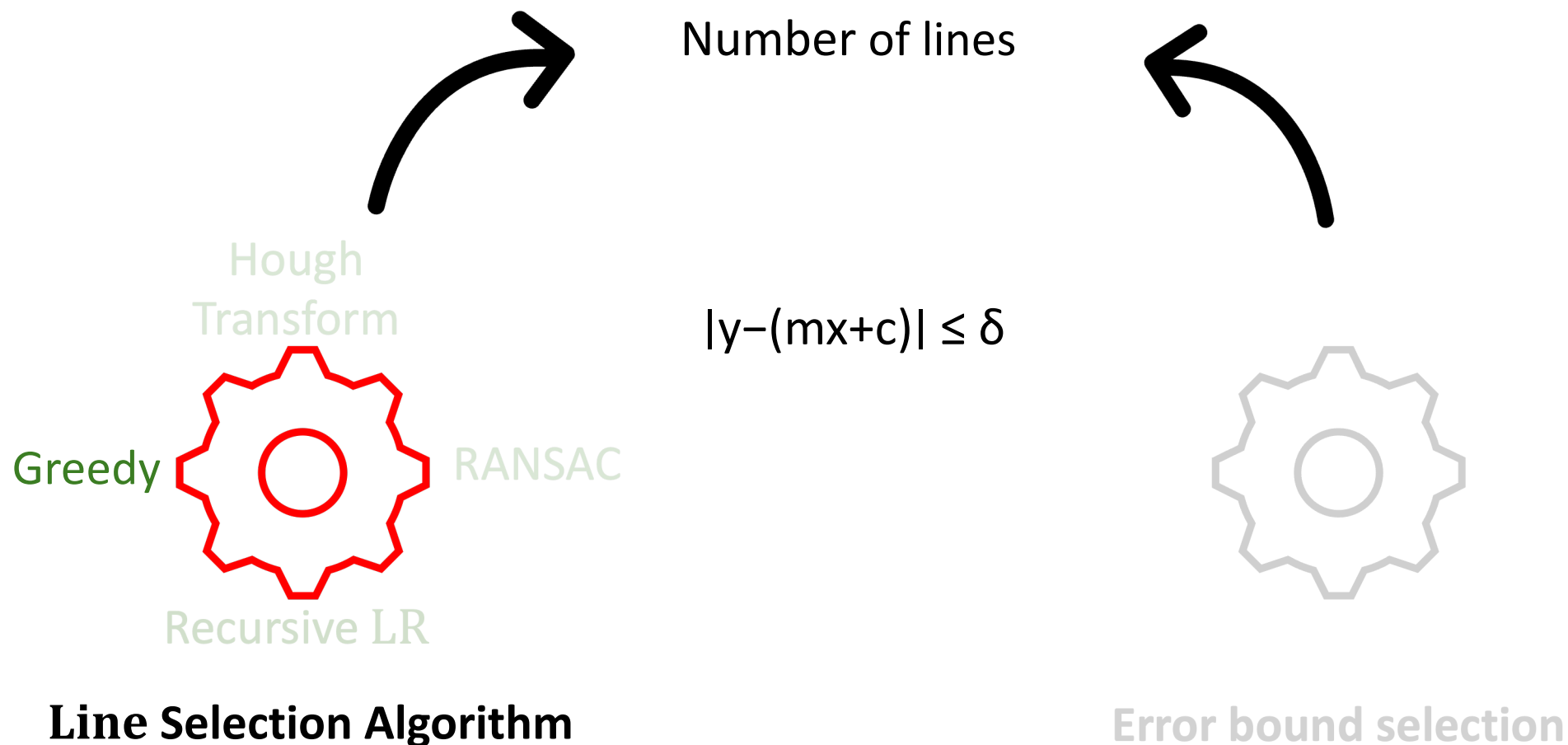
Design Considerations



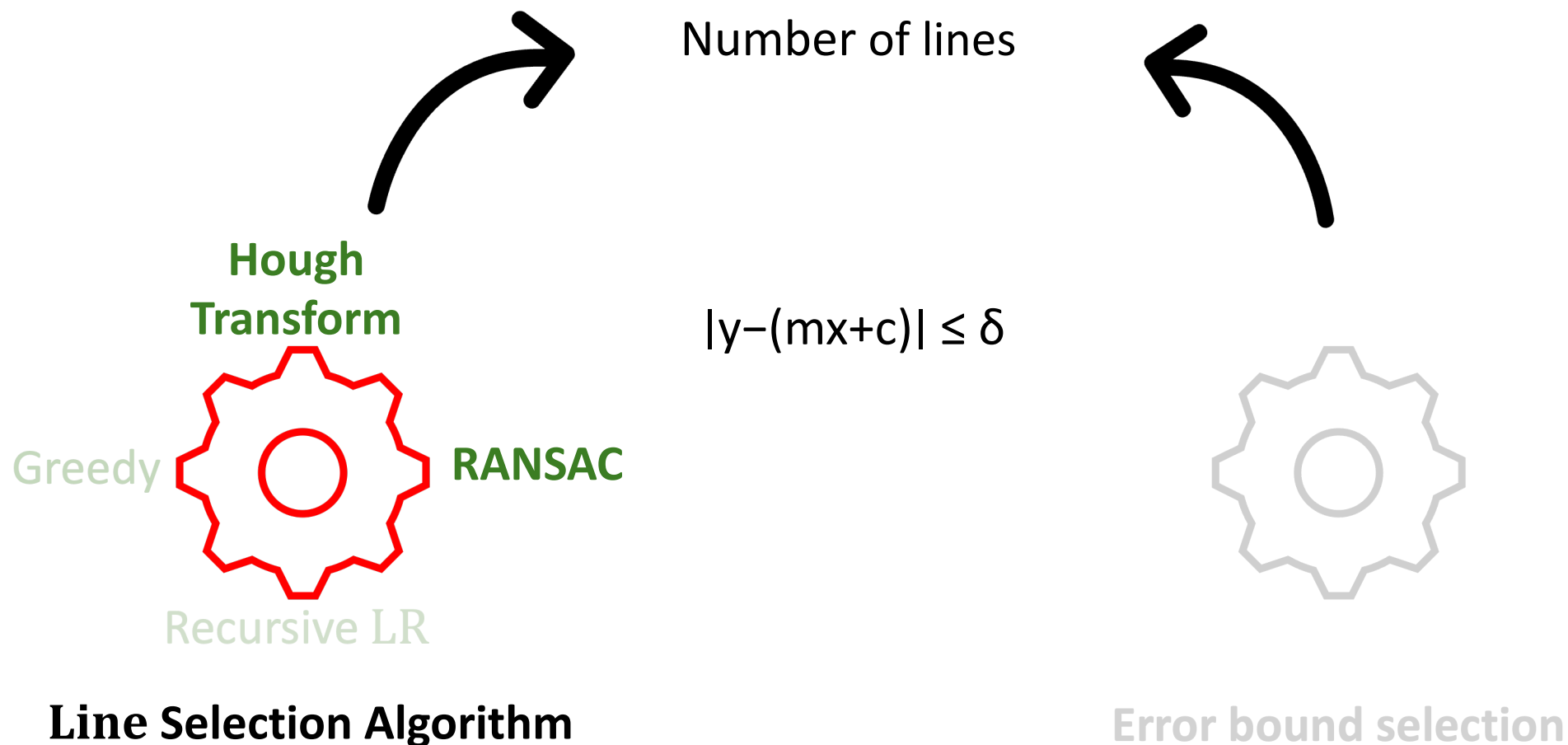
Design Considerations



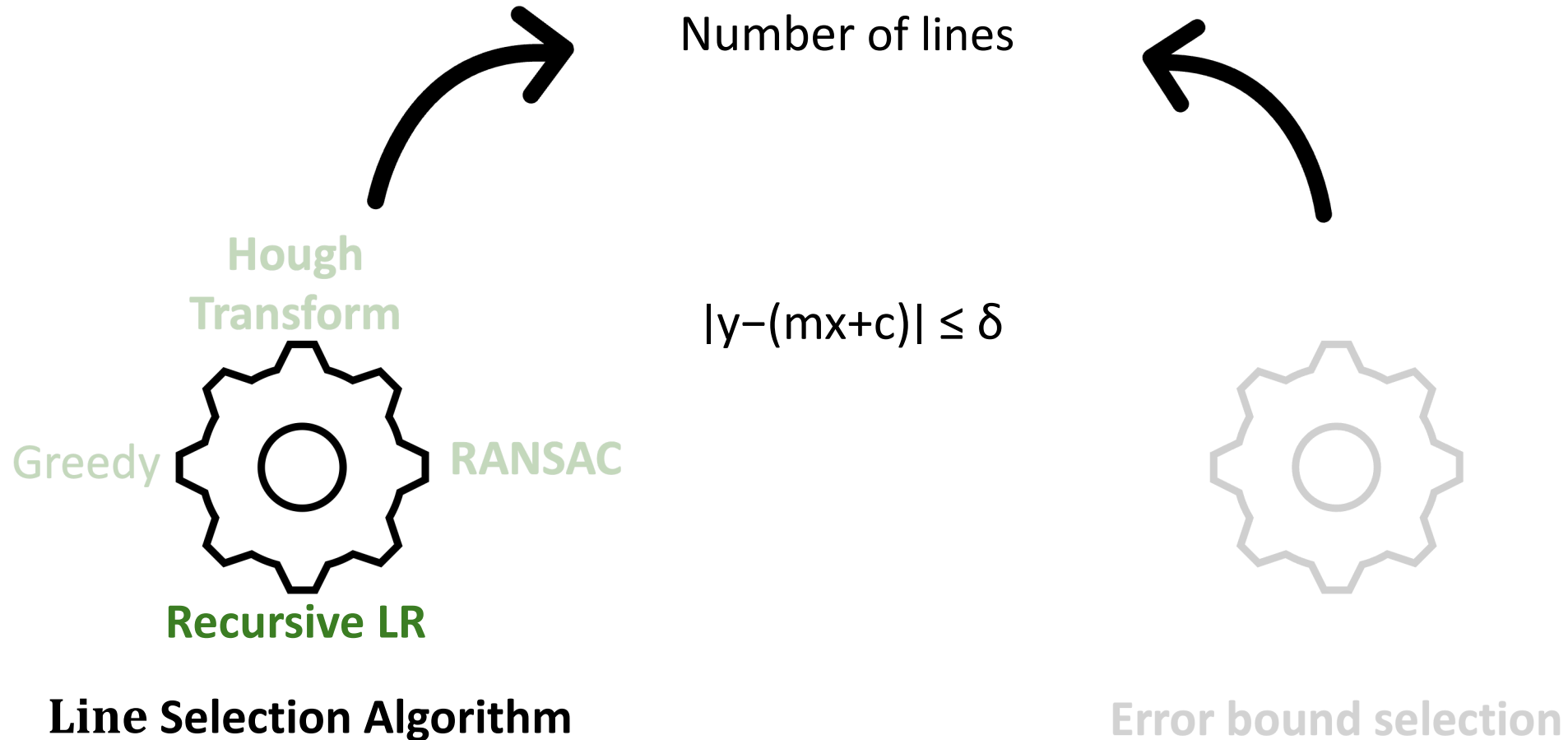
Design Considerations



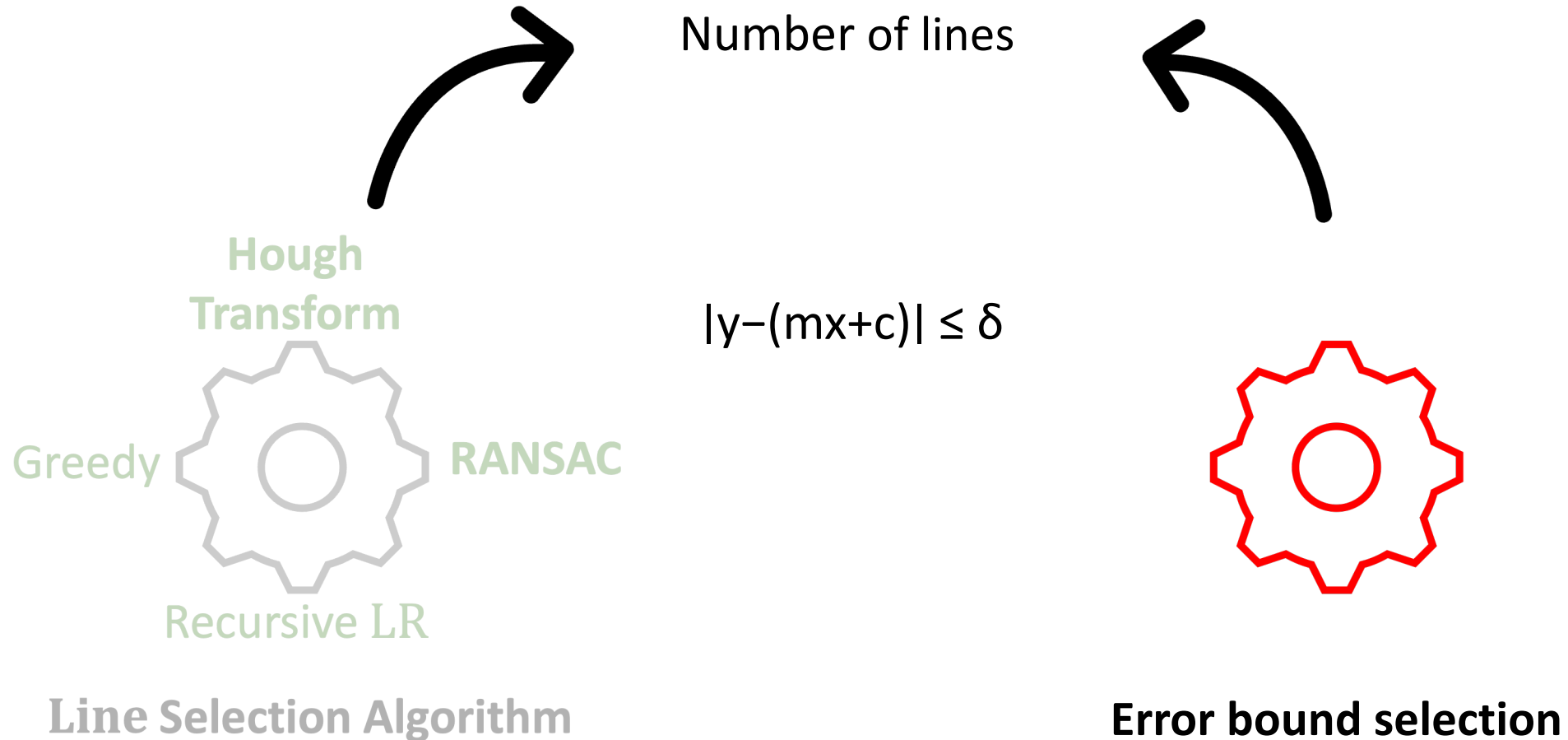
Design Considerations



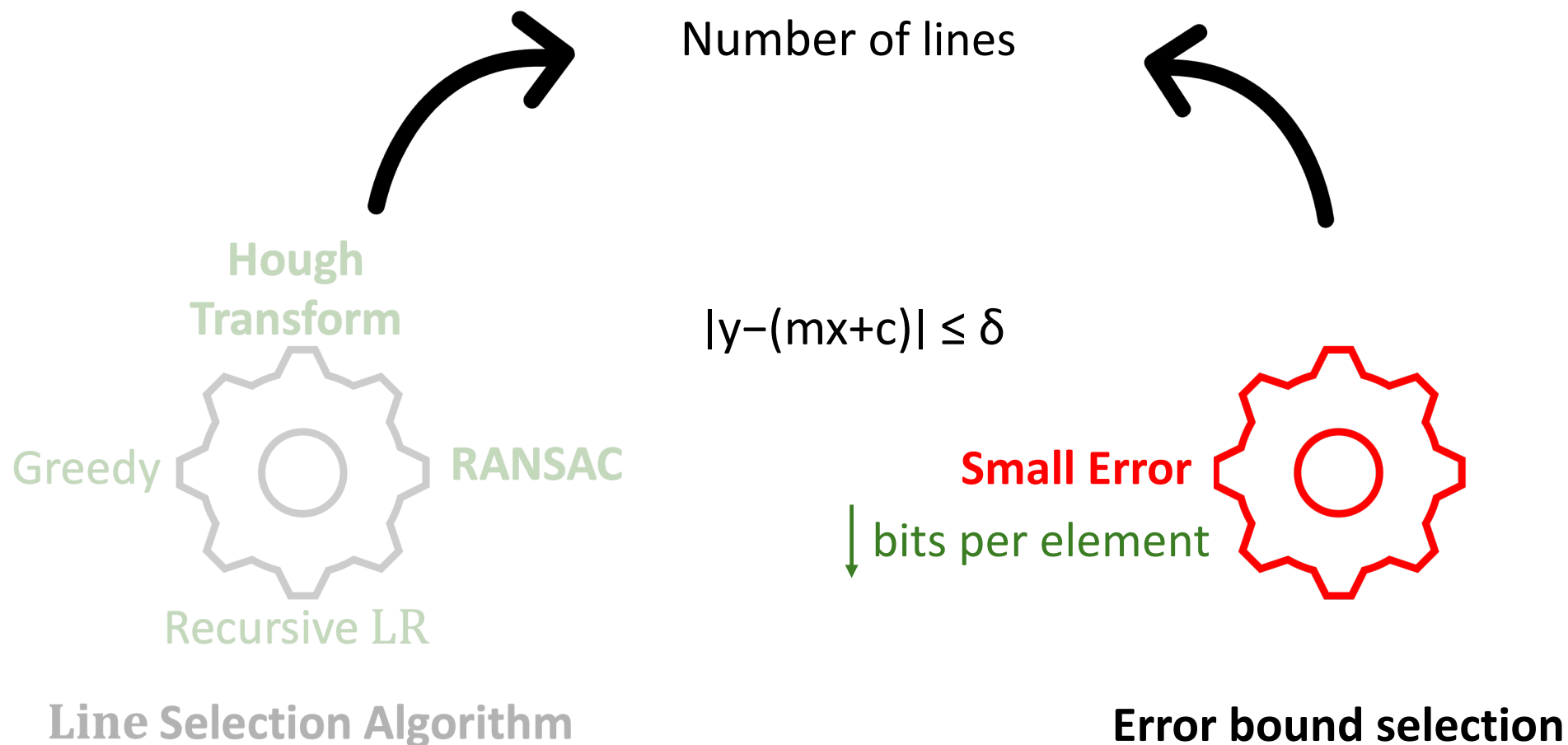
Design Considerations



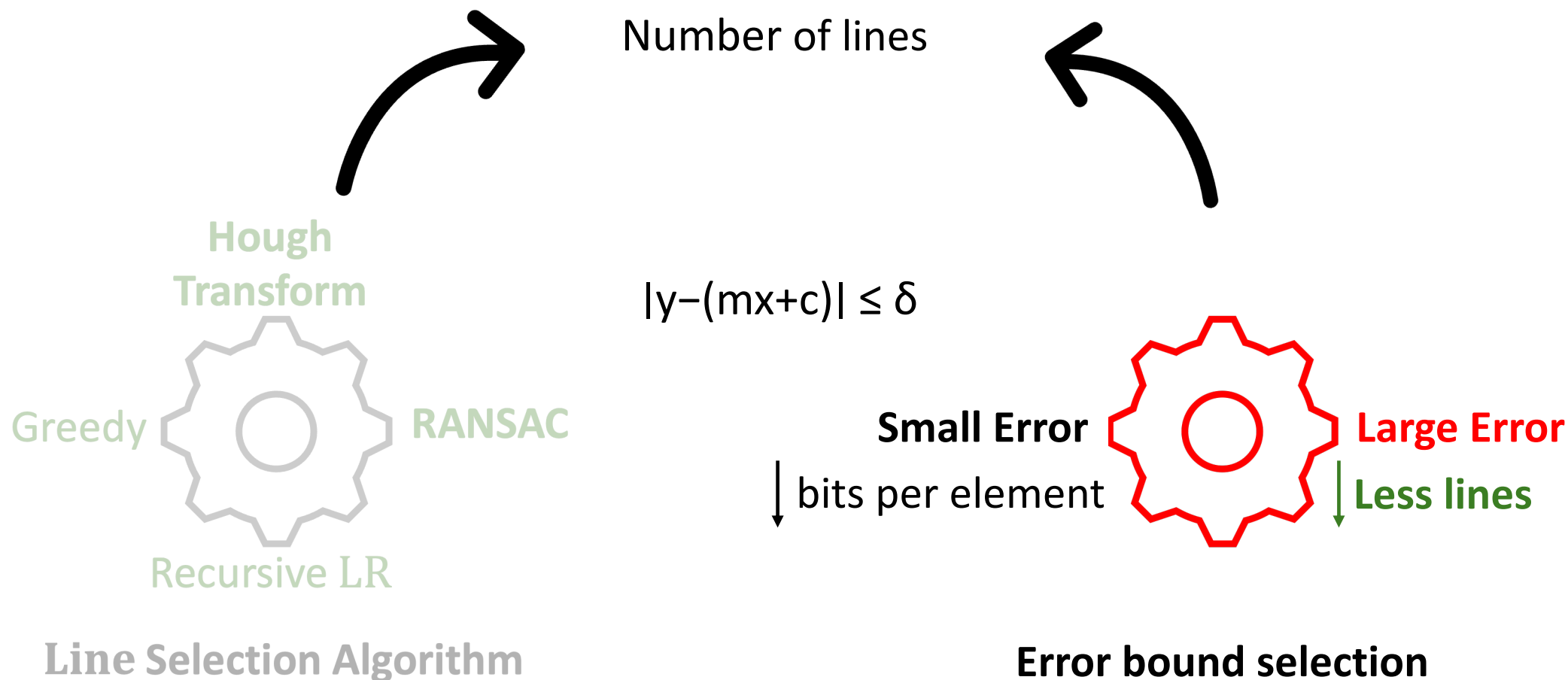
Design Considerations



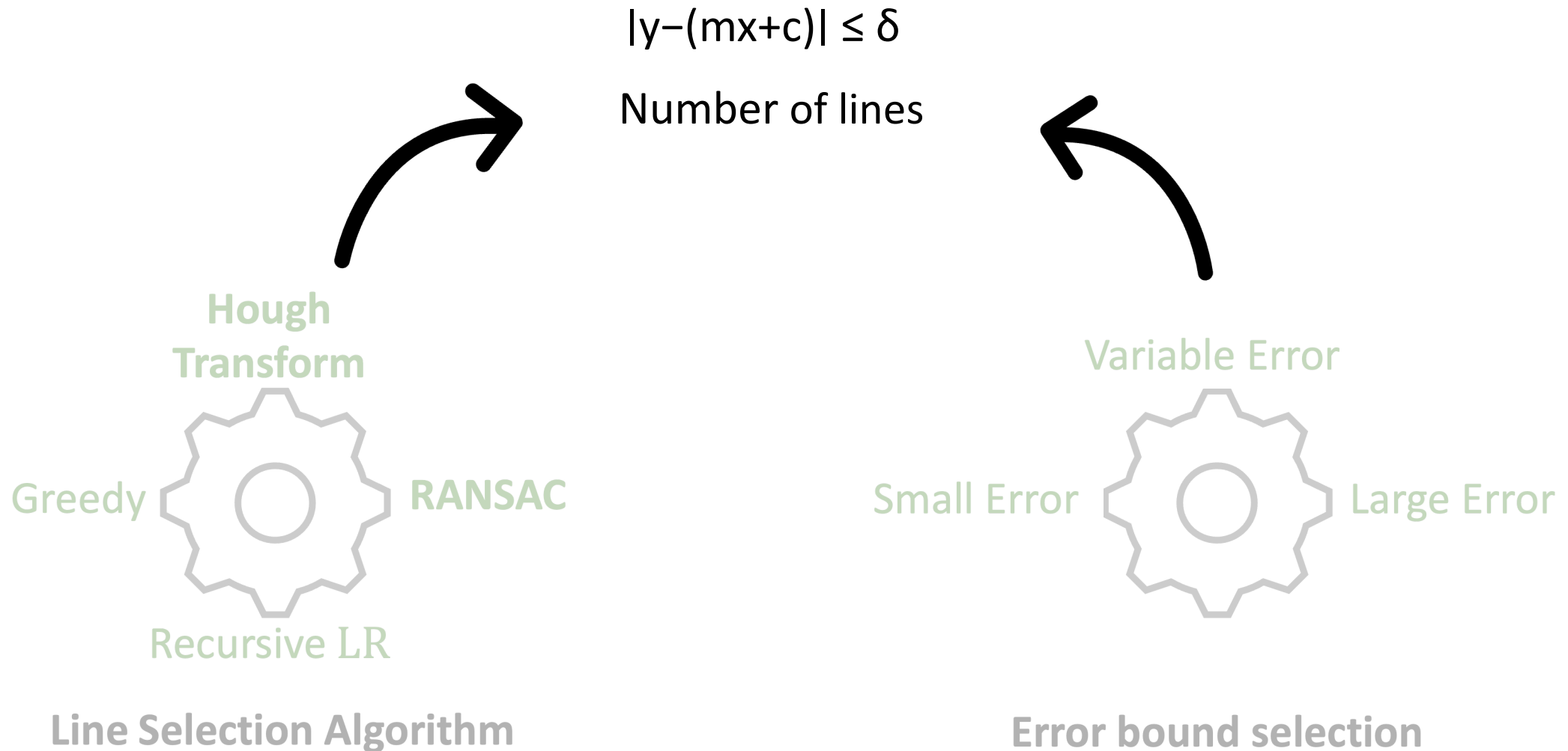
Design Considerations



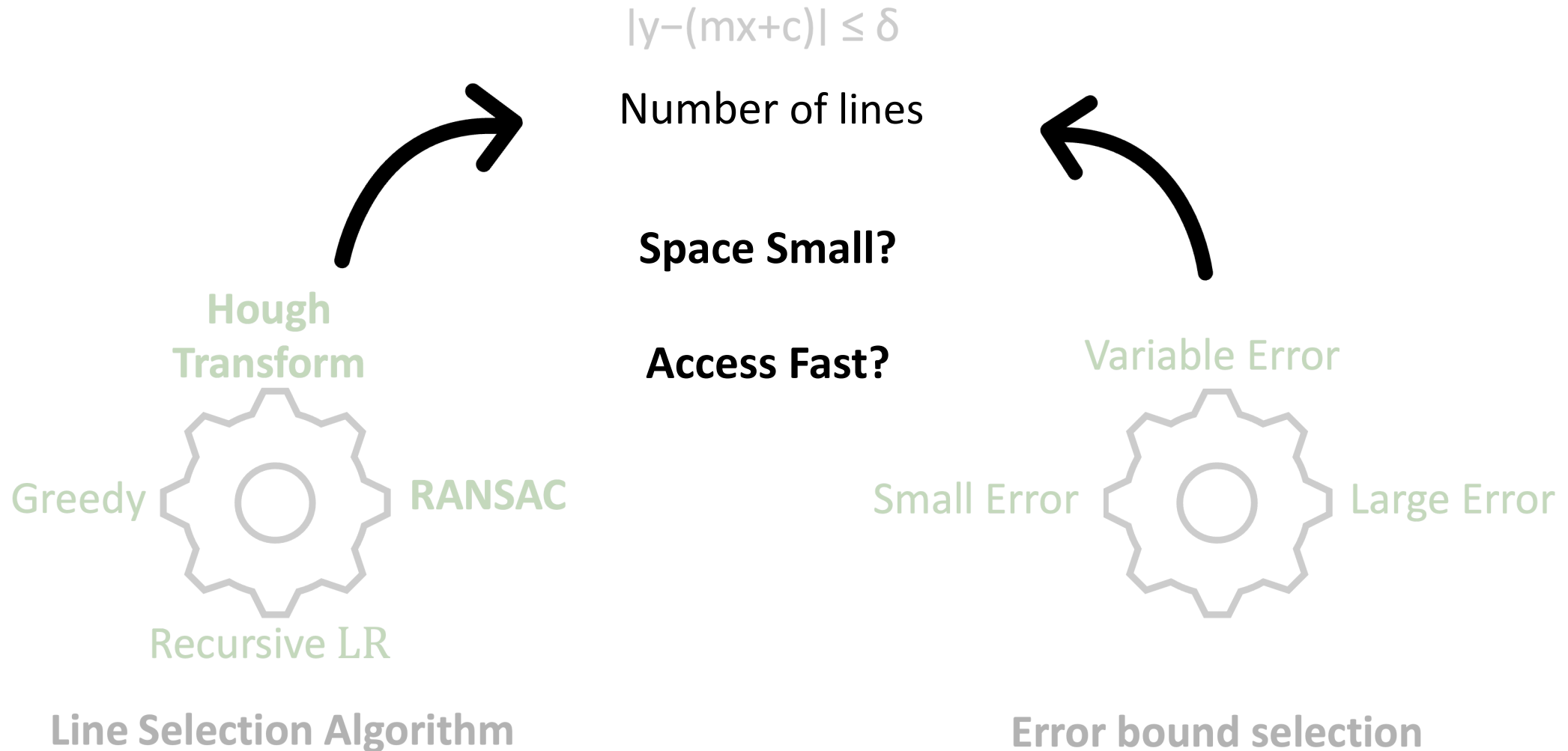
Design Considerations



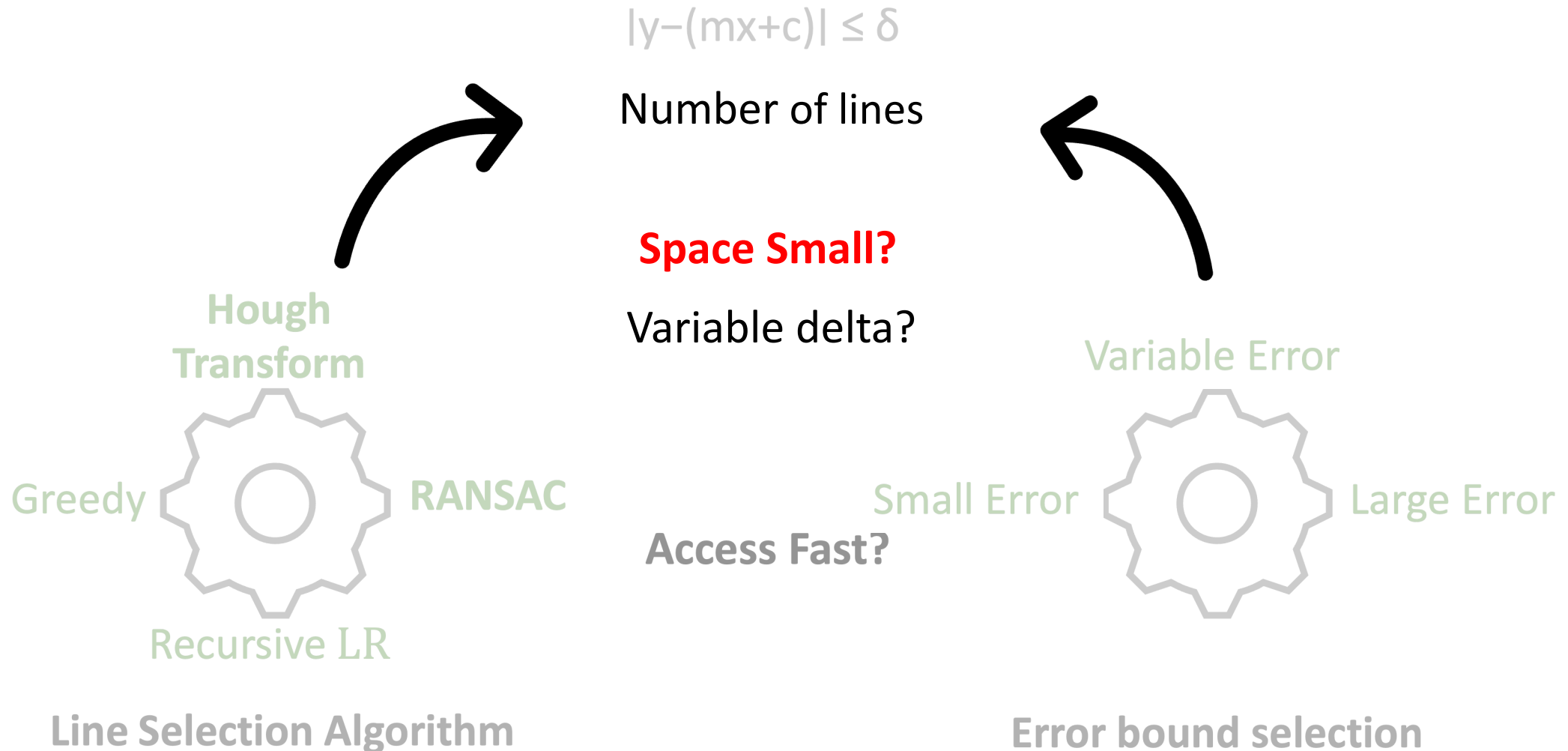
Are there other considerations?



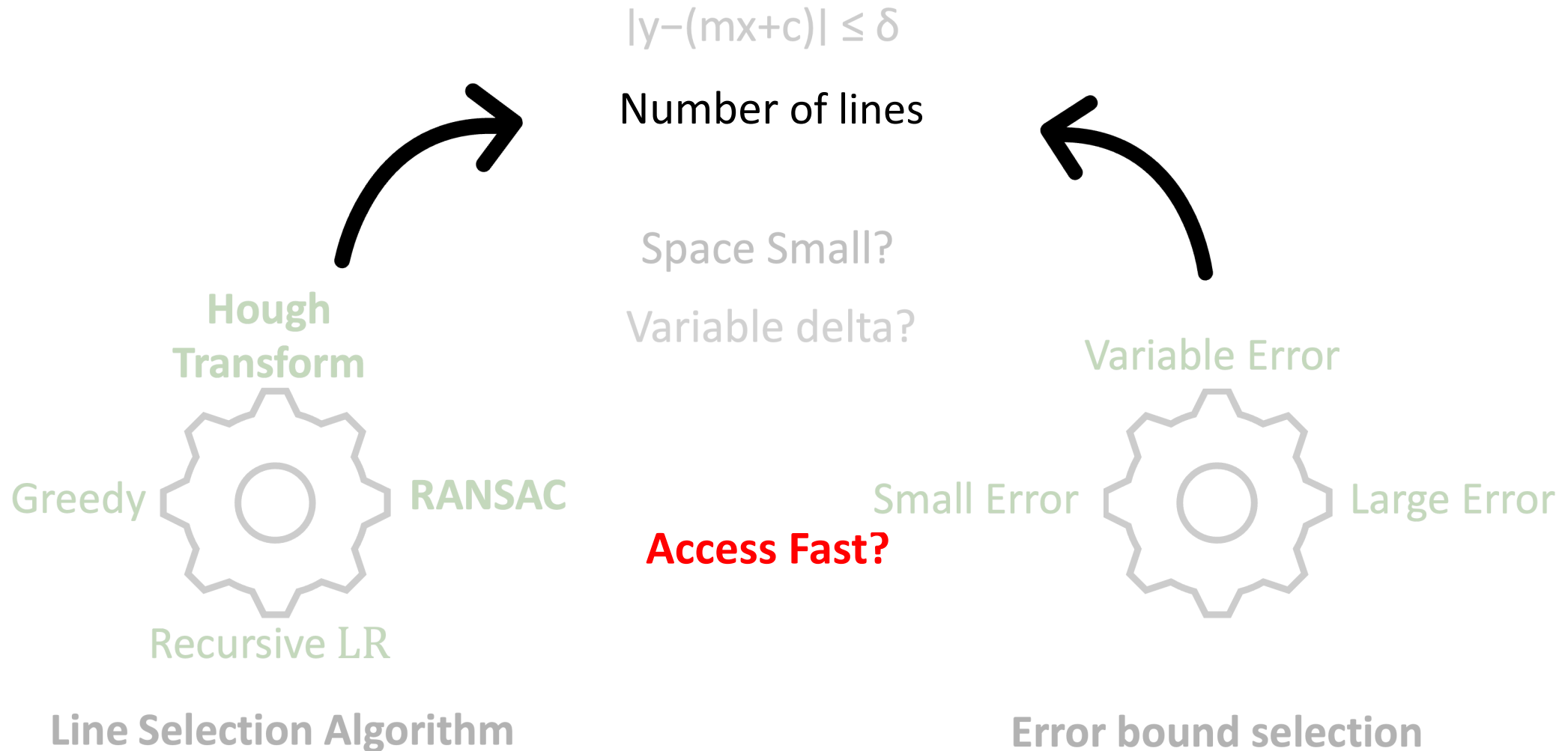
Are there other considerations?



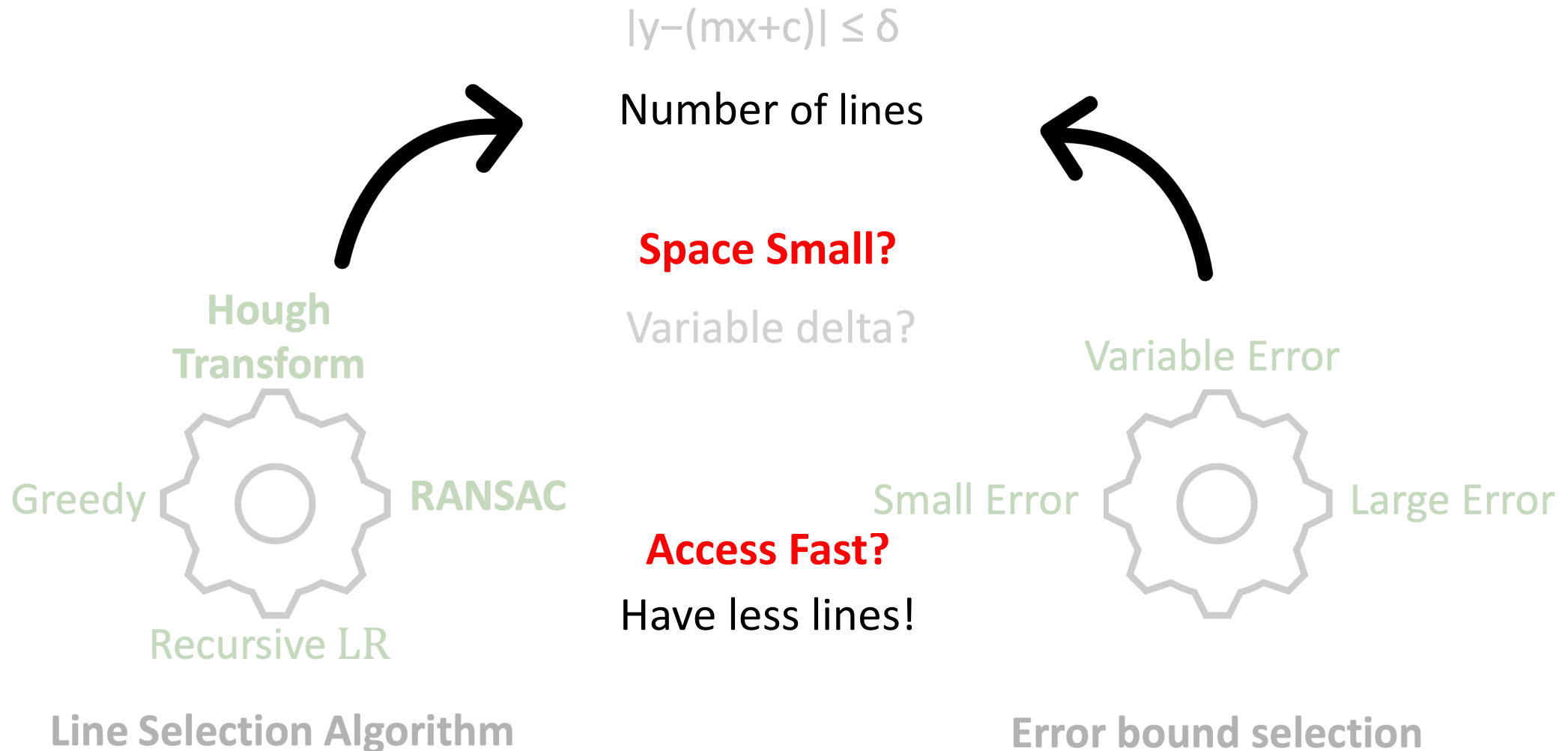
Are there other considerations?



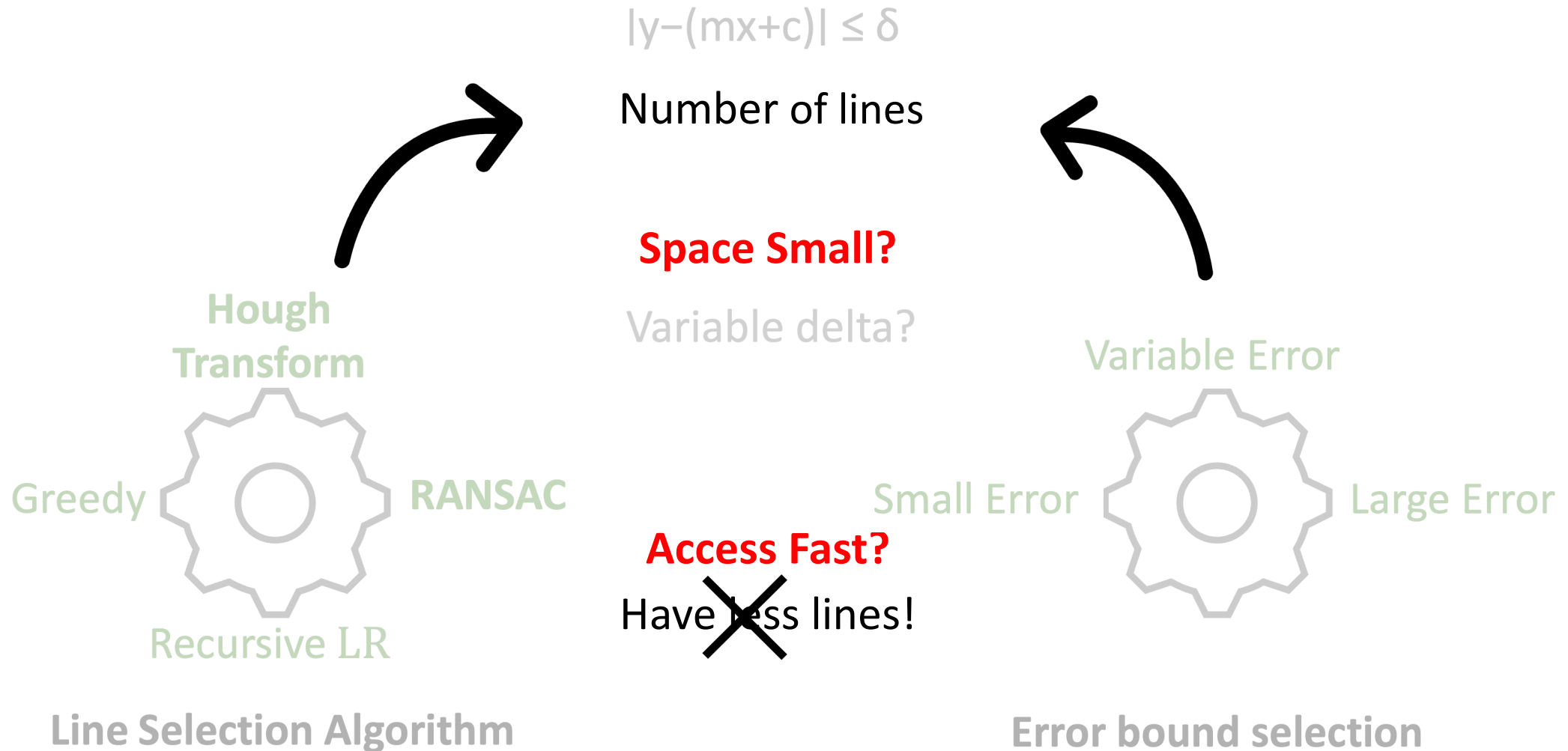
Are there other considerations?



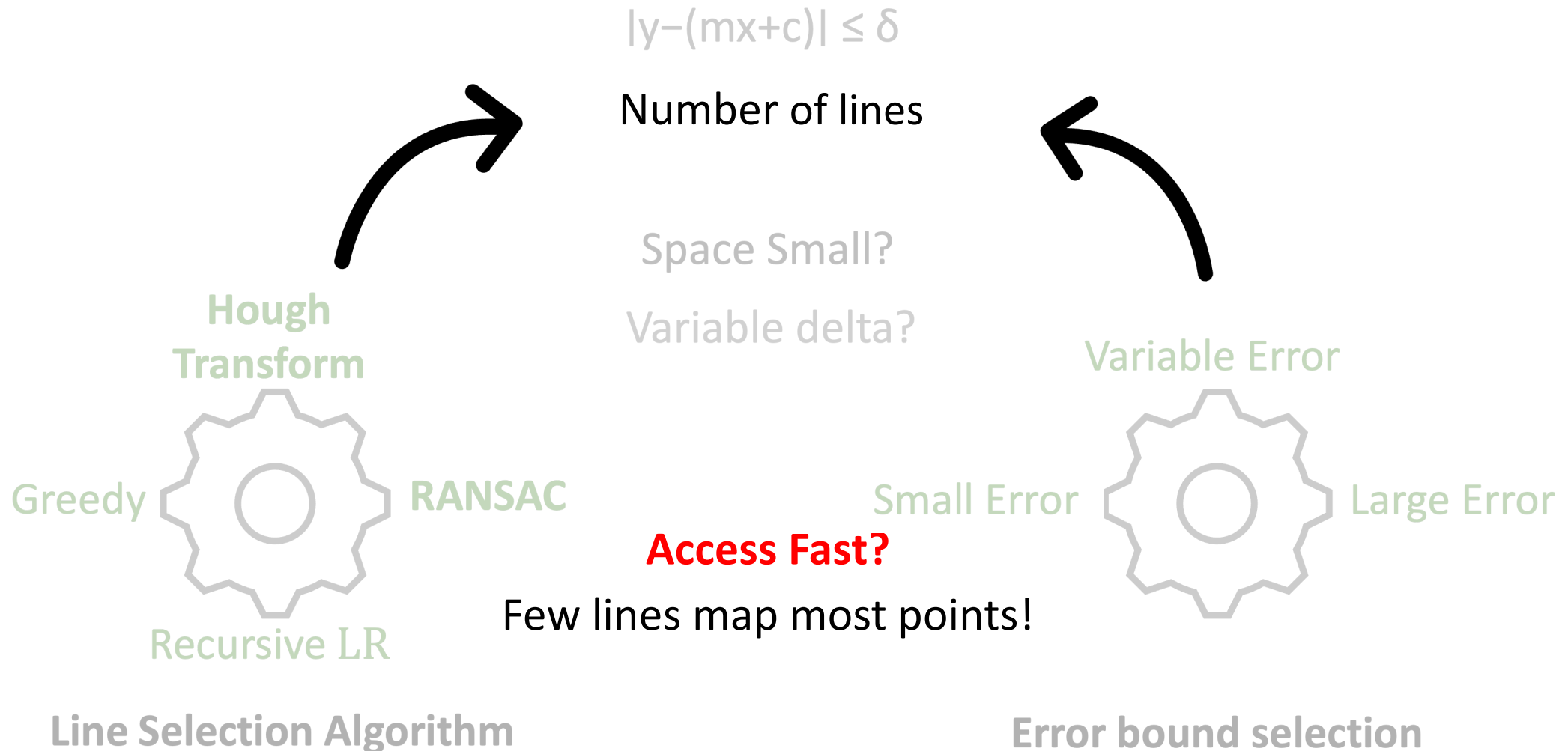
Are there other considerations?



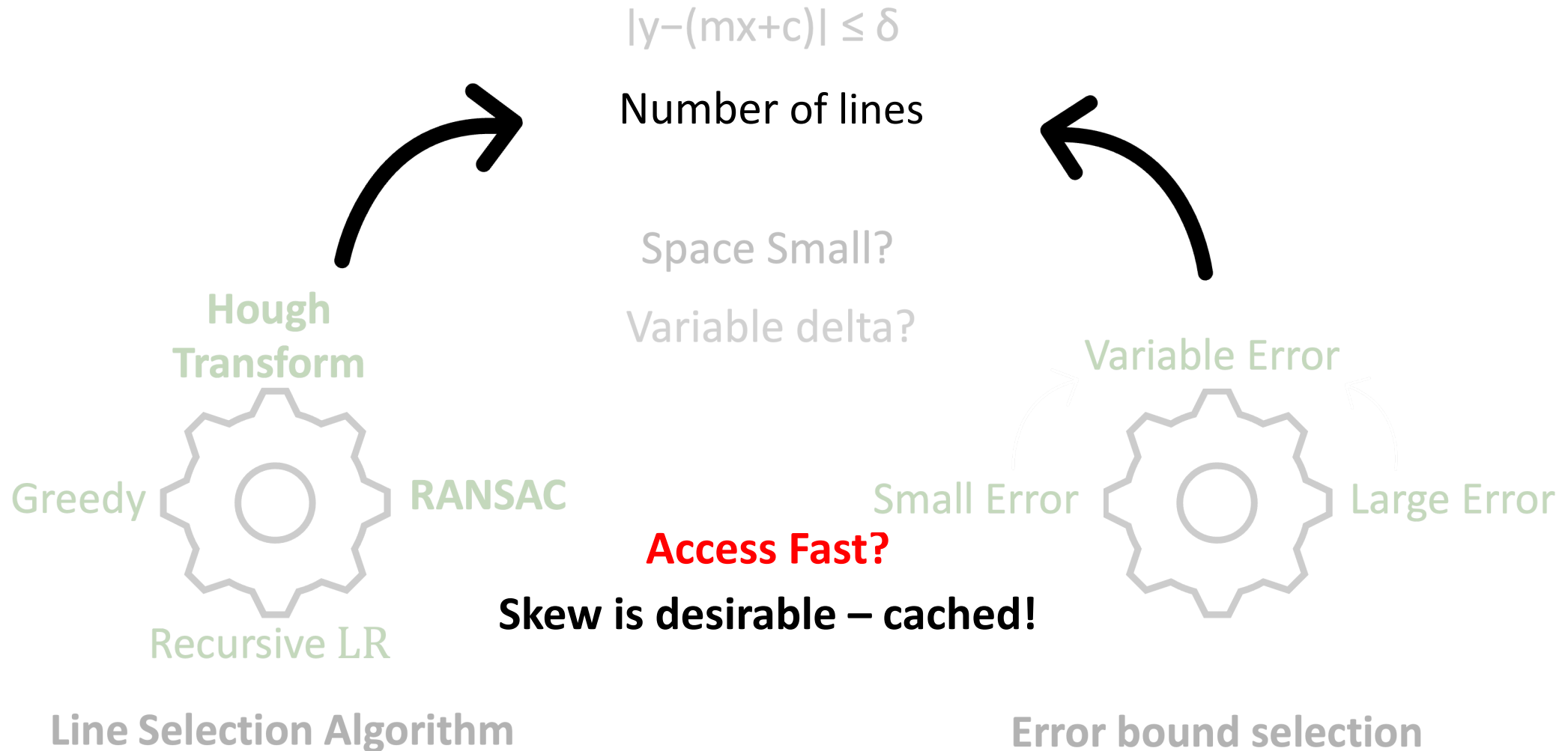
Are there other considerations?



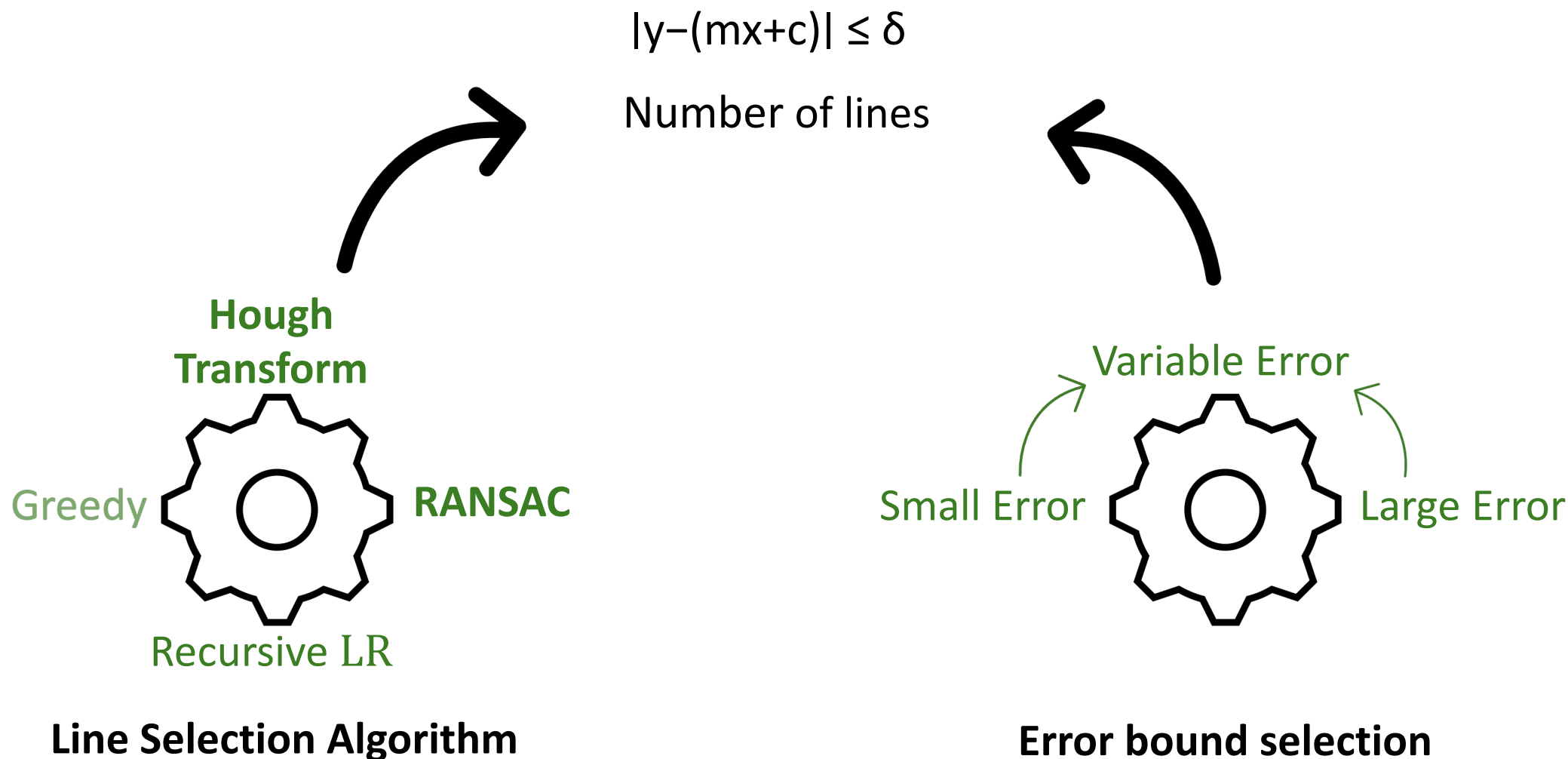
Are there other considerations?



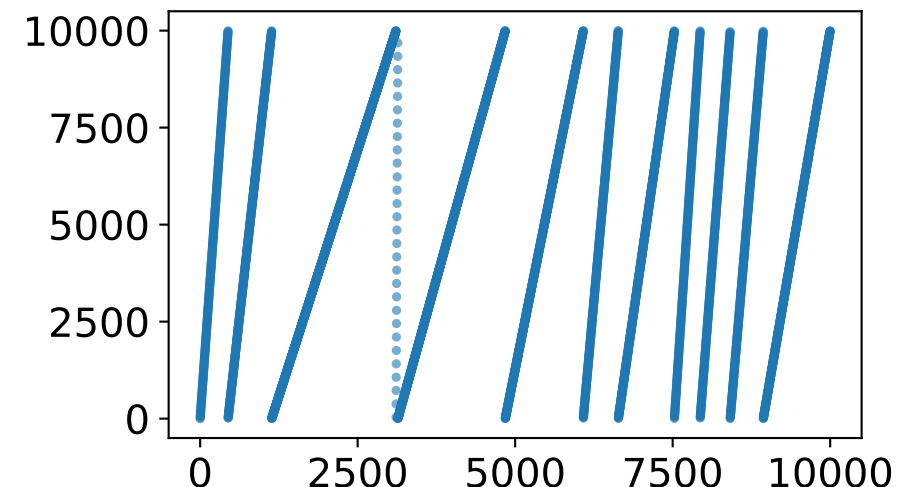
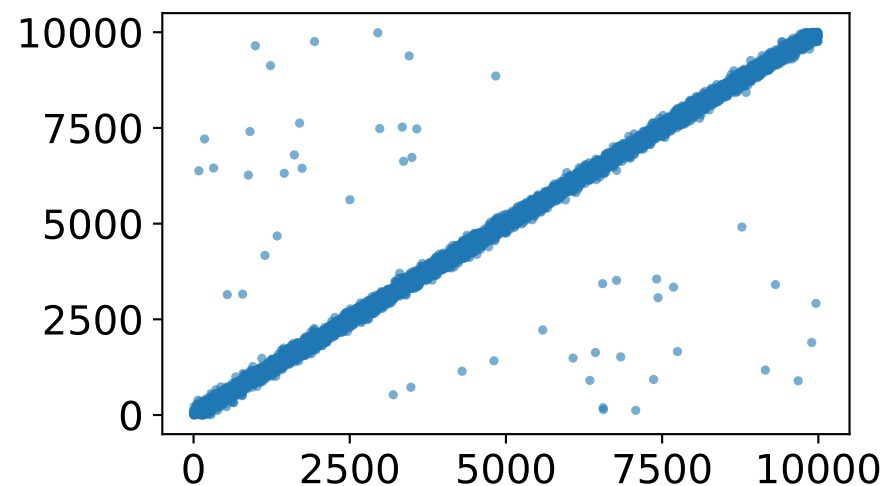
Are there other considerations?



Are there other considerations?

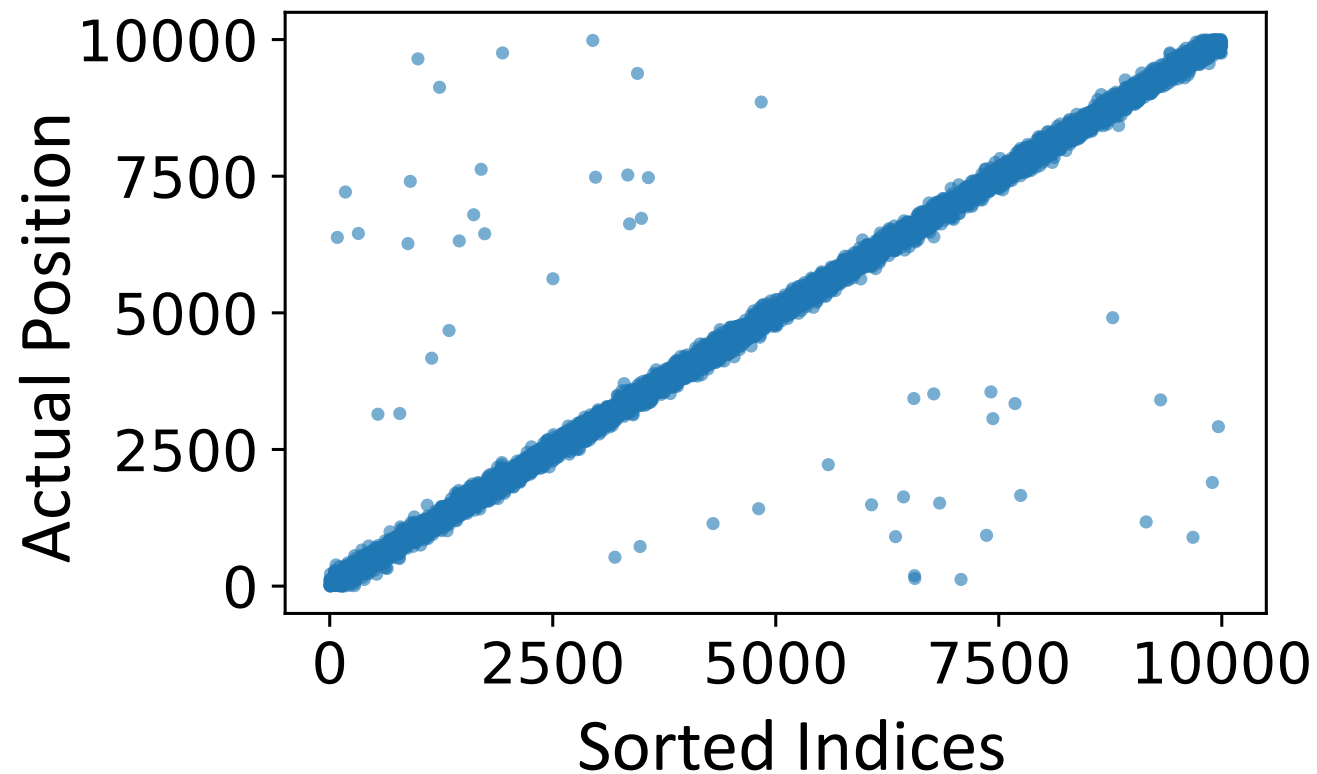


Constellation Maps in Action

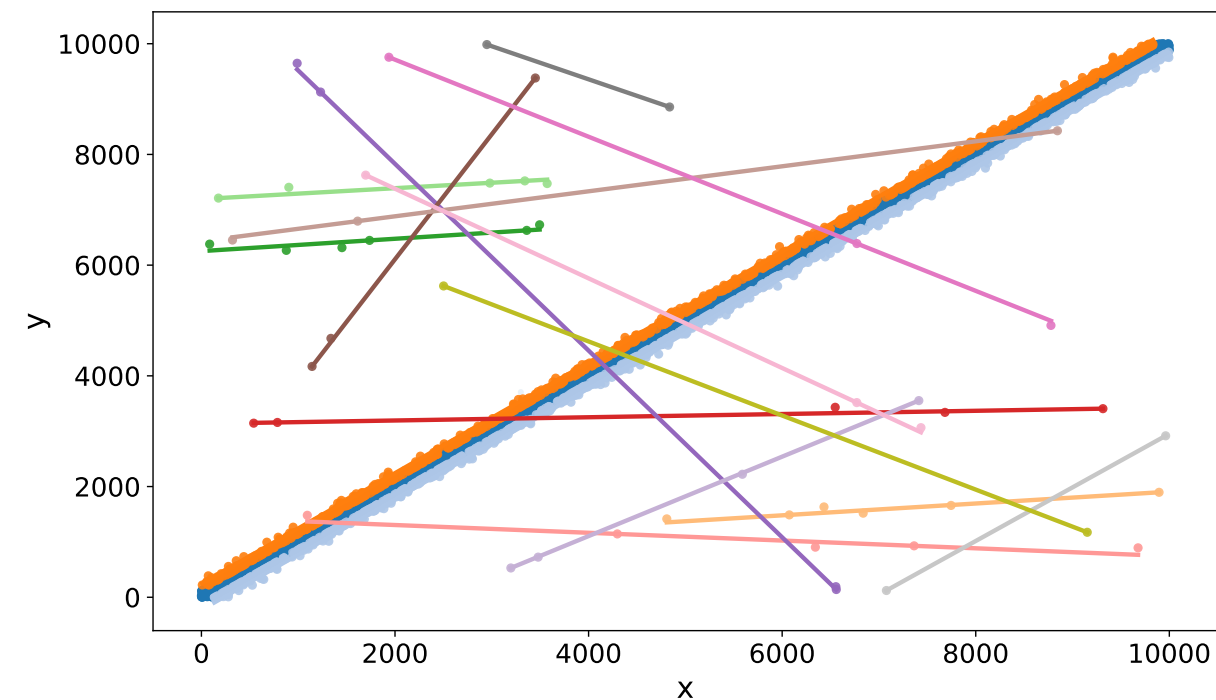


Datasets were donated by an anonymous corporation

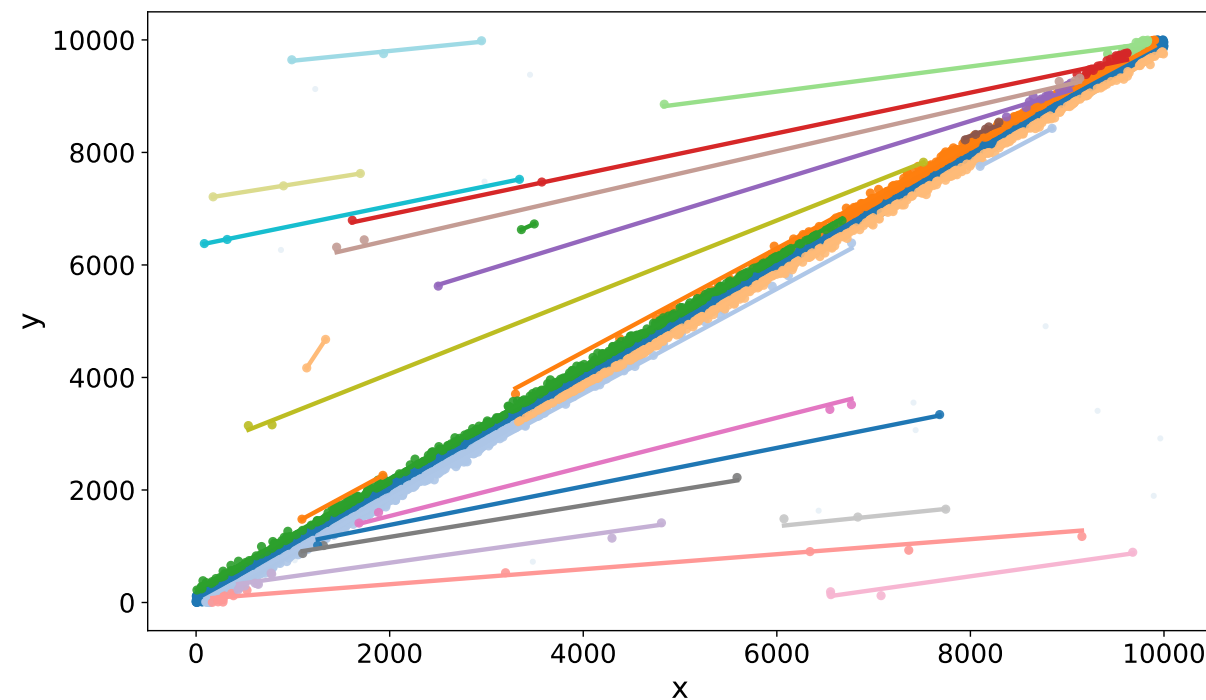
Constellation Maps in Action



Constellation Maps in Action

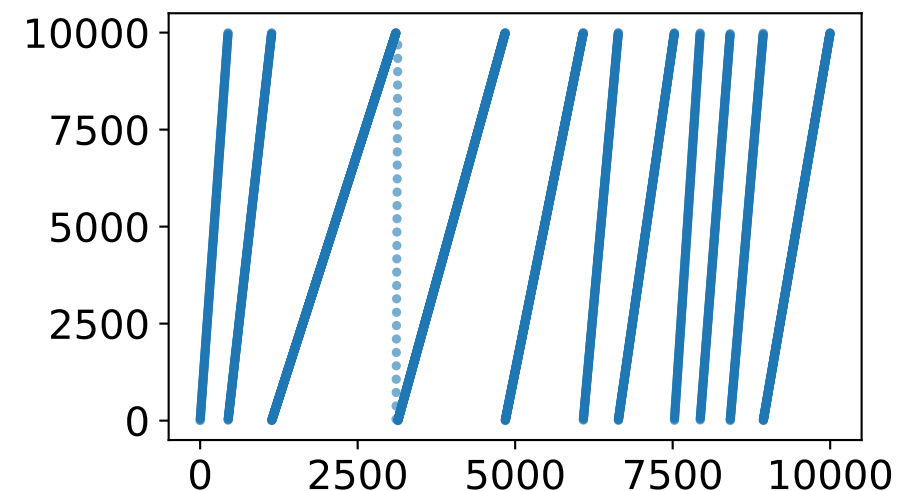
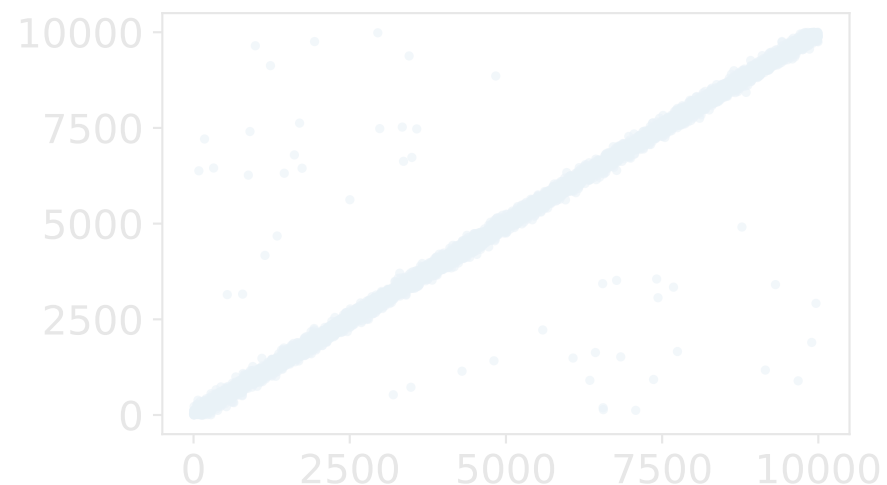


RANSAC = 17 lines

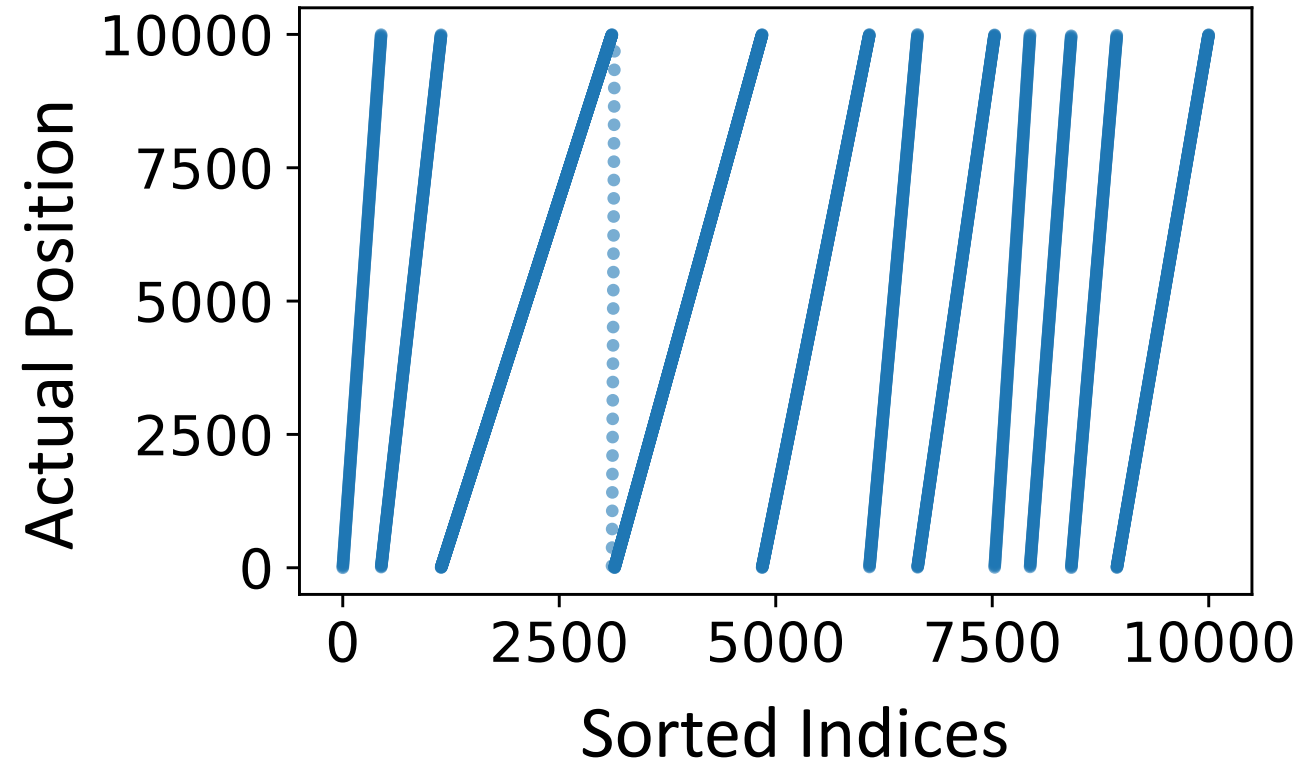


LR = 29 lines

Constellation Maps in Action

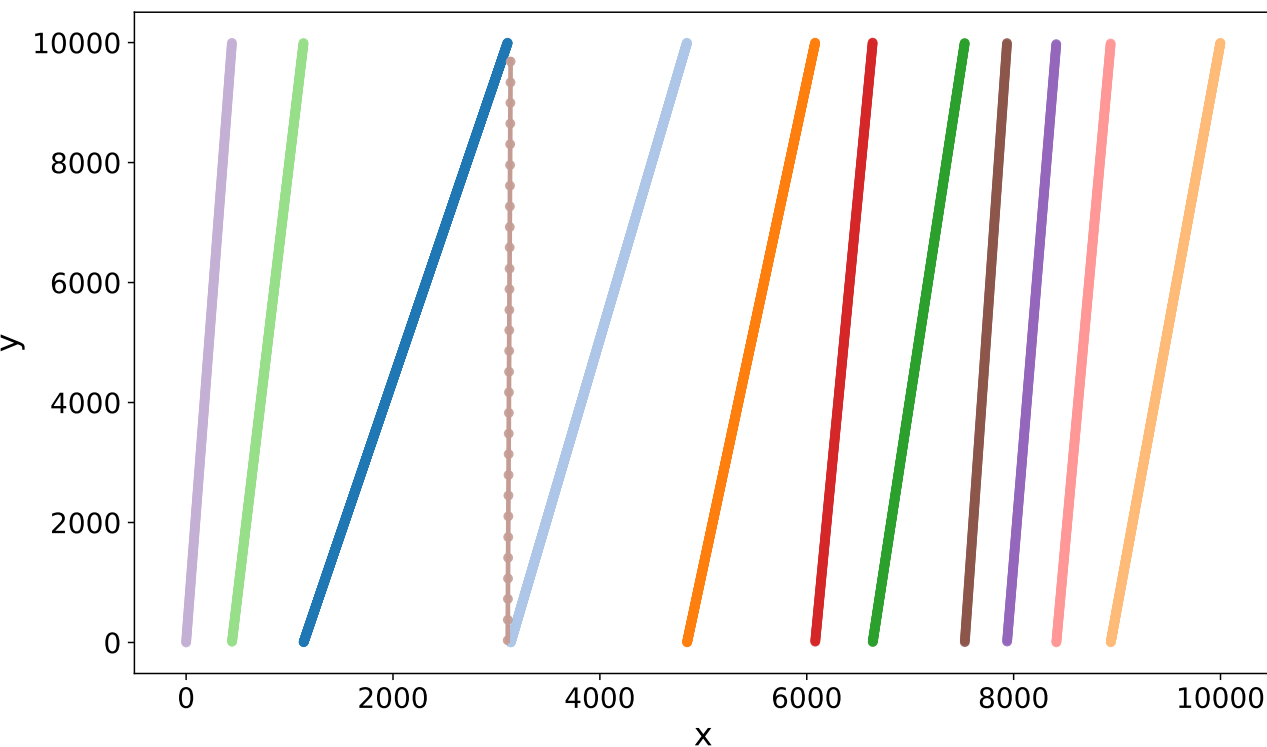


Constellation Maps in Action

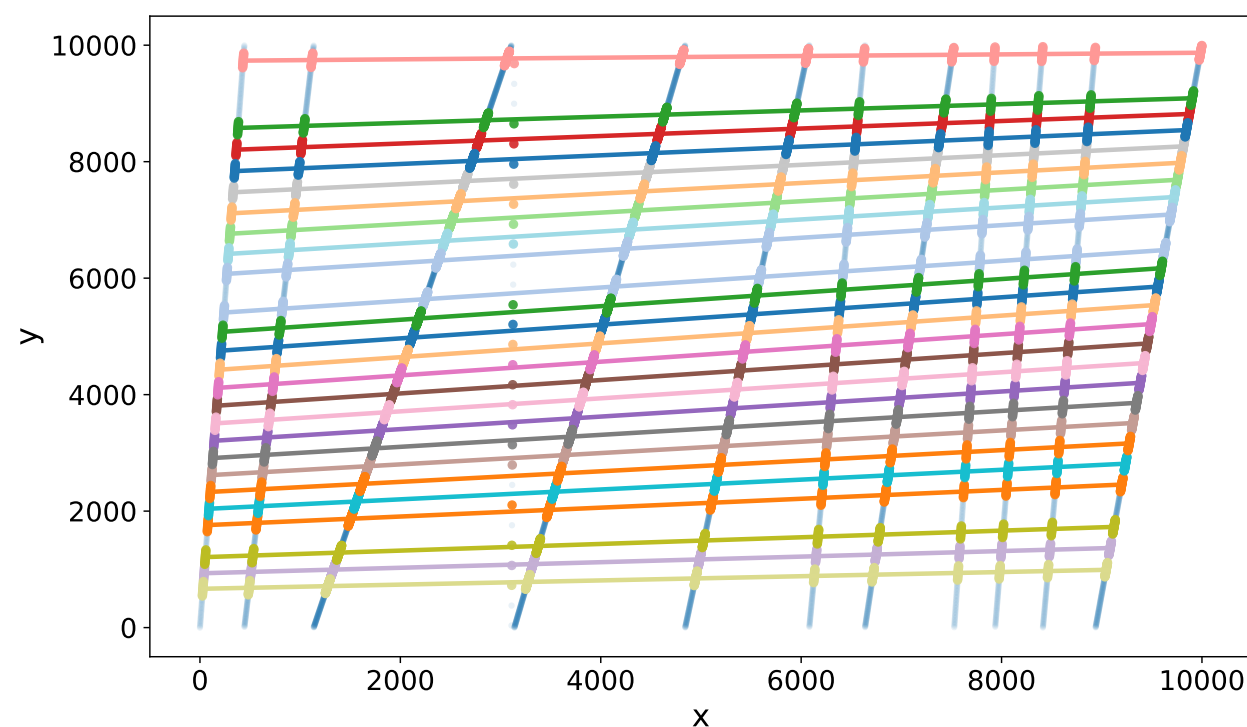


Datasets were donated by an anonymous corporation

Constellation Maps in Action

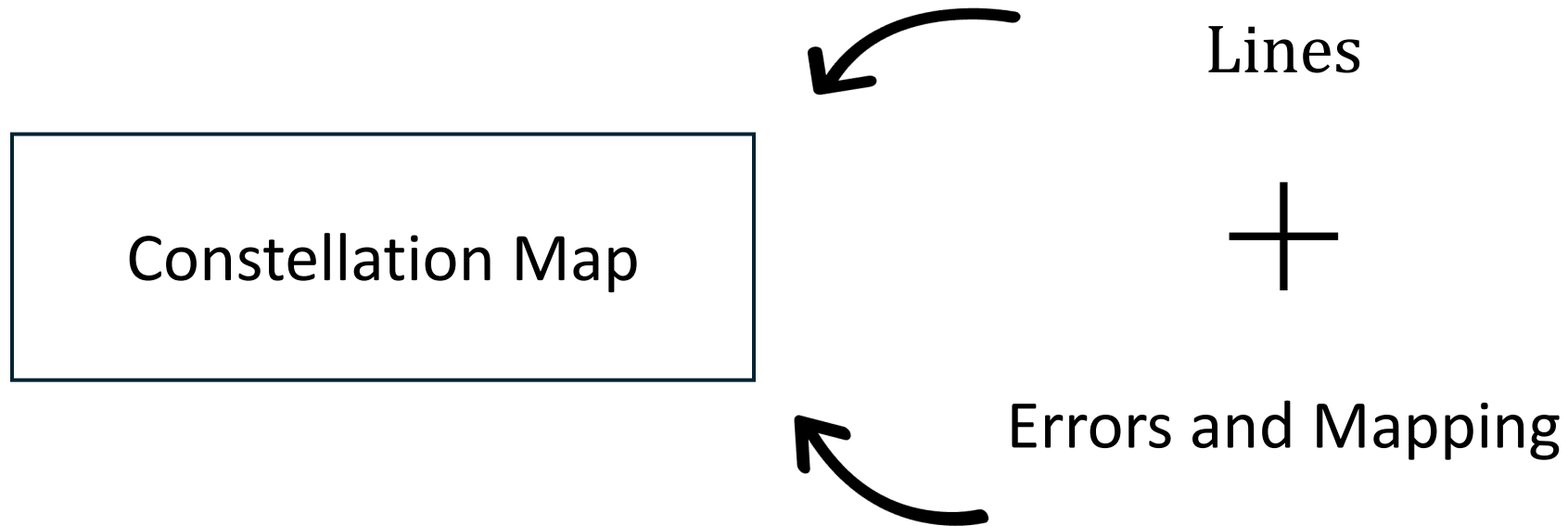


RANSAC = 12 lines

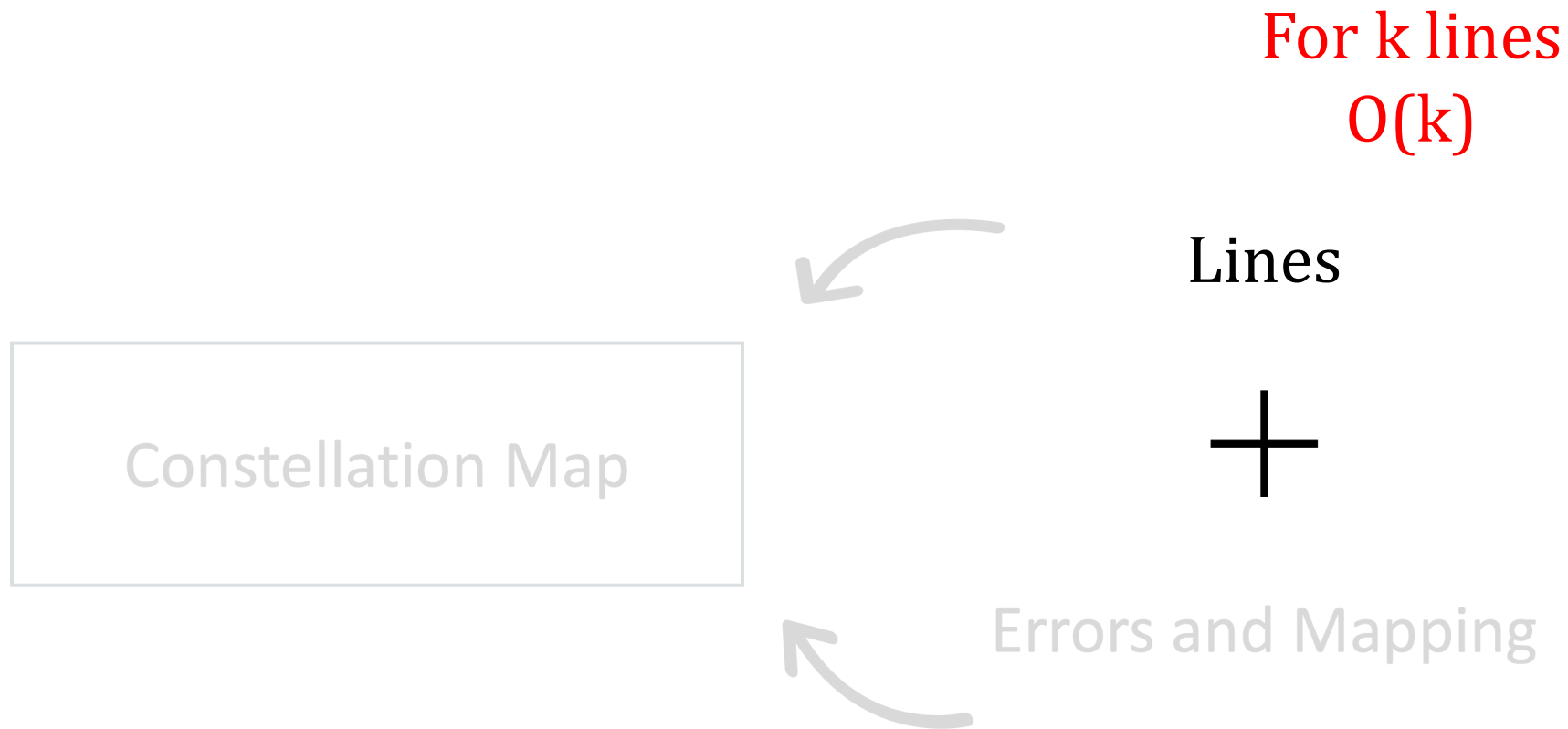


LR = 63 lines

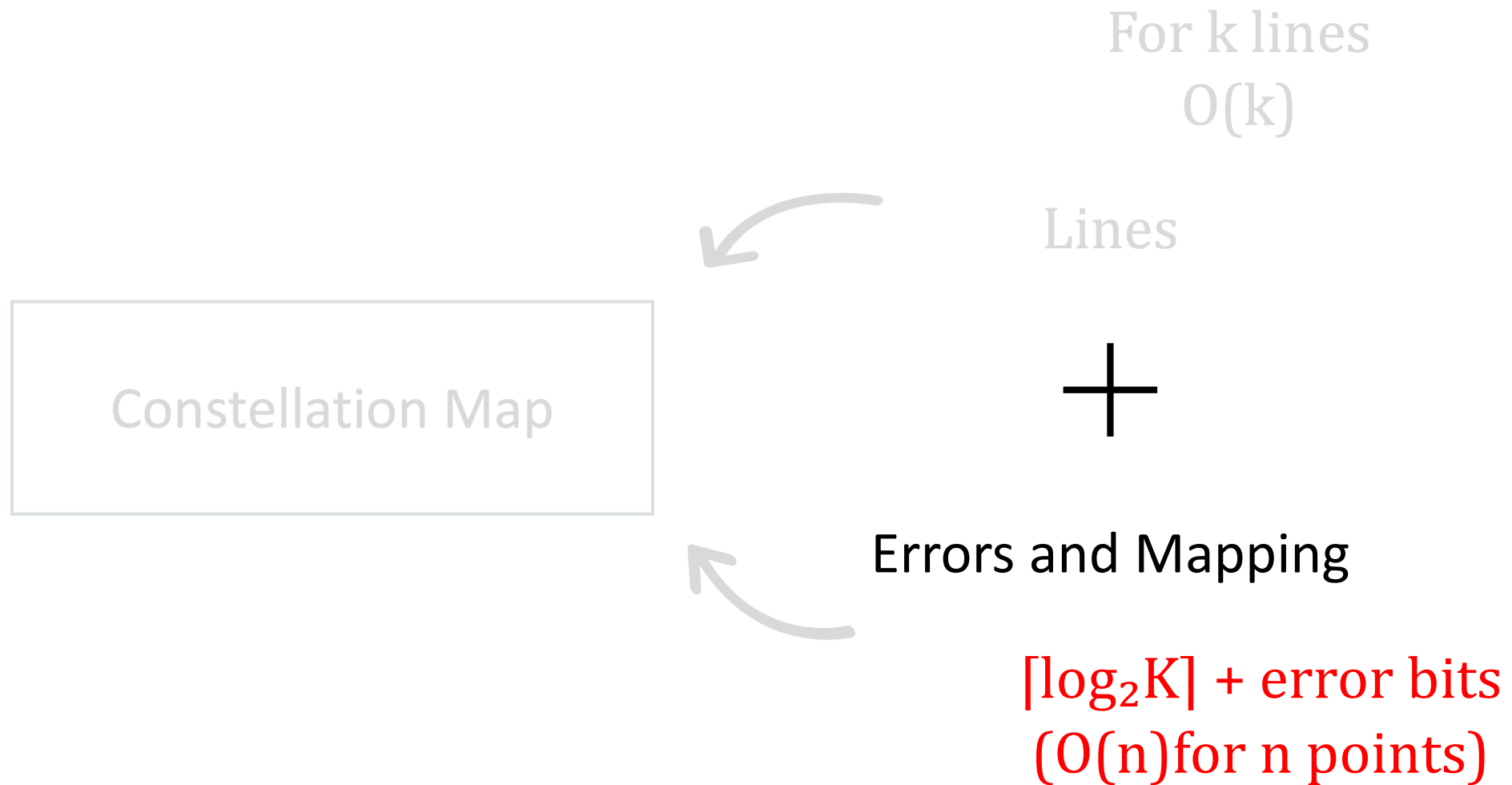
Optimizing Storage



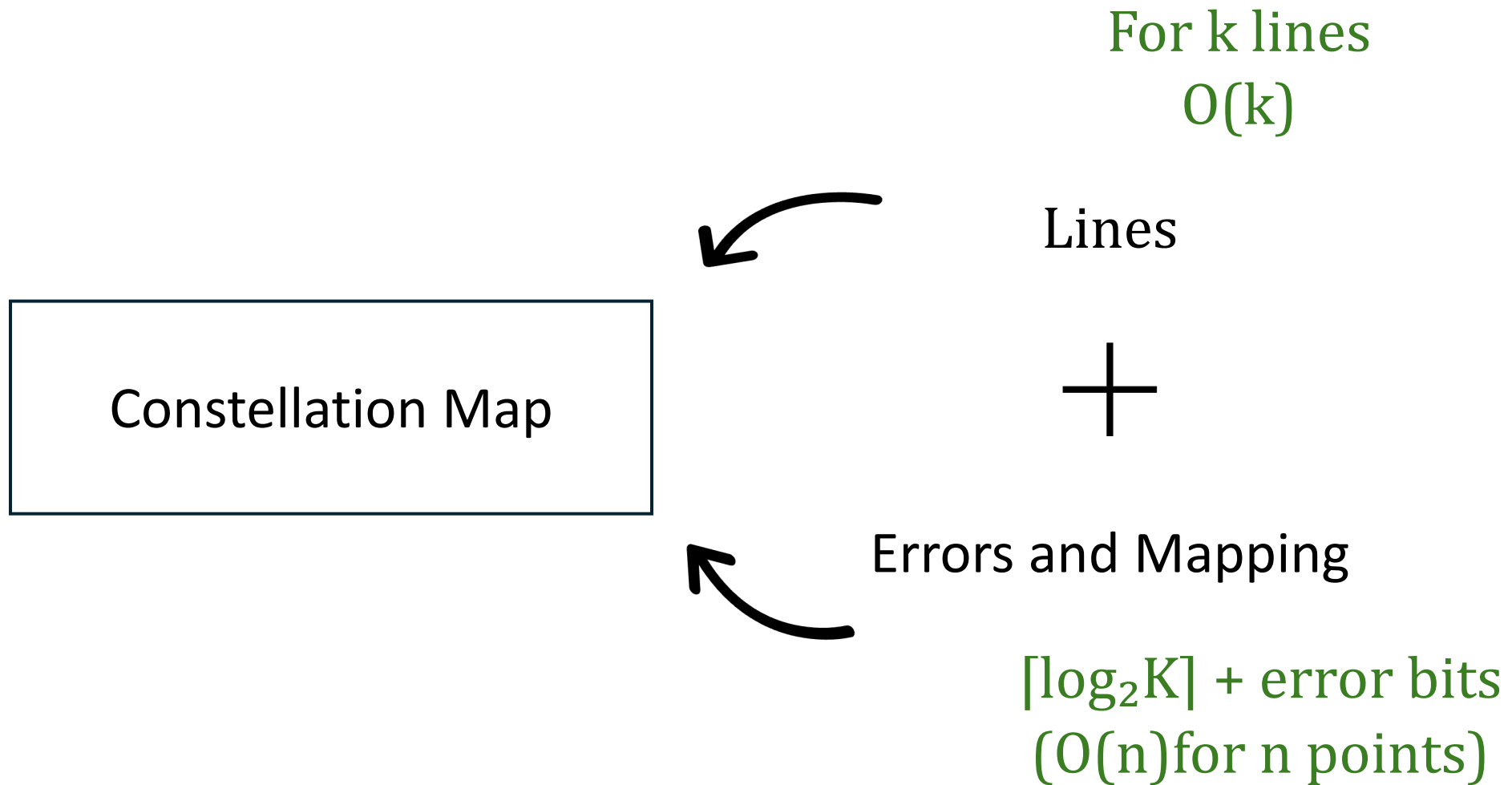
Optimizing Storage



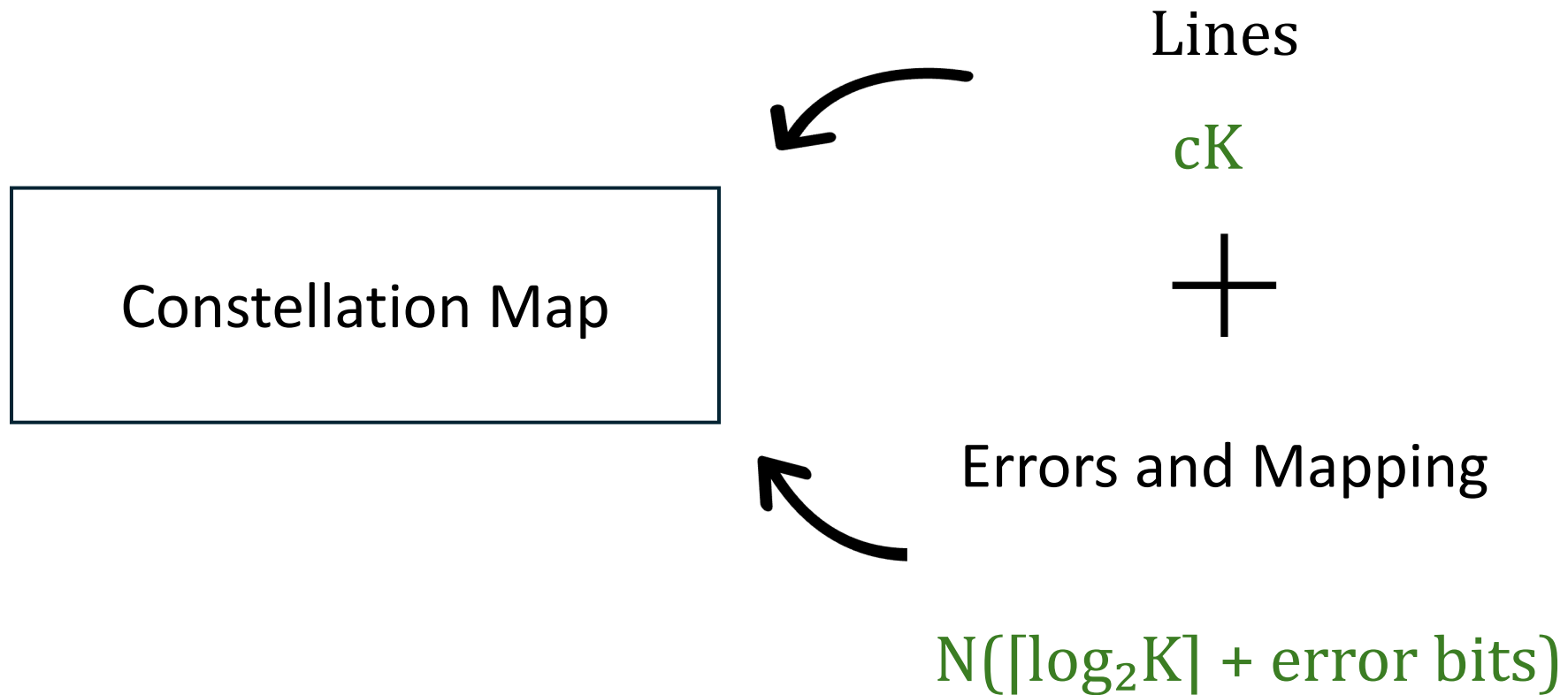
Optimizing Storage



Optimizing Storage



When does Constellation Map “Win”?



When does Constellation Map “Win”?

$$\underbrace{cK}_{\text{For lines}} + N[\log_2 K] + N.\textit{error} < N.b_{raw}$$

K = # of lines; c = constant bits for each line

When does Constellation Map “Win”?

$$cK + \underbrace{N[\log_2 K]}_{\text{mappings}} + N.\textit{error} < N.b_{\textit{raw}}$$

N = #of points; **K** = # of lines; **c** = constant bits for each line

When does Constellation Map “Win”?

$$cK + N[\log_2 K] + \underbrace{N.\text{error}}_{\text{error}} < N.b_{raw}$$

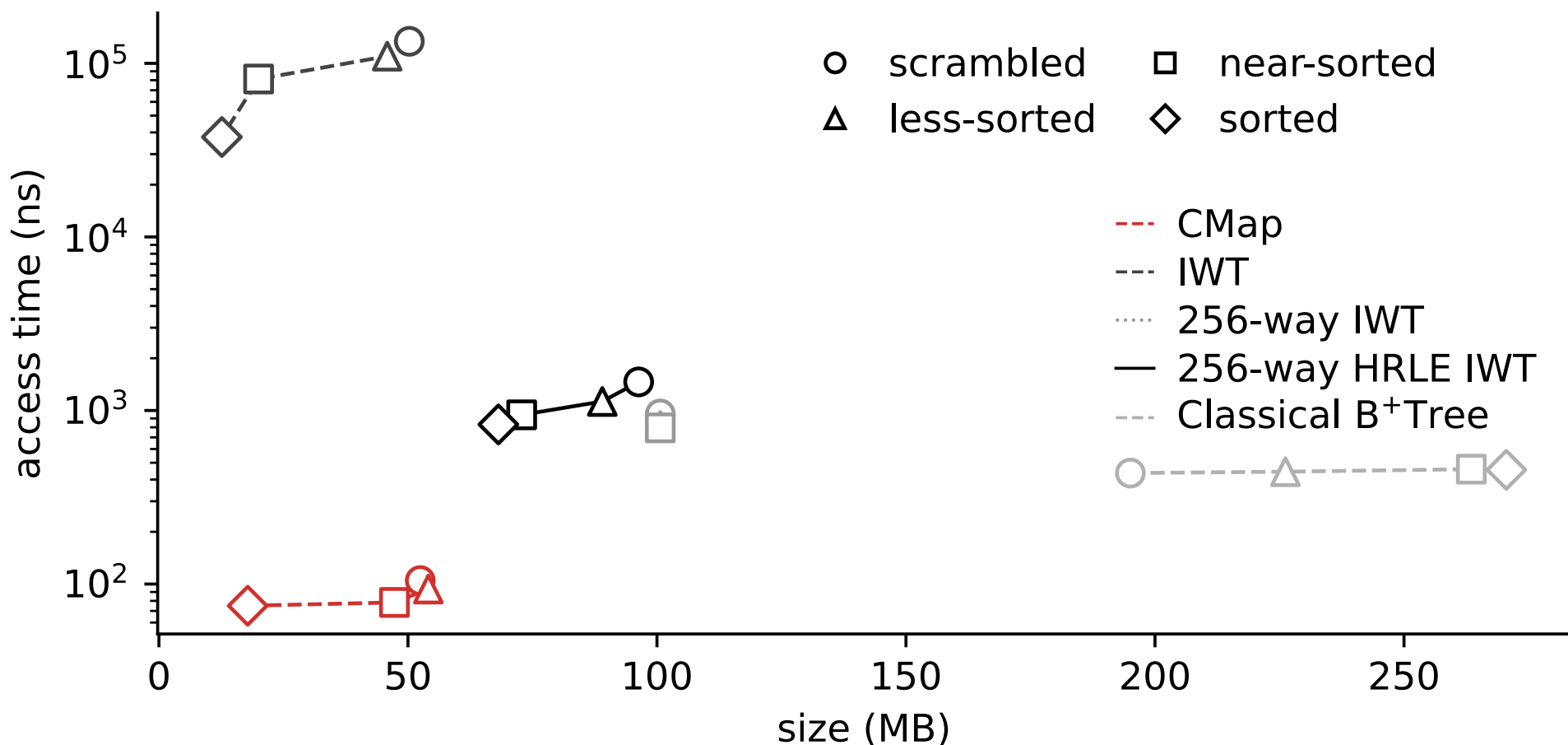
N = #of points; **K** = # of lines; **c** = constant bits for each line; **error** = error bits

When does Constellation Map “Win”?

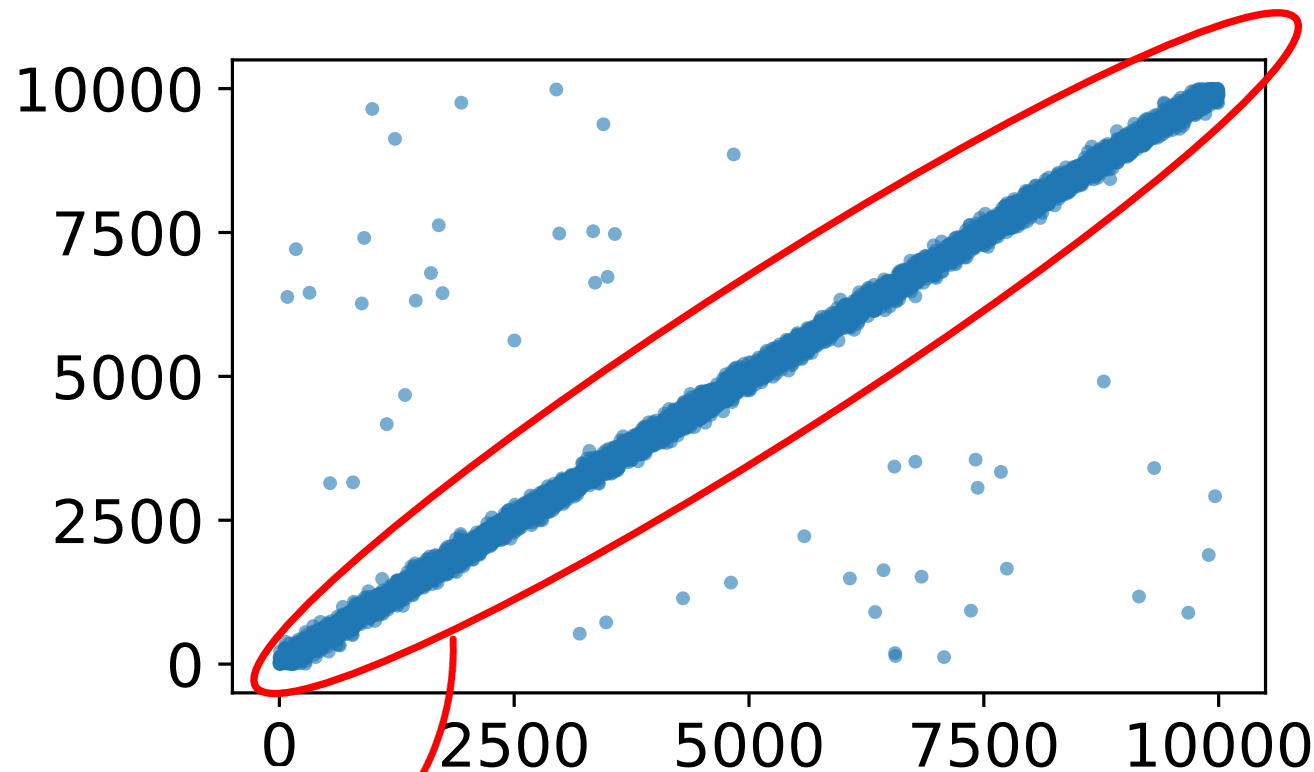
$$cK + N[\log_2 K] + N.\textit{error} < N.b_{raw}$$

N = #of points; **K** = # of lines; **c** = constant bits for each line; **error** = error bits; ***b_{raw}*** = raw bits per element

Initial Results with Naïve CMap

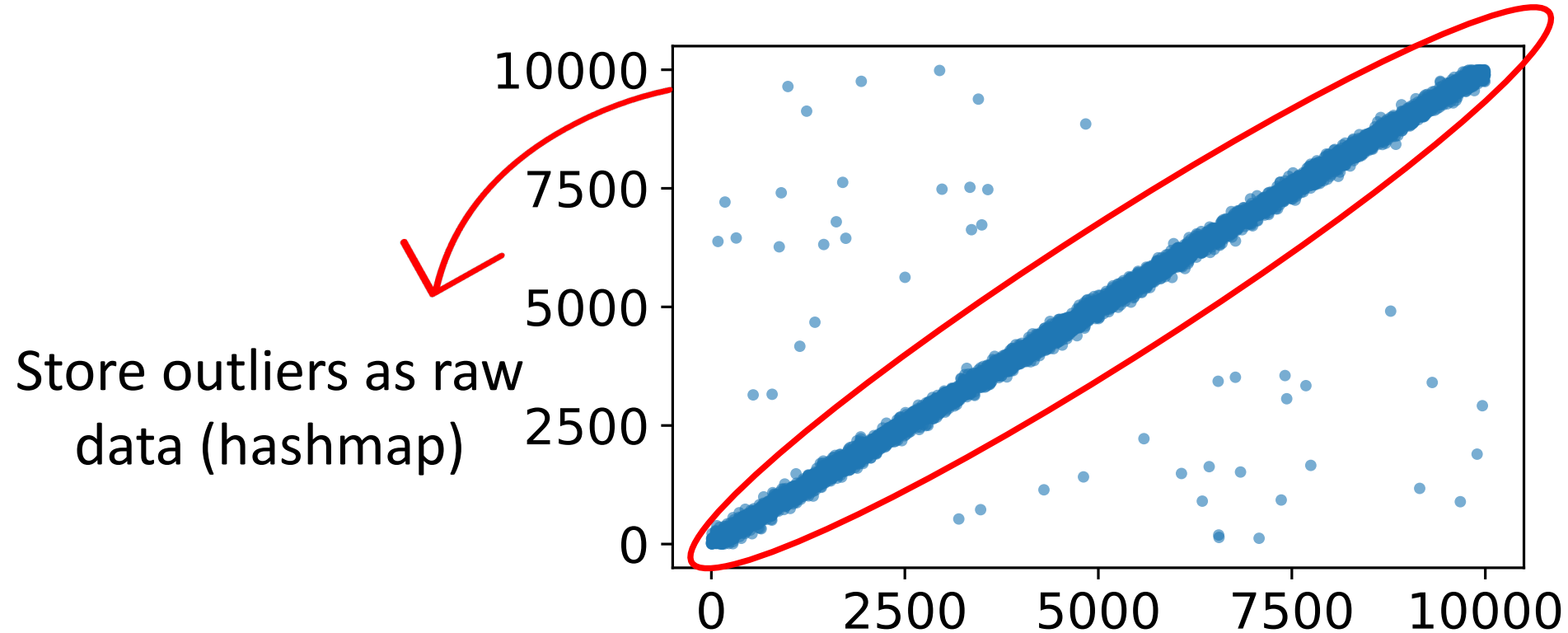


Hybrid algorithm with Backup



Only store lines when
advantageous!

Hybrid algorithm with Backup



What's next?

What's next?

Parallelize build

What's next?

Parallelize build

Implement updates through LSM-like delta merge strategy

What's next?

Parallelize build

Implement updates through LSM-like delta merge strategy

Balancing error and build algorithm



Summary



Geometric permutation mapping

Intrinsic compression capability

Space-Time tradeoff